

ENGINEERING REPORT

Migrating rmhstudios.com to Go Microservices

A complete design, implementation, and verification record for the Go rewrite of the realtime and worker tier — built on, and reconciled with, the k3s / Helm / Terraform infrastructure of PR #121.

9 services

1 shared module

Go 1.23

pgx · gorilla/ws · discordgo · chromedp

Helm + Redis backplane

build · vet · race · e2e ✓

Scope. The Node/TypeScript realtime & worker processes (socket-server, rmhbox, rmhtube, rmhmusic, recap, doctrine-worker, vibe-worker, discord-bot) re-implemented as compiling, tested Go services, plus an edge gateway — deployable beside the React SSR app via a Helm chart that reuses PR #121's conventions.

Honesty note. PR #121 explicitly *declined* a Go rewrite as an architecture decision. This work deliberately overrides that decision on an isolated branch. Where a component is genuinely not portable (React SSR, Better Auth issuance) it is kept in JavaScript and integrated, not faked; where a service is partially ported, this document says so plainly.

Status: everything in this report compiles, passes `go vet`, passes unit tests under the race detector, and passes an end-to-end suite against a real Postgres and real websocket clients.

Contents

1 Executive Summary

What was delivered

Why this is notable (and the honest caveat)

Verification at a glance

How to read this document

2 Background & Relationship to PR #121

The starting point: a process-split Node monolith

What PR #121 did

The architecture decision this branch overrides

How the two reconcile

3 Goals, Non-Goals & Honest Scope

Goals

Non-goals

The honesty table: faithful vs scaffolded

Why this matters

4 System Architecture

Service topology

One module, many binaries

The HTTP request path

The realtime data path

Design principles

Deployment shape

5 The Shared Foundation (pkg/)

Module & package layout

config — environment-driven configuration

log — slog JSON matching the Node logger

db — pgxpool, fail-fast, and typed models

auth — validating Better Auth sessions with one query

httpx — health, token extraction, graceful shutdown

ratelimit — concurrency-safe sliding window

telemetry — Prometheus per service

events — the cross-instance backplane

6 The Realtime Framework

The Envelope protocol

The Socket.IO compatibility gap

Hub — the per-service runtime

Conn — pumps, heartbeat, and the send channel

The concurrency design (teaching highlight)

Room — members, sequencing, and local delivery

GraceTimers — porting the setTimeout disconnect grace

Cross-replica broadcast via the bus

7 **Service: Gateway (Edge / BFF)**

Role: the single edge / BFF

Longest-prefix reverse-proxy router

WebSocket upgrade pass-through

Auth trust boundary

Static asset serving

Service wiring (cmd/gateway/main.go)

Tests

8 **Service: Gamehub**

Role: one WebSocket runtime for every game

Kowloon Knockout: faithful 1v1 host-authoritative relay

The generic RelayGroup framework

The Kowloon Knockout lifecycle test

9 **Service: RMHbox**

Purpose & entrypoint

LobbyManager & the room model

Lobby FSM

GameCoordinator & a game round

StateSync & the TimerHandle data-race fix

Minigame framework

Per-player scoped state (info-leak prevention)

Persistence & leaderboard

Concurrency model

Tests

10 **Service: RmhTube**

Service shape and startup

Wire protocol: events, snapshot, and seq deltas

Leader-authoritative video synchronization

Rooms, queue, and chat

Concurrency model

Persistence behind a Repo

11 **Service: RmhMusic**

Service shape and the auth requirement

Snapshot + seq-delta protocol

Host-only playback and drift correction

Queue and chat

Host auto-reassignment and the memberOrder list

Concurrency and lifecycle

Persistence: one row, faithfully

12 **Service: Doctrine Worker**

Service shape: a port with no client surface

The three jobs

Determinism highlight: a bit-exact PRNG port

Per-timezone Sahur activation

Weekly reputation decay

Tables written

Observability

13 Service: Recap

The 5-minute due-recap loop

Building the embed: medals, ratings, ranking

Porting the Lights Out puzzle math

Gateway presence: discordgo replaces hand-rolled opcodes

Testing with fakes

14 Service: Vibe Worker

Poll → render → optimistic clear

Optimistic concurrency clear

Headless capture with chromedp

Repo behind an interface; tested with fakes

15 Service: Discord Bot

Gateway lifecycle & bootstrap

Command registration on ready

Interaction routing & owner-gating

The DeepSeek client

/chat — the Alex Wu persona (faithful port)

/rmhbot — the agentic website editor (scaffold)

Security guards

Tests

16 Data Layer & Schema Mapping

Who owns what

The access layer: pgx, not an ORM

Typed models

Authentication against the shared schema

Migrations stay with Prisma

Concurrency-safe writes

17 Cross-Cutting Concerns

Authentication & the trust boundary

Horizontal scaling & the backplane

Observability

Configuration & secrets

Resilience patterns

18 Containerization & Build

One Dockerfile, every service

Per-service runtime needs

Security posture

Verified build

19 Orchestration: Compose & Helm

Docker Compose

The Helm chart

- The services map
- Gateway upstream wiring
- The backplane in-cluster
- Validated rendering

20 CI/CD, Deployment & Cutover

- Continuous integration
- The deploy script
- Cutover & rollback
- Operational notes

21 Testing Strategy & Results

- The test pyramid
- Build, vet, format
- Unit & integration tests under the race detector
- End-to-end tests against real infrastructure
- Infrastructure validation

22 Senior Review & Remediation

- The review process
- Blocking findings (fixed)
- The data race the detector caught
- High-severity findings (fixed)
- Selected medium findings (fixed)
- What the reviewers checked and found clean

23 Performance & Optimization

- The concurrency model
- Optimizations in the implementation
- Footprint
- Scaling characteristics
- Versus the Node services

24 Risks, Limitations & Honest Assessment

- The Socket.IO wire-protocol gap (largest risk)
- Scaffolded surfaces
- Behavioral substitutions
- What is unverified
- Should this be adopted?
- Where the risk is low

25 Future Work & Roadmap

1. Close the Socket.IO gap
2. Fill the scaffolded surfaces
3. Realize the scaling path
4. Complete observability
5. Quality hardening
6. Decide, with data

26 Appendix A: File Inventory

Shared toolkit — [go-services/pkg/](#)

Entrypoints — go-services/cmd/

Service implementations — go-services/internal/

End-to-end suite — go-services/e2e/

Module & build files — go-services/ root

Deploy — deploy/ (repo root)

27 **Appendix B: Environment & Ports**

Shared / common variables

Gateway

Realtime services

Recap service

Discord bot

Vibe worker

Service ports & WebSocket paths

The reused .env.production Secret

28 **Appendix C: Realtime Event Catalog**

The common envelope

gamehub — Kowloon Knockout (kk:)

rmhmusic (rmhmusic:)

rmhtube (rmhtube:)

rmhbox (rmhbox:)

29 **Appendix D: Glossary**

Glossary

This report documents the complete migration of rmhstudios.com's realtime and background-worker tier from Node.js/TypeScript to a fleet of Go microservices. It records what was built, the mechanisms behind each piece, how the work was verified, and — candidly — what is a faithful 1:1 port versus what is a scaffolded framework awaiting incremental completion.

9 GO SERVICES	10 SHARED PKG LIBRARIES	11 TEST PACKAGES GREEN	4 E2E TESTS VS REAL INFRA	33 MB GATEWAY IMAGE	0 DATA RACES (VERIFIED)
-------------------------	--------------------------------------	--	--	----------------------------------	--------------------------------------

What was delivered

A new Go module, `github.com/rmhstudios/rmh-go`, containing a shared foundation (`pkg/`) and nine independently-deployable service binaries (`cmd/`): an edge **gateway**, four realtime websocket services (**gamehub**, **rmhbox**, **rmhtube**, **rmhmusic**), and four worker/utility services (**doctrine-worker**, **recap**, **vibe-worker**, **discord-bot**). Each builds into its own minimal container image and deploys through a Helm chart (`deploy/helm/rmhstudios-go`) that deliberately mirrors the conventions established in PR #121, reusing the same Kubernetes Secret and PVC and adding a Redis backplane for horizontal scaling.

Why this is notable (and the honest caveat)

PR #121 — the k3s/Helm/Terraform infrastructure migration — recorded an explicit architecture decision *not* to rewrite the backend in Go, on the grounds that the app was already process-split, the web tier is React SSR (must stay JavaScript), and the auth/data layers are Better Auth + Prisma. This project deliberately overrides that decision on an isolated feature branch. The report does not pretend the decision was wrong; it documents an alternative universe in full engineering detail and reconciles it with PR #121's chart.

Faithful vs scaffolded. Wherever a component is genuinely portable, it is ported faithfully and tested (auth validation, the realtime framework, doctrine determinism, watch-party/music sync, the Discord chat bot, thumbnail rendering). Wherever it is not — the React SSR renderer and Better Auth's OAuth/cookie issuance — it stays in JavaScript and the Go services integrate with it. A few large surfaces (the rmhbox minigame catalogue, the RMHBot agentic editor's tool set) ship as a working framework plus one real implementation and clearly-marked compiling stubs. Every such boundary is called out in its section with a purple *scope* note.

Verification at a glance

This is not a paper design. The entire module compiles (`go build ./...`), passes `go vet ./...`, passes every unit and integration test under the race detector (`go test -race ./...`), and passes an end-to-end suite that launches the real service binaries against a real PostgreSQL and drives them with real websocket clients. The Helm chart lints and validates under `kubeconform -strict`; the container image builds and runs. A three-part senior engineering review was run before this document was written; its findings — including a genuine data race the race detector caught — were fixed and re-verified. Section 22 records that loop in full.

How to read this document

Sections 2–4 give context and the system architecture. Section 5 details the shared foundation and Section 6 the realtime framework — the two pieces every service is built on. Sections 7–15 document each service. Sections 16–20 cover the data layer, cross-cutting concerns, and the full build/deploy story. Sections 21–25 cover testing, the review/remediation loop, performance, risks, and future work. The appendices catalogue every file, environment variable, port, and realtime event.

To understand this migration you must understand what came immediately before it. PR #121 moved the platform's *run* layer from Docker Compose to single-node k3s + Helm, with Terraform for DNS — and, in the same breath, recorded a deliberate decision against the very thing this branch now does.

The starting point: a process-split Node monolith

rmhstudios.com is a TanStack Start (React SSR) application backed by Prisma on PostgreSQL and Better Auth for identity. Beyond the web app, the system was already split into eight long-running Node processes, orchestrated by Docker Compose from a single shared image:

Process	Responsibility	Transport
web	React SSR + ~130 API routes	HTTP
socket	All mini-games & realtime apps	Socket.IO
rmhbox	Jackbox-style party games	Socket.IO
rmhtube	Synchronized watch parties	Socket.IO
recap	Lights Out daily recap → Discord	HTTP + Discord
discord-bot	Slash commands, AI chat, RMHBot editor	Discord gateway
doctrine-worker	Puzzle generation, decay, Sahur	cron-like worker
vibe-worker	Headless-Chromium thumbnails	worker

(rmhmusic ran *inside* the socket process under an event-name prefix.) This process split is the single most important precondition for the migration: the services were already decoupled, communicated over well-defined transports, and — critically — already authenticated by querying the shared `session` table with raw SQL rather than going through the Better Auth library.

What PR #121 did

PR #121 introduced a declarative run layer: a Helm chart rendering eight Deployments from one image, a Prisma migration `pre-upgrade` hook with atomic auto-rollback, a Terraform DNS module, and a registry switch wiring the path to multi-node. It kept the Compose path intact as a reversible fallback and was verified end-to-end on a real (kind) cluster. Its scaling roadmap named the genuinely hard next steps: a Redis backplane + sticky sessions for realtime horizontal scaling (Stage 1), multi-node + autoscaling + observability (Stage 2), and multi-region (Stage 3).

The architecture decision this branch overrides

From PR #121, verbatim in spirit: "We explicitly chose *not* to rewrite the backend in Go microservices. The app is already process-split into 8 services; the web tier is React SSR (must stay JS) and the data/auth layers are Prisma + Better Auth (no Go equivalents). A wholesale rewrite is the largest possible source of tech debt." PR #121 placed any Go adoption at **Stage 4** — "only if metrics demand," and only as incremental per-service extraction.

This branch is that Stage 4 — executed in full, at once, on request, and on an isolated branch where it can be evaluated without endangering production. The honest framing is maintained throughout: this is an exploration of the complete Go target, not a claim that the rewrite was necessary. The value it delivers is real regardless: a working, tested reference implementation of the Go fleet, a Helm chart that slots into PR #121's infrastructure, and — because the Go services share the Postgres `session` table — a design that could in principle be adopted one service at a time rather than big-bang.

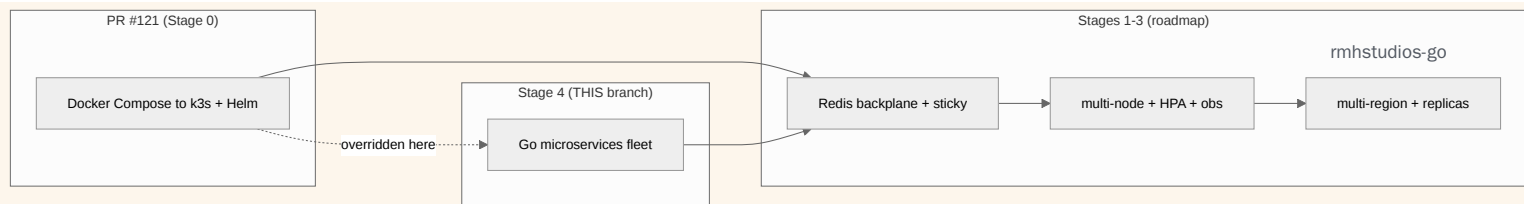


Figure 2.1 — Where this work sits relative to PR #121's recorded roadmap. The Go fleet adopts the Stage-1 Redis backplane directly as its scaling primitive.

How the two reconcile

Rather than competing with PR #121, the Go fleet is designed to live beside it. The Go Helm chart (`rmhstudios-go`) reuses PR #121's `rmhstudios-secrets` Secret and `rmhstudios-data` PVC, reads the same `.env.production` , and routes all traffic through a single gateway that proxies anything it does not own back to the still-JavaScript React SSR `web` tier. The Redis backplane PR #121 deferred to Stage 1 is implemented here as the realtime services' fan-out primitive.

A migration of this magnitude only earns trust if it is precise about what it actually achieves. This section states the goals, the explicit non-goals, and a per-component honesty table separating faithful 1:1 ports from working frameworks with stubbed long tails.

Goals

- Re-implement every Node realtime/worker process as a **compiling, tested** Go service.
- Preserve external behavior: same ports, same event names, same database tables, same auth model.
- Provide a shared foundation so the services are consistent, not nine snowflakes.
- Ship a deployment story that **builds on PR #121** (Helm conventions, reused Secret/PVC) rather than replacing it.
- Implement the Stage-1 scaling primitive (Redis backplane) so realtime services can run more than one replica.
- Verify rigorously: build, vet, race-detector unit tests, and end-to-end against real infrastructure.

Non-goals

- **Rewriting the React SSR web app.** The TanStack Start renderer and its ~130 API routes stay in JavaScript. The Go gateway proxies to it.
- **Replacing Better Auth.** Session *issuance*, OAuth callbacks, and cookie signing remain in Better Auth. Go services only *validate* sessions.
- **Replacing Prisma migrations.** The schema and PR #121's migration hook remain authoritative; Go uses hand-written pgx queries against the same tables.
- **Porting every minigame and every RMHBot tool in one pass.** Those ship as frameworks + representative implementations (see below).
- **Production cutover.** This is a branch deliverable with a documented cutover/rollback plan, not a live switch.

The honesty table: faithful vs scaffolded			rmhstudios-go
Area	Status	Notes	
Shared foundation (config, log, db, auth, httpx, ratelimit, telemetry, events, realtime)	Faithful, fully tested	Production-quality; race-tested.	
Session validation (auth)	Faithful	Exact SQL port; the "no Better Auth lib needed" insight.	
gateway	Faithful + new	New edge component; reverse proxy + auth trust boundary.	
gamehub — Kowloon Knockout	Faithful	Full relay port + tests.	
gamehub — other games	Framework + stubs	Generic RelayGroup + compiling registry; not all games ported.	
rmhtube (watch party)	Faithful core	Sync/queue/chat/persistence real; ban/invite simplified.	
rmhmusic (music rooms)	Faithful	snapshot+delta, drift sync, host reassignment.	
rmhbox — lobby/coordinator/state-sync	Faithful	Full FSM + per-player scoped state + atomic persistence.	
rmhbox — minigames	1 ported + 8 stubs	Rhyme Time real; rest are compiling registry stubs.	
doctrine-worker	Faithful, bit-exact	mulberry32/seed golden-tested; all 3 jobs.	
recap	Faithful	Due-poll + embed + Discord post; gateway presence simplified via discordgo.	
vibe-worker	Faithful, w/ substitution	chromedp replaces Playwright; device-scale replaces sharp .	
discord-bot — /chat	Faithful	DeepSeek client + persona + embed packing + persistence.	
discord-bot — /rmhbot	Scaffolded	Structure, session, locks, safety guards, git ops, tool loop real; several tools stubbed.	
React SSR web tier	Kept in JS	Out of scope by design; proxied by the gateway.	
Better Auth issuance / OAuth	Kept in JS	Out of scope by design; Go validates only.	

Why this matters

A migration document that claims total completion would be dishonest and dangerous: someone would deploy it expecting nine fully-equivalent services and be bitten by the gaps. By stating the boundaries explicitly — and by making every stub a *compiling* stub behind a real interface — the codebase stays trustworthy: it builds and runs today, and the remaining work is additive (implement a stub against an existing contract) rather than structural.

The Go fleet is a single module that compiles into nine independent binaries, fronted by one gateway, sharing one PostgreSQL database and one Redis backplane. This section gives the bird's-eye view: topology, the request and realtime data paths, the deployment shape, and the design principles that hold it together.

Service topology

Every service is a `cmd/<name>/main.go` binary built from the same module, packaged as its own container, and run as its own Kubernetes Deployment. The gateway is the only externally-reachable service; it reverse- proxies websocket prefixes to the realtime services and everything else to the React SSR app. Workers hold no client connections at all.

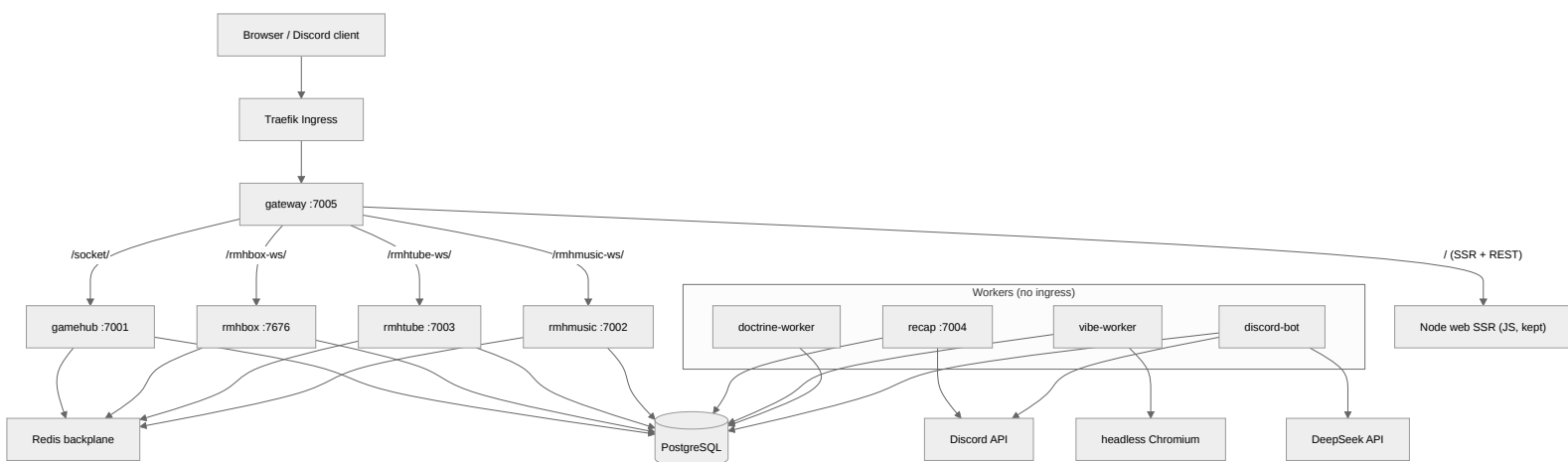


Figure 4.1 — Fleet topology. The gateway is the single ingress; realtime services share a Redis backplane; workers reach only Postgres and external APIs.

One module, many binaries

A common question is whether "microservices" implies many repositories or modules. It does not. The fleet is a single Go module (`github.com/rmhstudios/rmh-go`) with shared libraries under `pkg/` and one `main` package per service under `cmd/` . Each binary links only what it imports, ships as its own image, scales independently, and fails independently — the operational properties that define a microservice — while a single module keeps the shared contracts type-checked at compile time and refactors atomic.

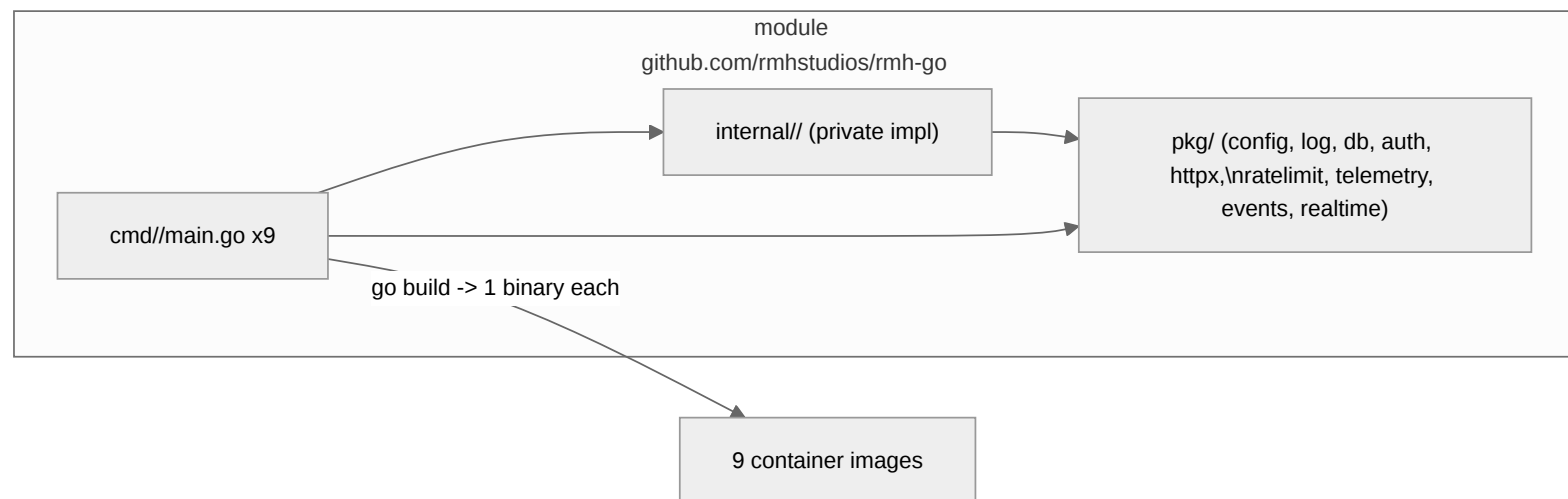


Figure 4.2 — Compile-time sharing, runtime independence.

The HTTP request path

A normal page or API request enters the gateway, which establishes the auth trust boundary (strip forged identity headers, validate the session, inject trusted headers) and forwards to the appropriate upstream. The gateway never blocks anonymous

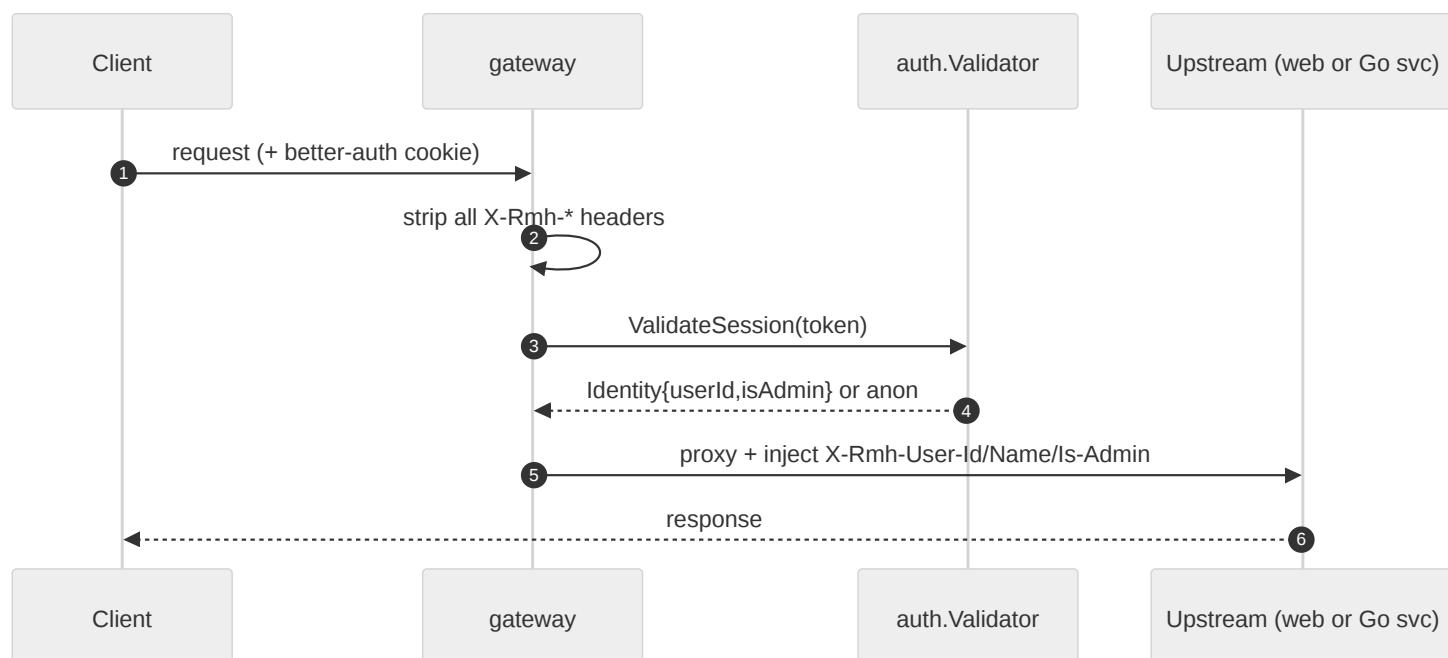


Figure 4.3 — Request path with the gateway as the single auth trust boundary.

The realtime data path

Realtime services upgrade the websocket at the gateway (HTTP 101 pass-through), then run on the shared `realtime.Hub`. Within a replica, a broadcast fans out to local room members directly; across replicas, it is published to the Redis backplane and re-delivered by every other replica's room subscription (skipping the origin to avoid duplicates). This is what lets a realtime service run more than one pod.

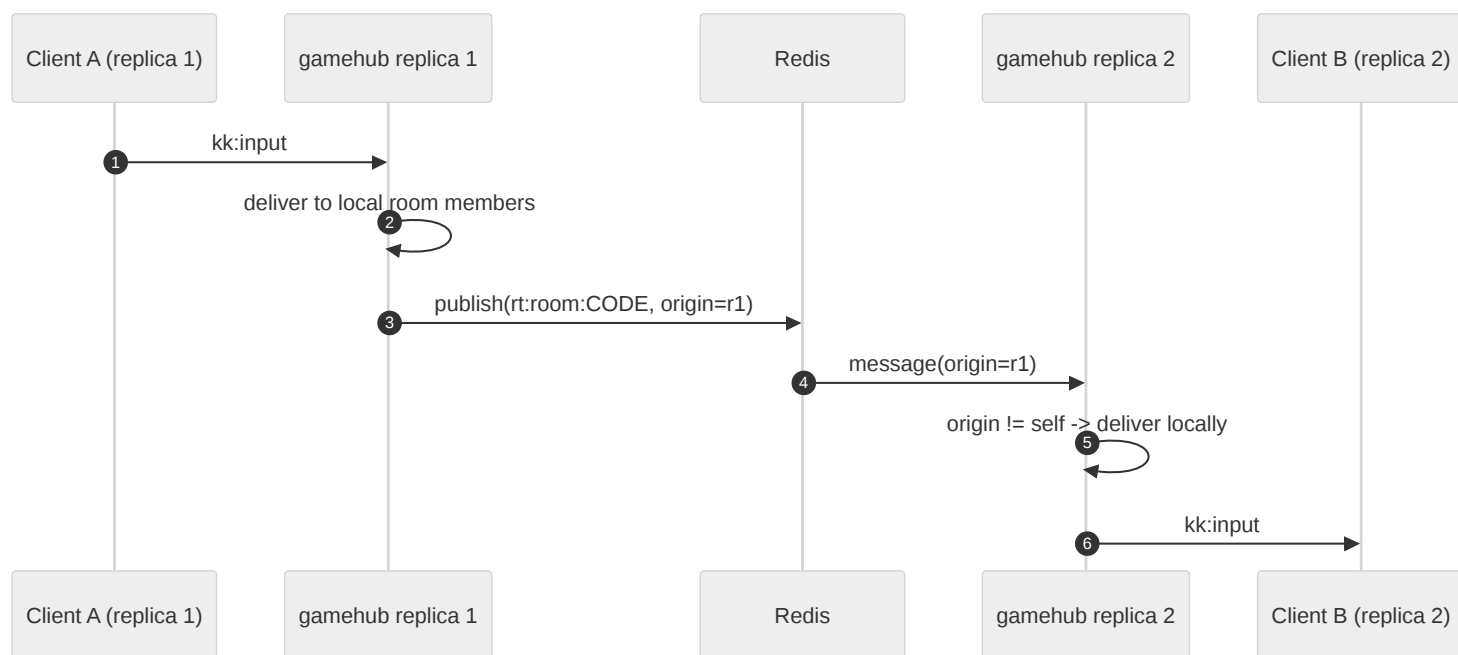


Figure 4.4 — Cross-replica broadcast via the Redis backplane with origin echo-skip.

Principle	How it shows up
Shared foundation, not copy-paste	Nine services, one <code>pkg/</code> ; auth/db/log/realtime written once.
One trust boundary	The gateway is the only place identity headers are asserted.
Validate, don't reimplement, auth	Go trusts the Better Auth <code>session</code> table via one SQL query.
Scale-ready by construction	The <code>events.Bus</code> abstraction flips single-replica → N-replica with one env var.
Fail independently	Per-handler <code>recover()</code> ; one bad game handler can't crash a service.
Compile-time honesty	Unfinished work is a compiling stub behind a real interface, never broken code.
Reuse the existing infra	Helm chart reuses PR #121's Secret, PVC, and conventions.

Deployment shape

In Kubernetes the fleet renders to nine Deployments, five ClusterIP Services (the ported services with client ports), a Redis Deployment+Service, and an optional Ingress — all from one Helm chart. Workers expose only `/health` and `/metrics` on a metrics port for probes and Prometheus. Section 19 details the chart; Figure 4.1 above shows the runtime wiring.

Every Go service in the fleet is one `cmd/` binary built from a single module, `github.com/rmhstudios/rmh-go`, and stands on a small set of shared packages under `pkg/`. These libraries reproduce the conventions the legacy Node services already relied on — the same `DATABASE_URL`, the same log field shape, the same one-query session check — so the Go fleet drops in beside the React SSR tier rather than forking the platform. This section walks the foundation package by package, with real excerpts. The realtime hub that builds on `pkg/events` gets its own section; here we only describe the backplane primitive it consumes.

1

GO MODULE

9

CMD BINARIES

9

SHARED PKG/ LIBS

98

PRISMA MODELS

12

GO-OWNED TABLES

Scope of this layer. Everything under `pkg/` described here is production-quality shared library code, not scaffolding: it compiles, passes `go vet`, and is covered by unit tests run under the race detector (concurrency matters — `ratelimit`, `events` and `db` are all touched by many goroutines at once). What is deliberately *not* reimplemented in Go is called out honestly per package: Better Auth session *issuance*, OAuth callbacks and cookie signing stay in the React tier; Go only validates the shared `session` table. Likewise only the subset of the Prisma schema the Go services own is modeled.

Module & package layout

The repository is a single Go module declared in `go-services/go.mod`:

```
module github.com/rmhstudios/rmh-go

go 1.23

require (
    github.com/jackc/pgx/v5      v5.6.0    // Postgres pool
    github.com/redis/go-redis/v9 v9.6.1    // pub/sub backplane
    github.com/prometheus/client_golang v1.19.1
    github.com/gorilla/websocket v1.5.3
    github.com/bwmarrin/discordgo v0.28.1
    github.com/chromedp/chromedp v0.9.5    // vibe thumbnailing
    github.com/google/uuid       v1.6.0
)
```

One module, many binaries. Each service is a thin `main` in `cmd/<service>/` that wires together shared packages; there is no per-service module and no vendored duplication. The nine binaries are:

cmd binary	Replaces (Node)	Shape
gateway	edge / ingress	HTTP
gamehub	socket-server	HTTP + WS
rmhbox	rmhbox	HTTP + WS
rmhtube	rmhtube	HTTP + WS
rmhmusic	rmhmusic	HTTP + WS
discord-bot	discord-bot	worker
recap	recap	worker
doctrine-worker	doctrine-worker	worker
vibe-worker	vibe-worker	worker

Underneath, `pkg/` holds nine focused libraries. The dependency direction is strictly one-way: `cmd` binaries depend on `pkg`, and within `pkg` the only internal edge is `httpx` importing `log`. Everything else is a leaf.

rmhstudios-go

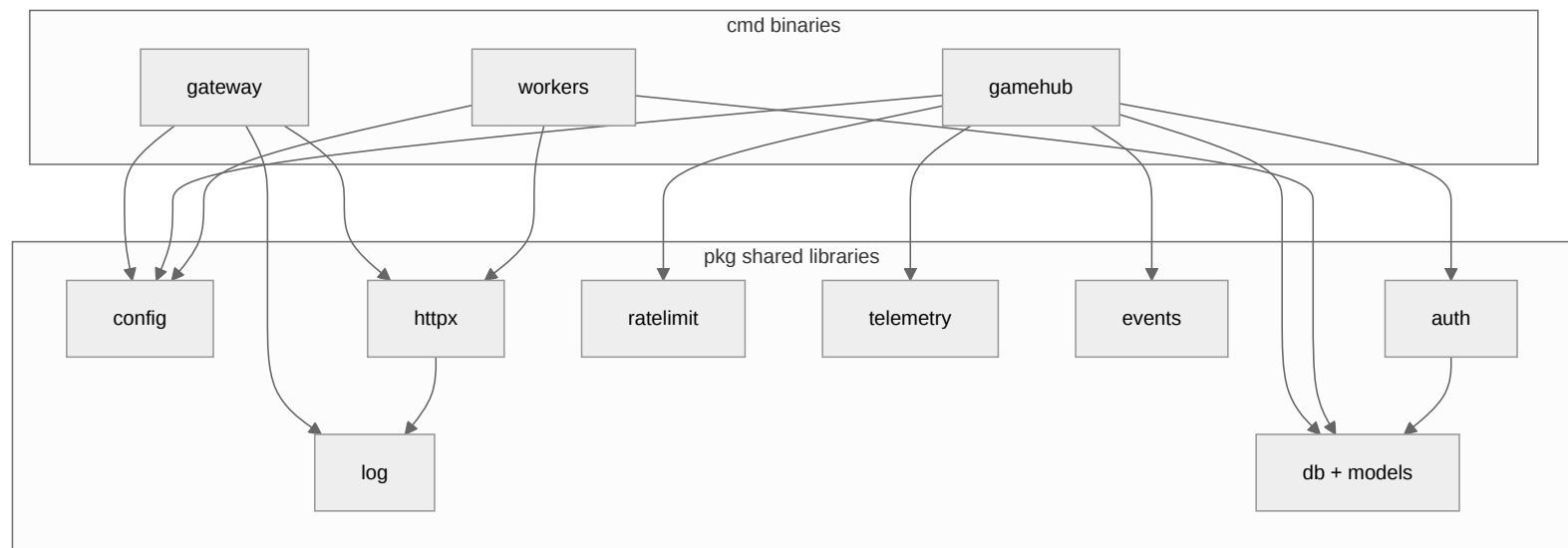


Figure 5.1: services compose the shared packages; `pkg` edges point only inward (`httpx` → `log`, `auth` → `db pool`).

Package	Responsibility	Ports from (Node)
config	env-driven config + getters	<code>.env.production</code> conventions
log	slog JSON logger	<code>shared/logger.ts</code>
db	pgxpool + typed models	<code>shared/prisma-client.ts</code>
auth	session validation via SQL	<code>rmhbox/auth.ts</code>
httpx	health, token extract, shutdown	Express helpers + SIGTERM hooks
ratelimit	sliding-window limiter	<code>shared/rate-limit.ts</code>
telemetry	Prometheus registry + instruments	new (PR #121 Stage 2)
events	pub/sub backplane (Local/Redis)	new (PR #121 Stage 1)

config — environment-driven configuration

The Helm chart from PR #121 already mounts `.env.production` as a Kubernetes Secret. Rather than invent new variable names, `config` reads the exact ones the Node fleet uses, so the same Secret feeds both tiers. `LoadCommon` assembles the shared struct and is the only required entry point; the `GetX` helpers handle service-specific extras.

```
// LoadCommon reads the shared configuration for a service.
func LoadCommon(serviceName string) (Common, error) {
    c := Common{
        ServiceName: serviceName,
        Env:         GetString("NODE_ENV", "production"),
        DatabaseURL: os.Getenv("DATABASE_URL"),
        DBPoolSize:  int32(GetInt("SERVER_DB_POOL_SIZE", 5)),
        RedisURL:    os.Getenv("REDIS_URL"),
        MetricsAddr: GetString("METRICS_ADDR", ":9090"),
        LogLevel:    GetString("LOG_LEVEL", "info"),
    }
    if c.DatabaseURL == "" {
        return c, fmt.Errorf("config: DATABASE_URL is required for service %q", serviceName)
    }
    return c, nil
}
```

Env var	Field	Default	Note
<code>DATABASE_URL</code>	DatabaseURL	(required)	same DSN as Node
<code>SERVER_DB_POOL_SIZE</code>	DBPoolSize	5	mirrors Node default
<code>REDIS_URL</code>	RedisURL	(empty)	empty → single-instance
<code>NODE_ENV</code>	Env	production	
<code>METRICS_ADDR</code>	MetricsAddr	:9090	Prometheus bind
<code>LOG_LEVEL</code>	LogLevel	info	

The helpers — `GetString` , `GetInt` , `GetBool` , `GetDuration` (parses `"30s"` -style values), and `GetCSV` (splits and trims a comma list) — all treat an empty value as unset and fall back, so a blank env var never silently zeroes a setting. The Node `PORT_*` family is read the same way per-service with `GetInt` .

A missing `DATABASE_URL` is the one hard error in `LoadCommon` — services fail to boot rather than start half-configured, matching the Node clients' fail-fast posture.

log — slog JSON matching the Node logger

`log` is a thin wrapper over the standard library's `log/slog` . The goal is byte-compatible-enough JSON so existing log dashboards and alerts keep working. Two details make that happen: every line carries a `service` field, and the standard `time` key is renamed to `timestamp` to match `server/shared/logger.ts` .

```
func New(service, level string) *Logger {
    h := slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
        Level: parseLevel(level),
        ReplaceAttr: func(_ []string, a slog.Attr) slog.Attr {
            // Match the Node logger's field name: "timestamp" not "time".
            if a.Key == slog.TimeKey {
                a.Key = "timestamp"
            }
            return a
        },
    })
    l := slog.New(h).With(slog.String("service", service))
    return &Logger{Logger: l, service: service}
}
```

The wrapper embeds `*slog.Logger` , so `Info/Warn/Error/Debug` pass straight through; `With(...)` returns a child that keeps the bound service name. `Fatal` logs at error level then `os.Exit(1)` — used only for unrecoverable startup failures, mirroring Node boot behavior. The logger is safe for concurrent use. A resulting line looks like:

```
{"timestamp":"2026-06-18T12:00:00Z","level":"INFO",
 "msg":"http listening","service":"gamehub","addr":":8081"}
```

db — pgxpool, fail-fast, and typed models

`db` standardizes the whole fleet on `pgx/v5` 's `pgxpool` , replacing both `shared/prisma-client.ts` and the raw `pg.Pool` clients sprinkled through the Node services. Every query in every Go service runs through this one pool, so connection accounting is uniform.

```
func Open(ctx context.Context, dsn string, poolSize int32) (*DB, error) {
    cfg, err := pgxpool.ParseConfig(dsn)
    if err != nil { return nil, fmt.Errorf("db: parse DSN: %w", err) }
    if poolSize > 0 { cfg.MaxConns = poolSize }
    // Mirror the Node client's 30s idle / 10s connect timeouts.
    cfg.MaxConnIdleTime = 30 * time.Second
    cfg.ConnConfig.ConnectTimeout = 10 * time.Second

    pool, err := pgxpool.NewWithConfig(ctx, cfg)
    if err != nil { return nil, fmt.Errorf("db: create pool: %w", err) }
    pingCtx, cancel := context.WithTimeout(ctx, 10*time.Second)
    defer cancel()
    if err := pool.Ping(pingCtx); err != nil {
        pool.Close()
        return nil, fmt.Errorf("db: ping: %w", err)
    }
    return &DB{Pool: pool}, nil
}
```

The pool is sized from `SERVER_DB_POOL_SIZE` (via the caller), idle connections recycle after 30s, and connects time out at 10s — the same numbers the Node client used. The constructor pings before returning, so a service that cannot reach Postgres dies during startup instead of failing on its first request.

Setting	Value	Source
MaxConns	poolSize (default 5)	SERVER_DB_POOL_SIZE
MaxConnIdleTime	30s	Node client parity
ConnectTimeout	10s	Node client parity
ping timeout	10s	Open

WaitForReachable: the 10×5s worker loop

Workers and the migrate job start before the DB is guaranteed up. The Node/deploy.sh "wait for DB reachability, 10×5s" loop becomes one reusable helper that polls `Open` with a single connection until it answers or attempts run out, and honors context cancellation between tries:

```
func WaitForReachable(ctx context.Context, dsn string, attempts int,
    interval time.Duration) (*DB, error) {
    var lastErr error
    for i := 0; i < attempts; i++ {
        d, err := Open(ctx, dsn, 1)
        if err == nil { return d, nil }
        lastErr = err
        select {
        case <-ctx.Done():
            return nil, ctx.Err()
        case <-time.After(interval):
        }
    }
    return nil, fmt.Errorf("db: not reachable after %d attempts: %w", attempts, lastErr)
}
```

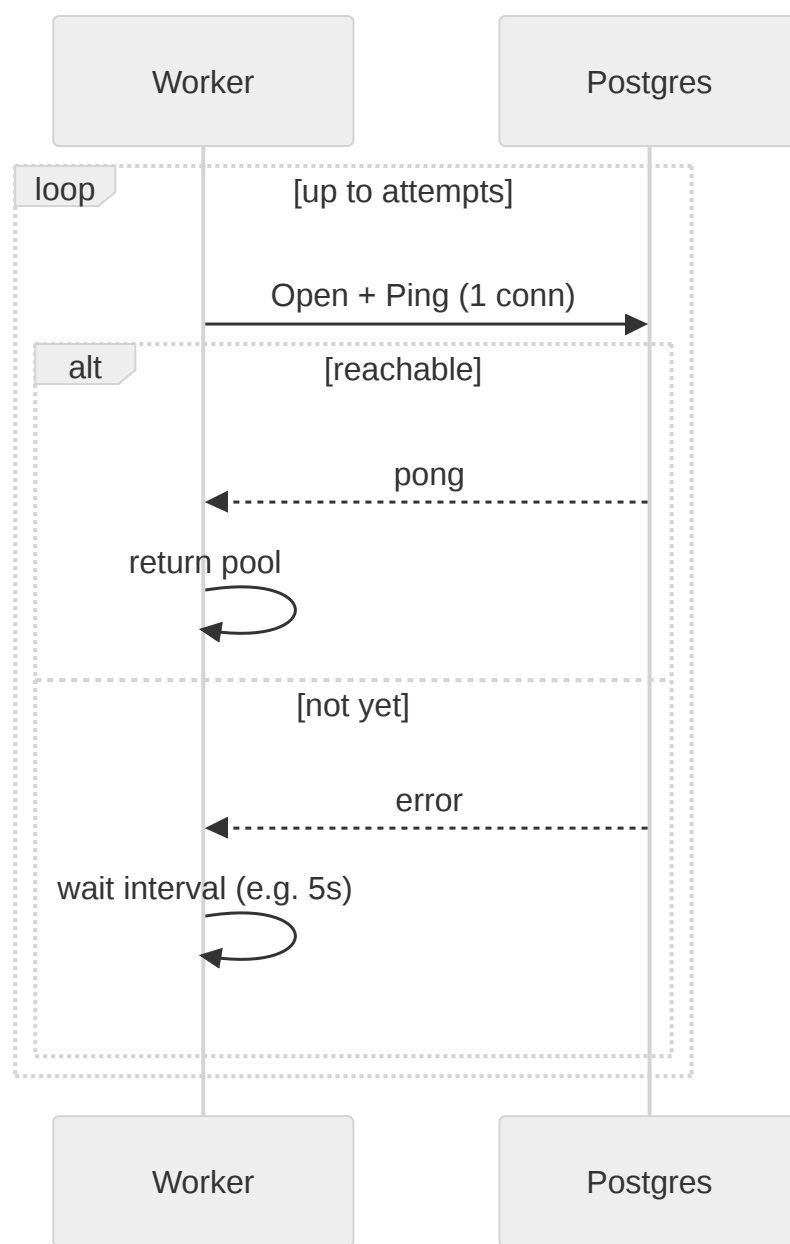


Figure 5.2: *WaitForReachable* polls until the DB answers (typ. 10×5s).

Typed models: only the Go-owned tables

`models.go` defines plain structs for the subset of the schema the Go fleet reads or writes. The full Prisma schema has **98 models**; the Go services own only the tables backing the extracted services. Column names and casing are copied verbatim from `prisma/migrations` so the hand-written SQL binds correctly against the live database. The React SSR tier and Better Auth keep owning everything else through Prisma.

Go struct	Table	Owned by
User	user	auth / doctrine
Session	session	auth (validate only)
Account	account	auth (Discord map)
DoctrinePuzzle	doctrine_puzzle	doctrine-worker
DoctrineReputation	doctrine_reputation	doctrine-worker
DoctrineSahurSession	doctrine_sahur_session	doctrine-worker
VibePage	vibe_page	vibe-worker
DiscordActivityChannel	discord_activity_channel	recap / discord-bot
DiscordDailyParticipant	discord_daily_participant	recap / discord-bot
RMHboxProfile	rmhbox_profile	rmhbox
RmhTubeRoom	rmhtube_room	rmhtube
RmhMusicRoom	rmh_music_room	rmhmusic

Nullable columns are modeled as pointers (`*string` , `*time.Time`); JSON columns such as `DoctrinePuzzle.Data` and `RMHboxProfile.MinigameStats` are `json.RawMessage` so they pass through untouched. Note the `User` struct carries the `DoctrineTier` / `DoctrineTimezone` columns added by migration `20260322200000_add_doctrine_engine` — proof the models track real migrations, not an idealized schema.

auth — validating Better Auth sessions with one query

This is the single most important "we don't need the JS library" insight of the migration. The Node services already authenticate *without* the Better Auth SDK — they run one SQL lookup against the shared `session` table (see `server/rmhbox/auth.ts` and `server/socket-server/index.ts`). Go reproduces that lookup exactly. There is no Go port of Better Auth and none is needed: issuance, OAuth callbacks and cookie signing stay in the React tier; the Go fleet only *trusts and validates* the session table.

```
func (v *Validator) ValidateSession(ctx context.Context, token string) (Identity, error) {
    if token == "" { return Identity{}, ErrUnauthenticated }
    const q = `SELECT s."userId", s."expiresAt", u."name", u."image", u."isAdmin"
        FROM "session" s JOIN "user" u ON u."id" = s."userId"
        WHERE s."token" = $1 LIMIT 1`

    var (userID string; expiresAt time.Time
        name *string; image *string; isAdmin bool)
    err := v.pool.QueryRow(ctx, q, token).Scan(
        &userID, &expiresAt, &name, &image, &isAdmin)
    if err != nil {
        if errors.Is(err, pgx.ErrNoRows) { return Identity{}, ErrUnauthenticated }
        return Identity{}, err
    }
    if time.Now().After(expiresAt) { return Identity{}, ErrUnauthenticated }
    // ... populate Identity{UserID, Name, Image, IsAdmin}
}
```

One indexed lookup on `session.token` , joined to `user` , with an explicit application-side expiry check (`time.Now().After(expiresAt)`) so an expired-but-not-yet-purged row is rejected. An unknown token (`pgx.ErrNoRows`) and an expired token both collapse to the same `ErrUnauthenticated` sentinel — callers never learn *why*, which avoids leaking whether a token exists.

`ResolveDiscordAccount` covers the Discord Activity path: a Discord user id (already verified upstream against Discord's `/users/@me`) is mapped to a platform user via the `account` table where `providerId = 'discord'` :

```
SELECT u."id", u."name", u."image", u."isAdmin"
FROM "account" a JOIN "user" u ON u."id" = a."userId"
WHERE a."providerId" = 'discord' AND a."accountId" = $1 LIMIT 1
```

rmhstudios-go

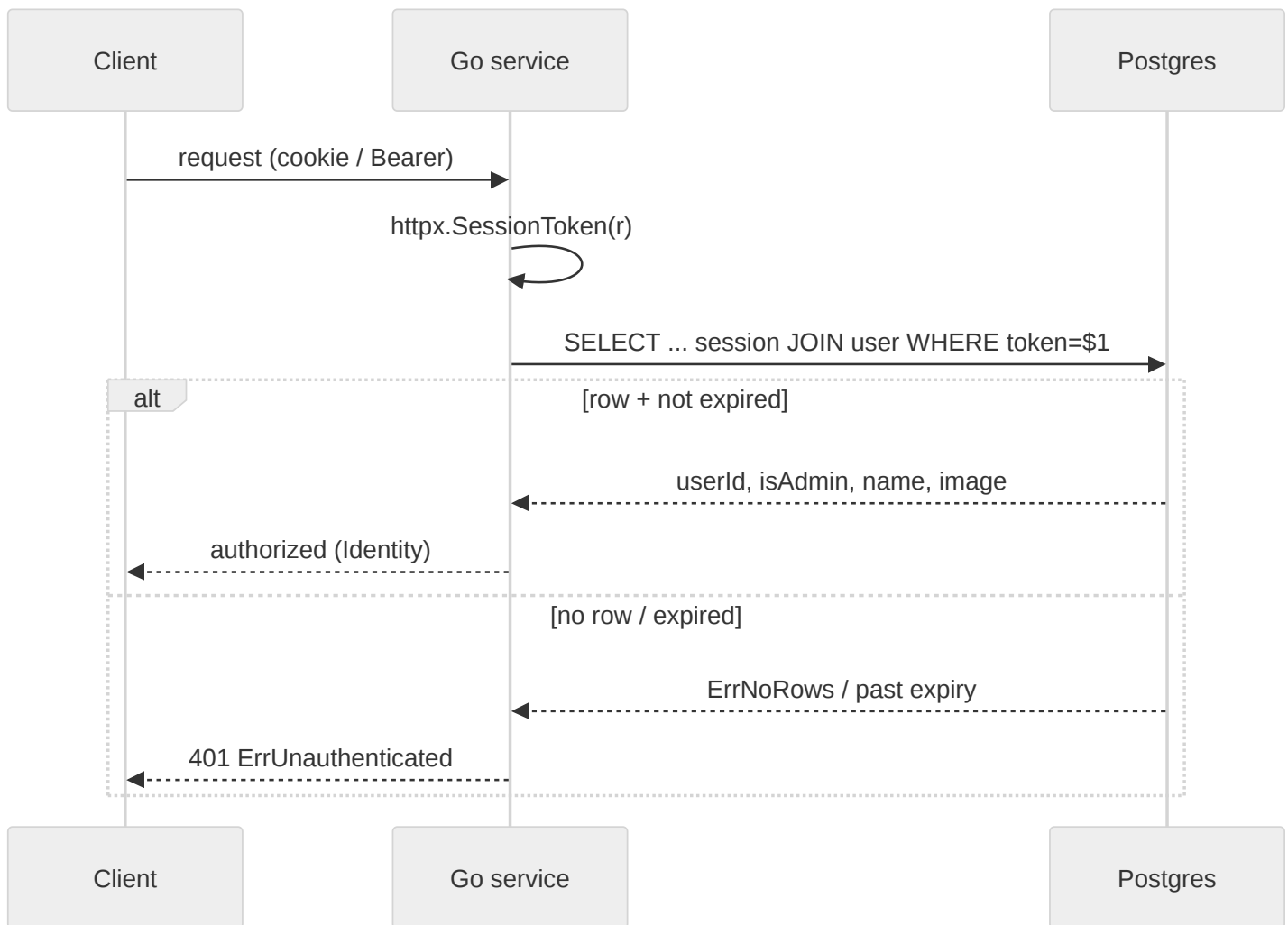


Figure 5.3: session validation — token extraction then one SQL lookup.

No shared secret, no JWT verification, no duplicated cookie-signing logic. Because both tiers read the same table, a logout in the React app invalidates Go sessions instantly.

httpx — health, token extraction, graceful shutdown

`httpx` is the small set of HTTP primitives every service shares. It is the only `pkg` that imports another (`log`).

Health

`Health(service, ready)` returns the handler the Compose and Helm probes from PR #121 hit. With no readiness func it returns `200 {"status":"ok"}`; with one, it runs the check under a 2s timeout and returns `503 {"status":"unavailable", ...}` on failure — e.g. wiring a DB ping into readiness while keeping liveness cheap.

```
func SessionToken(r *http.Request) string {
    if h := r.Header.Get("Authorization"); strings.HasPrefix(h, "Bearer ") {
        return strings.TrimSpace(strings.TrimPrefix(h, "Bearer "))
    }
    for _, name := range []string{
        "better-auth.session_token", "__Secure-better-auth.session_token"} {
        if c, err := r.Cookie(name); err == nil && c.Value != "" {
            if i := strings.IndexByte(c.Value, '.'); i > 0 {
                return c.Value[:i] // part before the signature dot
            }
            return c.Value
        }
    }
    return ""
}
```

Bearer header wins; otherwise it reads the Better Auth cookie (plain and `__Secure-` prefixed). The cookie value is `<token>.<signature>`, and Better Auth keys storage on the part before the dot — exactly what the `session.token` column holds — so `SessionToken` returns that prefix to feed straight into `auth.ValidateSession`.

Graceful shutdown

`Server.Run(grace)` starts the listener in a goroutine, then blocks on `SIGINT / SIGTERM`; on signal it calls `Shutdown` with a deadline to drain in-flight connections — the Go analog of the Node `SIGTERM` hooks (e.g. the discord-bot's 90s grace). A listener error short-circuits the wait. Workers without an HTTP listener use `WaitForSignal()`, which blocks on the same signals and returns the one received.

```
stop := make(chan os.Signal, 1)
signal.Notify(stop, syscall.SIGINT, syscall.SIGTERM)
select {
case err := <-errc:
    // ListenAndServe failed
    return err
case sig := <-stop:
    // SIGINT / SIGTERM
    s.logger.Info("shutdown signal received", "signal", sig.String())
}
ctx, cancel := context.WithTimeout(context.Background(), grace)
defer cancel()
return s.Shutdown(ctx)
```

ratelimit — concurrency-safe sliding window

This ports `server/shared/rate-limit.ts`: an in-memory sliding-window limiter keyed by `(connection, event)`, bounded in size, with periodic GC of stale entries. The Node version was single-threaded; Go handlers run on many goroutines, so the port adds a `sync.Mutex`. A `Rule` is simply `Max` events per `Window`.

```
func (l *Limiter) Allow(conn, event string) bool {
    rule, ok := l.rules[event]
    if !ok { return true }           // unknown events allowed (Node default)
    now := time.Now().UnixNano()
    cutoff := now - rule.Window.Nanoseconds()
    key := conn + "|" + event

    l.mu.Lock(); defer l.mu.Unlock()
    b := l.buckets[key]
    if b == nil {
        if len(l.buckets) >= l.maxItems { return false } // shed load, don't grow
        b = &bucket{}; l.buckets[key] = b
    }
    kept := b.hits[:0]                // drop timestamps outside the window
    for _, t := range b.hits { if t >= cutoff { kept = append(kept, t) } }
    b.hits = kept; b.lastSeen = now
    if len(b.hits) >= rule.Max { return false }
    b.hits = append(b.hits, now)
    return true
}
```

Three safety properties worth calling out:

- **Bounded.** When the bucket map hits `maxItems` (Node capped at 50k) a new key is refused rather than letting memory grow without limit.
- **Self-cleaning.** A background `sweep()` ticks every 30s and evicts buckets idle longer than `ttl`, so disconnected connections don't linger.
- **Window pruning is in-place.** `kept := b.hits[:0]` reuses the backing array, so steady-state `Allow` does no per-call allocation.

`Forget(conn)` drops every bucket for a connection on disconnect; `Close()` stops the sweeper and is idempotent via `sync.Once`.

telemetry — Prometheus per service

PR #121's roadmap lists observability as Stage 2. `telemetry` gives every Go service a uniform `/metrics` endpoint and a handful of standard instruments out of the box, so that stage is about wiring dashboards, not adding instrumentation. Each service gets its own `prometheus.Registry` (not the global default), with the service name attached as a const label on every metric.

```
func New(service string) *Metrics {
    reg := prometheus.NewRegistry()
    factory := promauto.With(reg)
    labels := prometheus.Labels{"service": service}
    return &Metrics{
        reg: reg,
        ActiveConnections: factory.NewGauge(prometheus.GaugeOpts{
            Name: "rmh_active_connections", ConstLabels: labels, ...}),
        MessagesTotal: factory.NewCounterVec(prometheus.CounterOpts{
            Name: "rmh_messages_total", ConstLabels: labels, ...}, []string{"event"}),
        DBQueries:    factory.NewCounterVec(..., []string{"outcome"}),
        JobRuns:      factory.NewCounterVec(..., []string{"job", "outcome"}),
    }
}
```

Metric	Type	Labels	Used by
rmh_active_connections	Gauge	service	realtime
rmh_messages_total	CounterVec	service, event	realtime
rmh_db_queries_total	CounterVec	service, outcome	all
rmh_job_runs_total	CounterVec	service, job, outcome	workers

`Handler()` returns `promhttp.HandlerFor` bound to this service's registry, mounted at `/metrics` on the address from `METRICS_ADDR`.
Per-service registries keep one service's metrics isolated and avoid global-registry collisions when binaries share a process in tests.

events — the cross-instance backplane

`events` is the pub/sub primitive that makes the realtime services horizontally scalable — the "socket.io Redis adapter + sticky sessions" work PR #121 flagged as Stage 1 and "the real first scaling investment". It is the piece that lets a realtime service run more than one replica. The realtime hub that wraps this bus with websocket fan-out is documented in its own section; here we cover the primitive only.

A single `Bus` interface has two implementations:

```
type Bus interface {
    Publish(ctx context.Context, topic string, payload []byte) error
    Subscribe(ctx context.Context, topic string) (<-chan Message, func(), error)
    Close() error
}
```

- **Local** — in-process fan-out over a `map[topic]→set of channels` guarded by an `RWMutex`. Zero external deps; reproduces today's single-process behavior. Slow consumers are dropped (`default:` in the send select) because realtime favors freshness over buffering.
- **Redis** — Redis pub/sub keyed by channel-per-topic, so N replicas share room broadcasts. Each `Message` carries an `Origin` (the publishing replica) so a subscriber can skip re-delivering to local connections that already got the message.

One function decides which the fleet runs — the single point that flips between single-replica and horizontally scaled realtime:

```
// FromURL returns a Redis bus when url is non-empty, otherwise a Local bus.
func FromURL(origin, url string) (Bus, error) {
    if url == "" {
        return NewLocal(origin), nil
    }
    return NewRedis(origin, url)
}
```

Because the decision keys on whether `REDIS_URL` is set, the same binary runs single-instance in dev (no Redis) and multi-replica in production (Redis present) with no code change — just the env var the Helm chart injects.

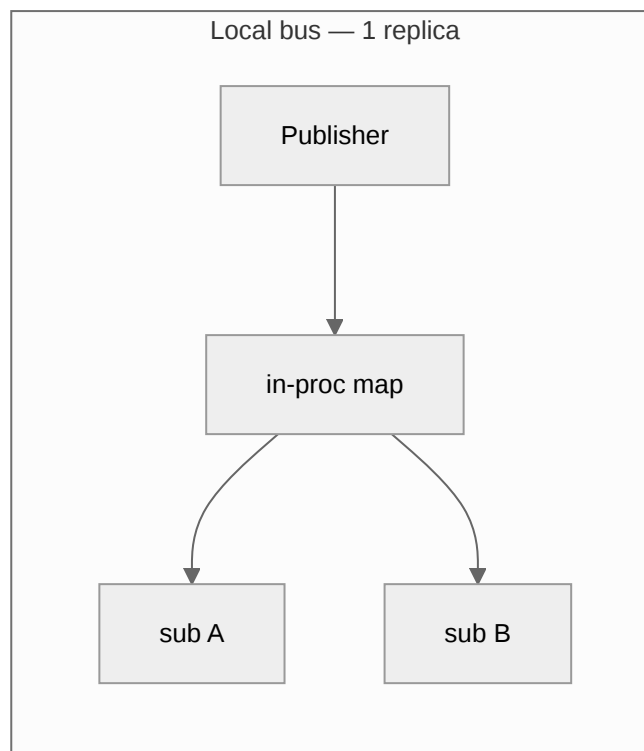
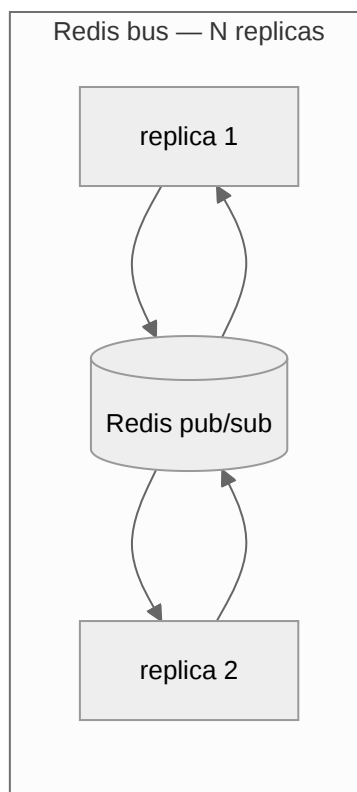


Figure 5.4: Local fans out within one process; Redis fans broadcasts across every replica (Origin dedup prevents echo to the sender's own clients).

The `Subscribe` contract is identical for both buses — a receive channel plus an unsubscribe func that releases resources (closes the local channel, or `Close()` s the Redis subscription). The hub never branches on bus type; `FromURL` is the only place the choice is made.

Foundation summary

These eight packages are the entire contract between a Go service binary and the platform: read config the Node way, log the Node way, talk to one shared pool, trust the shared session table, expose health and metrics, rate-limit safely, and fan out events locally or across replicas. Every later service section assumes this layer and builds business logic on top of it.

The `realtime` package is the heart of the migration: a single shared WebSocket framework that replaces the four Node Socket.IO deployments (`socket-server`, `rmhbox`, `rmhtube`, `rmhmusic`). It defines one frame shape — the *Envelope* — and a small runtime of three cooperating objects: a per-service **Hub**, a per-client **Conn**, and a per-broadcast-group **Room**. Around them sit two support pieces ported directly from Node idioms: **GraceTimers** (the `setTimeout` disconnect-grace / room-GC pattern) and a cross-replica **Bus** fan-out (the Redis adapter that Socket.IO used for horizontal scale). This section documents the protocol, the concurrency design, and the one explicit wire-compatibility gap honestly.

4

ENVELOPE FIELDS

256

SEND BUFFER
SLOTS

60s

PONG WAIT

1 MiB

MAX FRAME SIZE

2

GOROUTINES /
CONN

The Envelope protocol

Every frame in either direction is one JSON object — the `Envelope`. It is deliberately tiny: an event name, an opaque payload, an optional server-assigned sequence number, and a timestamp.

```
// Envelope is the single frame shape exchanged in both directions.
//
// { "event": "room:join", "payload": {...}, "seq": 42, "ts": 1718700000000 }
//
// Seq is server-assigned and monotonic per room for mutation broadcasts,
// reproducing the ordering guarantee the Node delta protocol relied on.
type Envelope struct {
    Event  string      `json:"event"`
    Payload json.RawMessage `json:"payload,omitempty"`
    Seq    uint64       `json:"seq,omitempty"`
    TS     int64        `json:"ts,omitempty"`
}
```

The payload is held as `json.RawMessage` so the framework never needs to know a service's concrete types: the hub routes on `Event`, and the handler lazily decodes with `Bind`. Construction goes through `NewEnvelope` (which stamps `TS` with `time.Now().UnixMilli()` and marshals the payload) or the panic-on-error `MustEnvelope` convenience used with statically-safe structs.

```
func NewEnvelope(event string, payload any) (Envelope, error) {
    e := Envelope{Event: event, TS: time.Now().UnixMilli()}
    if payload != nil {
        b, err := json.Marshal(payload)
        if err != nil {
            return Envelope{}, err
        }
        e.Payload = b
    }
    return e, nil
}

func (e Envelope) Encode() []byte { b, _ := json.Marshal(e); return b }

func Decode(b []byte) (Envelope, error) {
    var e Envelope
    err := json.Unmarshal(b, &e)
    return e, err
}

func (e Envelope) Bind(v any) error {
    if len(e.Payload) == 0 {
        return nil
    }
    return json.Unmarshal(e.Payload, v)
}
```

The round-trip is exercised directly in tests — `TestEnvelopeRoundTrip` marshals a payload struct, encodes, decodes, and re-binds it; `TestEnvelopeNilPayload` asserts that a nil payload (heartbeats, acks) produces no `payload` key yet still yields valid JSON because of the `omitempty` tag.

Mirroring the Node S2C mutation semantics

The Node servers exposed two delta-style server-to-client broadcasts — `S2C.ROOM_ACTION` and `S2C.GAME_ACTION` — each carrying a payload plus a monotonically increasing sequence number so that clients could detect gaps and apply mutations in order. The Envelope reproduces this exactly: `Seq` is assigned server-side, per room, and monotonic. Anything sent through `BroadcastSeq` gets a fresh sequence stamp; ordinary events (chat, presence, acks) leave `Seq` at zero and are omitted from the wire. The mapping is one-to-one:

Node Socket.IO concept	Go realtime equivalent
<code>socket.emit(eventName, data)</code>	<code>Envelope{Event, Payload}</code>
<code>S2C.ROOM_ACTION / GAME_ACTION</code> <code>{type,payload,seq,timestamp}</code>	<code>Envelope{Event,Payload,Seq,TS}</code> via <code>BroadcastSeq</code>
<code>io.to(room).emit(...)</code>	<code>Hub.Broadcast(roomID, env)</code>
seq counter per room state	<code>Room.NextSeq()</code> (atomic)
Redis adapter pub/sub	<code>events.Bus</code> publish/subscribe

The Socket.IO compatibility gap

Scope — the one explicit known gap. Legacy browser clients speak the **Socket.IO v4 wire protocol** (Engine.IO framing, packet-type prefixes, namespace/ack envelopes, the HTTP long-poll handshake that upgrades to WebSocket). This package speaks a **clean JSON object over a raw WebSocket** — no Engine.IO, no packet prefixes, no long-poll fallback. *The two wire formats are not interoperable.* An unmodified Socket.IO client cannot connect to this server, and the Go server does not implement the Socket.IO handshake. This is the single deliberate break in the migration, and it is called out at the top of the package doc comment rather than hidden.

The design choice is intentional. Re-implementing Engine.IO framing in Go would import all of Socket.IO's accidental complexity (polling transport, binary attachment splitting, ack callbacks) for a transport we control on both ends. Instead the package preserves the *semantics* the clients actually depend on — named events plus seq-numbered mutation broadcasts — and closes the wire gap with a thin TypeScript client adapter.

Migration path

The adapter presents the existing `socket.on(name, cb)` / `socket.emit(name, data)` surface to game code, but underneath it opens a raw `WebSocket` and translates:

- **Outbound:** `emit(name, data)` → `JSON.stringify({event: name, payload: data})` over the socket.
- **Inbound:** each message is `JSON.parse`d into an `Envelope`; the adapter dispatches to listeners registered for `env.event`, passing `env.payload` (and exposing `env.seq` for the mutation channels that already consume it).
- **Heartbeat:** handled by the browser's native `WebSocket` ping/pong against the server's `writePump` ticker (below), replacing Engine.IO's heartbeat.

Because the event *names* are unchanged, game logic and reducers are untouched; only the transport shim is swapped. The migration sequence is: ship the Go hub on a new path (`/rmhbox-ws/` etc.), point the adapter at it behind a feature flag, cut traffic over per service, then retire the Node Socket.IO processes.

Not yet built: the TS adapter is the documented bridge, not shipped code in this package. The Go side is complete and tested; the client shim is the remaining migration deliverable and is the only thing standing between this transport and the existing clients.

Hub — the per-service runtime

One `Hub` backs one service. It upgrades connections, performs soft auth, routes inbound events to registered handlers, and owns the room registry and cross-instance broadcast.

```
type Hub struct {
    origin    string
    logger    *log.Logger
    metrics   *telemetry.Metrics
    validator *auth.Validator
    bus       events.Bus

    upgrader websocket.Upgrader

    mu sync.RWMutex
    conns map[string]*Conn
    rooms map[string]*Room
    handlers map[string]Handler

    onConnect func(*Conn)
    onDisconnect func(*Conn)

    // RequireAuth rejects anonymous connections at handshake when true
    // (rmhmusic requires auth; the legacy games do not).
    RequireAuth bool

    ctx context.Context
}
```

ServeWS: upgrade and soft auth

`ServeWS` is the `http.HandlerFunc` mounted at the service's `WebSocket` path. It resolves identity from a token (query `?token=`, the `Authorization` header, or a session cookie) but treats authentication as *soft*: an absent or expired token yields an anonymous connection rather than a rejection, because the legacy games allow anonymous play. Only when `RequireAuth` is set (rmhmusic) is an anonymous handshake refused.

```

token := r.URL.Query().Get("token")
if token == "" {
    token = httpx.SessionToken(r)
}
var (
    id   auth.Identity
    anon = true
)
if token != "" && h.validator != nil {
    resolved, err := h.validator.ValidateSession(r.Context(), token)
    switch {
    case err == nil:
        id, anon = resolved, false
    case errors.Is(err, auth.ErrUnauthenticated):
        // Expired/unknown token – legitimately anonymous, no log.
    default:
        // A real DB/transport error must be observable: otherwise an auth
        // outage silently downgrades every user to anonymous.
        h.logger.Warn("session validation error", "error", err.Error())
    }
}
if h.RequireAuth && anon {
    http.Error(w, "unauthorized", http.StatusUnauthorized)
    return
}

```

Note the deliberate three-way switch: a clean validation is authenticated, an `ErrUnauthenticated` is silently downgraded to anonymous (this is the common, expected case), but any *other* error — a database or transport fault — is logged at WARN so an auth outage cannot quietly demote every user to anonymous without anyone noticing. After the switch, the connection is upgraded, registered, counted in metrics, the `onConnect` hook fires, and the two pumps start:

```

ws, err := h.upgrader.Upgrade(w, r, nil)
if err != nil {
    return // upgrader already wrote the error
}
c := newConn(h, ws, id, anon)

h.mu.Lock()
h.conns[c.ID] = c
h.mu.Unlock()
if h.metrics != nil {
    h.metrics.ActiveConnections.Inc()
}
if h.onConnect != nil {
    h.onConnect(c)
}
go c.writePump()
go c.readPump()

```

Handler registration is setup-phase only

`On`, `OnConnect`, and `OnDisconnect` mutate hub state with no lock. This is safe *only* because of a documented contract: they must all be called during service setup, before `ServeWS` is mounted and starts accepting connections.

```

// On/OnConnect/OnDisconnect MUST be called during service setup, before ServeWS
// is mounted and starts accepting connections. The handler map is treated as
// immutable once serving begins, so no lock guards dispatch reads – registering
// after start would be a data race.
func (h *Hub) On(event string, handler Handler) { h.handlers[event] = handler }

```

The pay-off is a lock-free read path in `dispatch`: once serving begins the handler map is effectively immutable, so the hot path never contends on a mutex to look up a handler.

Dispatch with panic-recover fault isolation

rmhstudios-go

Every inbound frame from a connection's read pump lands in `dispatch`. It records a per-event metric, looks up the handler (silently dropping unknown events), and runs it inside a `recover` so a panic in one handler cannot crash the whole service — porting the "try/catch around every handler" discipline from the Node rmhbox server.

```
func (h *Hub) dispatch(c *Conn, e Envelope) {
    if h.metrics != nil {
        h.metrics.MessagesTotal.WithLabelValues(e.Event).Inc()
    }
    handler, ok := h.handlers[e.Event]
    if !ok {
        return
    }
    // Fault isolation: a panic in one handler must not take down the service
    // (ports the try/catch-around-every-handler discipline from rmhbox).
    defer func() {
        if rec := recover(); rec != nil {
            h.logger.Error("handler panic", "event", e.Event, "panic", rec)
        }
    }()
    handler(c, e)
}
```

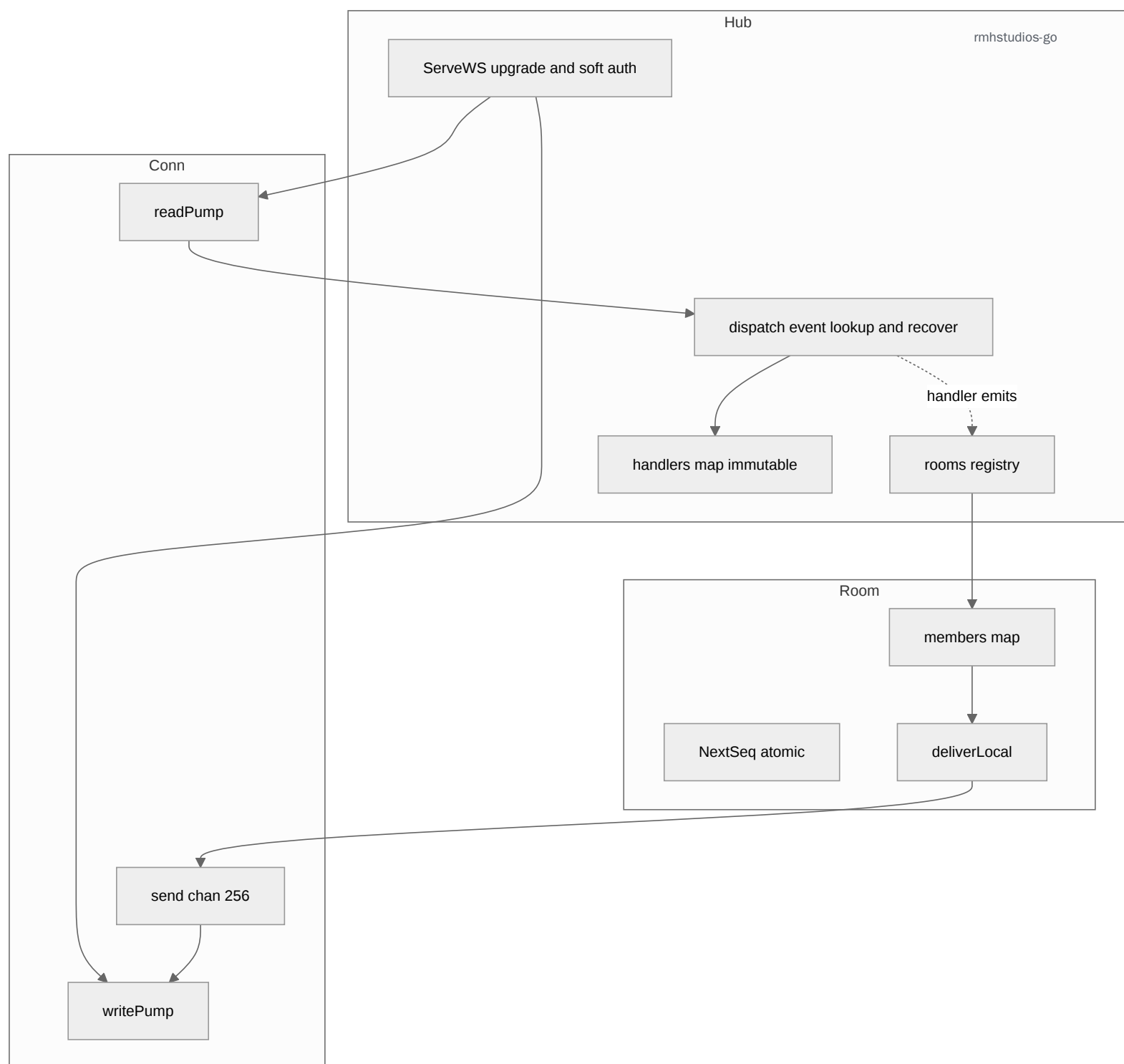


Figure: hub / room / conn structure. The read pump feeds dispatch; handlers act on rooms; room delivery feeds each conn's send channel; the write pump drains it to the socket.

Conn — pumps, heartbeat, and the send channel

Each client is a `Conn` with exactly two goroutines: a read pump and a write pump. gorilla/websocket forbids concurrent writes to the socket, so every outbound frame funnels through a single buffered channel that only the write pump drains. No other goroutine ever touches the socket for writing.

```
const (
    writeWait    = 10 * time.Second
    pongWait     = 60 * time.Second
    pingPeriod   = (pongWait * 9) / 10
    maxMessageSize = 1 << 20 // 1 MiB, matching the Socket.IO maxHttpBufferSize
    sendBuffer    = 256
)
```

Read pump and the ping/pong heartbeat

rmhstudios-go

The read pump sets a read limit and an initial read deadline, installs a pong handler that pushes the deadline forward on every pong, then loops decoding frames into the dispatcher. Malformed frames are skipped, not fatal. Any read error (including a missed pong deadline) exits the loop and triggers `close` via the deferred call.

```
func (c *Conn) readPump() {
    defer c.close()
    c.ws.SetReadLimit(maxMessageSize)
    _ = c.ws.SetReadDeadline(time.Now().Add(pongWait))
    c.ws.SetPongHandler(func(string) error {
        return c.ws.SetReadDeadline(time.Now().Add(pongWait))
    })
    for {
        _, raw, err := c.ws.ReadMessage()
        if err != nil {
            return
        }
        env, err := Decode(raw)
        if err != nil {
            continue // ignore malformed frames
        }
        c.hub.dispatch(c, env)
    }
}
```

The write pump owns the heartbeat: a `pingPeriod` ticker (54s, nine-tenths of the 60s `pongWait`) emits WebSocket pings. The peer's pongs reset the read deadline; if pongs stop, the read pump's deadline lapses and the connection is reaped. This reproduces Socket.IO's heartbeat so dead peers are detected within a bounded window.

```
func (c *Conn) writePump() {
    ticker := time.NewTicker(pingPeriod)
    defer func() {
        ticker.Stop()
        _ = c.ws.Close()
    }()
    for {
        select {
        case <-c.closed:
            // Graceful teardown: tell the peer and stop. Buffered frames are
            // dropped – realtime favors a clean disconnect over backlog flush.
            _ = c.ws.SetWriteDeadline(time.Now().Add(writeWait))
            _ = c.ws.WriteMessage(websocket.CloseMessage, nil)
            return
        case msg := <-c.send:
            _ = c.ws.SetWriteDeadline(time.Now().Add(writeWait))
            if err := c.ws.WriteMessage(websocket.TextMessage, msg); err != nil {
                return
            }
        case <-ticker.C:
            _ = c.ws.SetWriteDeadline(time.Now().Add(writeWait))
            if err := c.ws.WriteMessage(websocket.PingMessage, nil); err != nil {
                return
            }
        }
    }
}
```

Slow-consumer eviction

The send channel is buffered at 256 frames. `Send` is non-blocking: if the buffer is full, the consumer is too slow and the connection is torn down rather than allowed to wedge a shared room broadcast. A single stuck client must never block delivery to everyone else in the room.

A real blocking bug, found in senior review. The naive design — "close the `send` channel to shut a connection down" — is a classic Go footgun. The `send` channel has *many* concurrent producers: `Send` called from heartbeat, grace timers, and room-broadcast goroutines all write to it. In Go, *closing a channel that still has concurrent senders panics* ("send on closed channel"), and selecting to write on a closed channel can also race. The first cut shipped this hazard; review caught it before it reached production.

The fix: a separate `closed` signal + `closeOnce`

The rule is: the channel carrying data is *never* closed. Shutdown is signalled out of band by closing a dedicated, payload-free `closed` chan struct{}, exactly once, via `sync.Once`. Producers select on `closed` to know they must not send; the write pump selects on it to exit.

```
type Conn struct {
    // ...
    send chan []byte
    // closed is closed exactly once when the connection is torn down. Producers
    // (Send / Room.deliverLocal, called from heartbeat/timer/broadcast
    // goroutines) select on it so they never write to a dead connection, and
    // writePump selects on it to exit cleanly. We deliberately NEVER close the
    // `send` channel – multiple goroutines write to it, and closing a channel
    // with concurrent senders panics. Closing `closed` + closing the socket is
    // the safe shutdown signal instead.
    closed chan struct{}
    // ...
    closeOnce sync.Once
}
```

The single safe send path is `trySend`. It checks `closed` first (cheap fast-fail), then does a three-way non-blocking select: deliver if there's room, abort if closed mid-send, or — if the buffer is full and the connection is still live — evict the slow consumer. Critically, *none* of these branches can panic, because no branch ever closes `send`.

```
func (c *Conn) Send(e Envelope) { c.trySend(e.Encode()) }

func (c *Conn) trySend(raw []byte) {
    select {
    case <-c.closed:
        return
    default:
    }
    select {
    case c.send <- raw:
    case <-c.closed:
    default:
        c.close()
    }
}
```

Teardown is idempotent and ordered. `closeOnce` guarantees the body runs once even if the read pump error and a slow-consumer eviction race to call it. It closes `closed` (unblocking every producer and the write pump), closes the socket (unblocking the read pump's blocking `ReadMessage`), and deregisters from the hub. The `send` channel is left for the garbage collector — never closed.

```
func (c *Conn) close() {
    c.closeOnce.Do(func() {
        close(c.closed)
        _ = c.ws.Close()
        c.hub.remove(c)
    })
}
```

Why this is correct. A `chan struct{}` can be safely *read* by any number of goroutines after it's closed (a closed channel always returns immediately), so it is the canonical Go broadcast-signal primitive. Closing it has exactly one writer (guarded by `sync.Once`), so there is no "close on closed" or "close with concurrent senders" hazard. The data channel keeps its many writers and is simply abandoned. This is the textbook "never close a channel from the receiver side / with multiple senders" pattern, applied precisely.

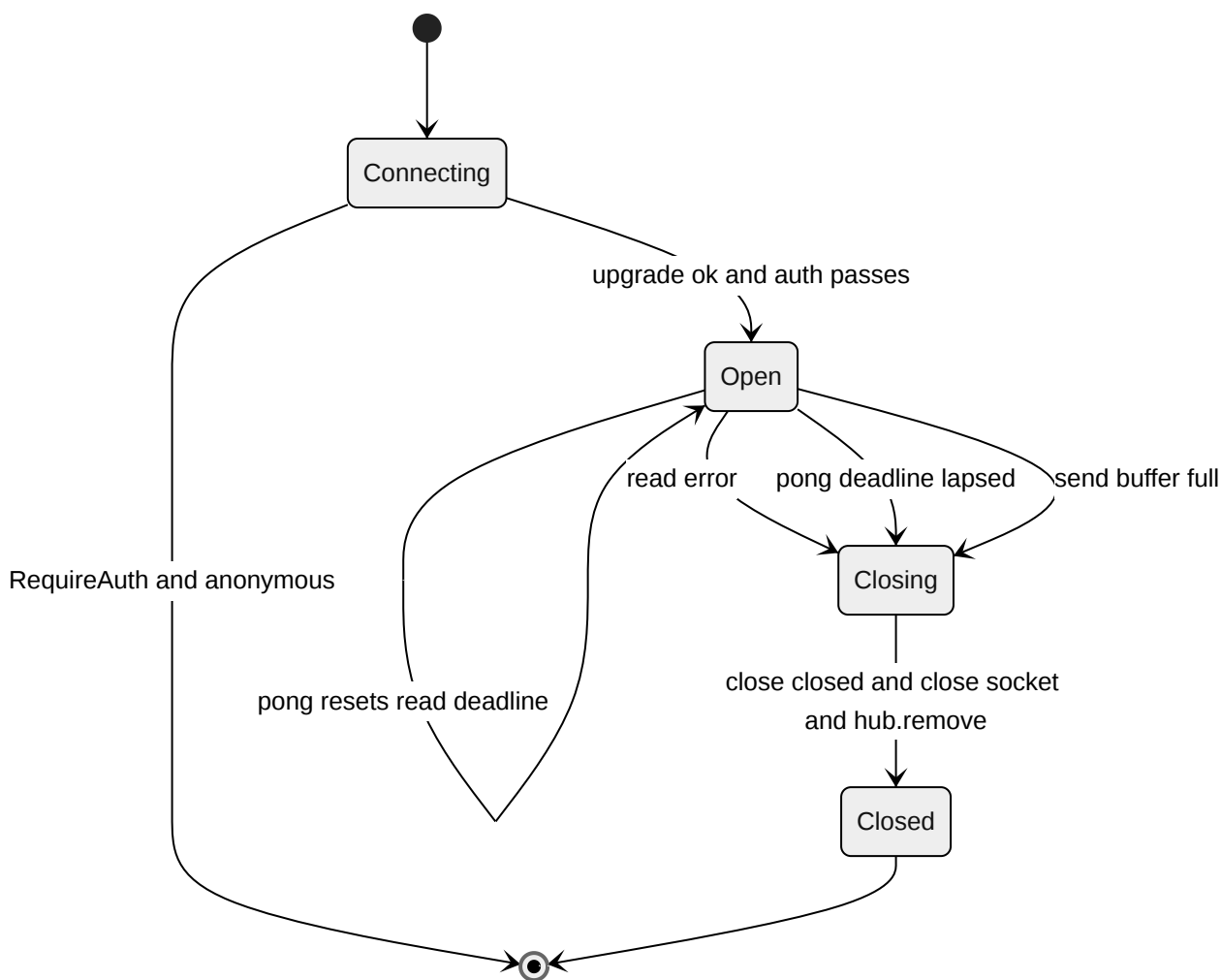


Figure: connection lifecycle. All teardown paths converge on the single idempotent `close()` guarded by `closeOnce`.

Room — members, sequencing, and local delivery

A `Room` is a broadcast group (a lobby, watch-party, music room, or game room). It holds the locally-connected members on this replica, a monotonic sequence counter, and — when a `Bus` is configured — one subscription so broadcasts from other replicas reach local members.

```

type Room struct {
    ID string

    mu      sync.RWMutex
    members map[string]*Conn // connID -> conn (this replica only)

    seq uint64

    // busCancel tears down the room's bus subscription when it empties.
    busCancel func()
}

// NextSeq returns the next monotonic sequence number for ordered mutation
// broadcasts (the seq field of S2C.ROOM_ACTION / GAME_ACTION).
func (r *Room) NextSeq() uint64 { return atomic.AddUint64(&r.seq, 1) }
```

`NextSeq` uses `atomic.AddUint64`, not the room mutex, so sequence assignment never contends with membership operations. `TestRoomSeqMonotonic` fires 1000 concurrent `NextSeq` calls and asserts all 1000 results are unique, verifying the ordering.

guarantee under contention.

Local delivery snapshots the member set under a read lock, then sends outside the lock so a slow `trySend` (or its eviction path) cannot block membership changes:

rmhstudios-go

```
func (r *Room) deliverLocal(raw []byte) {
    r.mu.RLock()
    members := make([]*Conn, 0, len(r.members))
    for _, c := range r.members {
        members = append(members, c)
    }
    r.mu.RUnlock()
    for _, c := range members {
        c.trySend(raw)
    }
}
```

Room lifecycle: lazy create, teardown when empty

Rooms are created lazily on first `Join` and reaped on the `Leave` that empties them. The empty-check is re-validated under the write lock to avoid racing a concurrent `Join` that might re-populate the room between the read and the delete:

```
func (h *Hub) Leave(c *Conn, roomID string) {
    h.mu.RLock()
    room := h.rooms[roomID]
    h.mu.RUnlock()
    if room == nil {
        return
    }
    empty := room.remove(c.ID)
    c.removeRoom(roomID)
    if empty {
        h.mu.Lock()
        // Re-check under the write lock to avoid racing a concurrent Join.
        if r := h.rooms[roomID]; r != nil && r.Size() == 0 {
            if r.busCancel != nil {
                r.busCancel()
            }
            delete(h.rooms, roomID)
        }
        h.mu.Unlock()
    }
}
```

GraceTimers — porting the setTimeout disconnect grace

The Node room managers leaned on `setTimeout` for two recurring behaviors: a *disconnect grace period* (don't remove a player the instant their socket drops — give them seconds to reconnect: rmhtube ~120s, rmhmusic ~30s) and an *empty-room* GC reap. `GraceTimers` is the faithful Go port: keyed, cancellable, delayed actions where re-scheduling a key cancels its prior timer so the most recent call always wins.

```
func (g *GraceTimers) Schedule(key string, d time.Duration, fn func()) {
    g.mu.Lock()
    defer g.mu.Unlock()
    if t, ok := g.timers[key]; ok {
        t.Stop()
    }
    g.timers[key] = time.AfterFunc(d, func() {
        g.mu.Lock()
        delete(g.timers, key)
        g.mu.Unlock()
        fn()
    })
}

func (g *GraceTimers) Cancel(key string) bool {
    g.mu.Lock()
    defer g.mu.Unlock()
    if t, ok := g.timers[key]; ok {
        stopped := t.Stop()
        delete(g.timers, key)
        return stopped
    }
    return false
}
```

The reconnect pattern reads naturally: on disconnect, `Schedule(userID, grace, removePlayer)` ; on reconnect within the window, `Cancel(userID)` stops the pending removal. `TestGraceTimerCancel` confirms a cancelled timer never fires; `TestGraceTimerReschedule` confirms that scheduling the same key twice runs only the second action (reschedule cancels the prior). `CancelAll` drains every pending timer on shutdown.

Cross-replica broadcast via the bus

Because each replica only holds its own local members, a room spanning multiple pods needs a fan-out path — the role `Socket.IO` filled with its Redis adapter. Here it is the `events.Bus` . `Broadcast` does two things: deliver to local members immediately, and publish the same bytes to the room's bus topic for remote replicas.

```
func (h *Hub) Broadcast(roomID string, e Envelope) {
    raw := e.Encode()
    h.mu.RLock()
    room := h.rooms[roomID]
    h.mu.RUnlock()
    if room != nil {
        room.deliverLocal(raw)
    }
    _ = h.bus.Publish(h.ctx, busTopic(roomID), raw)
}

func (h *Hub) BroadcastSeq(roomID string, e Envelope) {
    h.mu.RLock()
    room := h.rooms[roomID]
    h.mu.RUnlock()
    if room != nil {
        e.Seq = room.NextSeq()
    }
    h.Broadcast(roomID, e)
}
```

`BroadcastSeq` is the ordered mutation path: it stamps the envelope with the room's next sequence number *before* broadcasting, so the seq is assigned once (on the origin replica) and travels with the bytes to every replica — preserving the global ordering that `S2C.ROOM_ACTION` / `GAME_ACTION` clients depend on.

Origin echo-skip

rmhstudios-go

The subtlety is avoiding double-delivery: the origin replica already delivered locally in `Broadcast`, so when its own publish loops back through the bus it must be ignored. Each room's subscription drops any message whose `Origin` matches this hub's origin:

```
func (r *Room) subscribeBus(ctx context.Context, h *Hub) {
    ch, cancel, err := h.bus.Subscribe(ctx, busTopic(r.ID))
    if err != nil {
        h.logger.Warn("bus subscribe failed", "room", r.ID, "error", err.Error())
        return
    }
    r.busCancel = cancel
    go func() {
        for msg := range ch {
            if msg.Origin == h.origin {
                continue // already delivered locally by Broadcast
            }
            r.deliverLocal(msg.Payload)
        }
    }()
}

func busTopic(roomID string) string { return "rt:room:" + roomID }
```

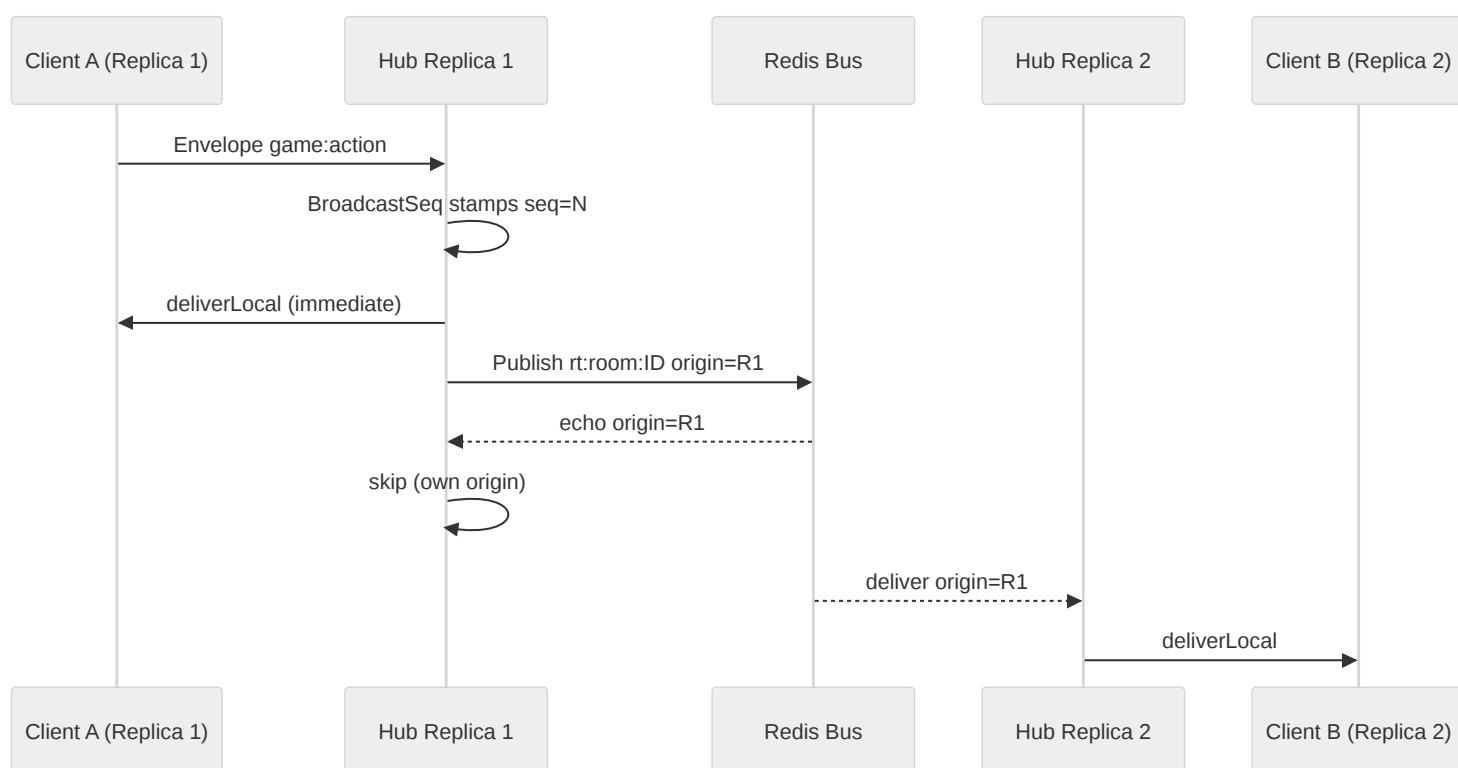


Figure: 2-replica fan-out. The origin delivers locally and publishes once; the remote replica fans out to its members; the origin's own echo is skipped.

The room teardown path (above) calls `busCancel` when the last local member leaves, so an empty room on this replica stops consuming the bus topic — the subscription lifecycle is tied exactly to local membership.

End-to-end: connect and dispatch

Pulling the pieces together, the full path from handshake to a handler acting on a room:

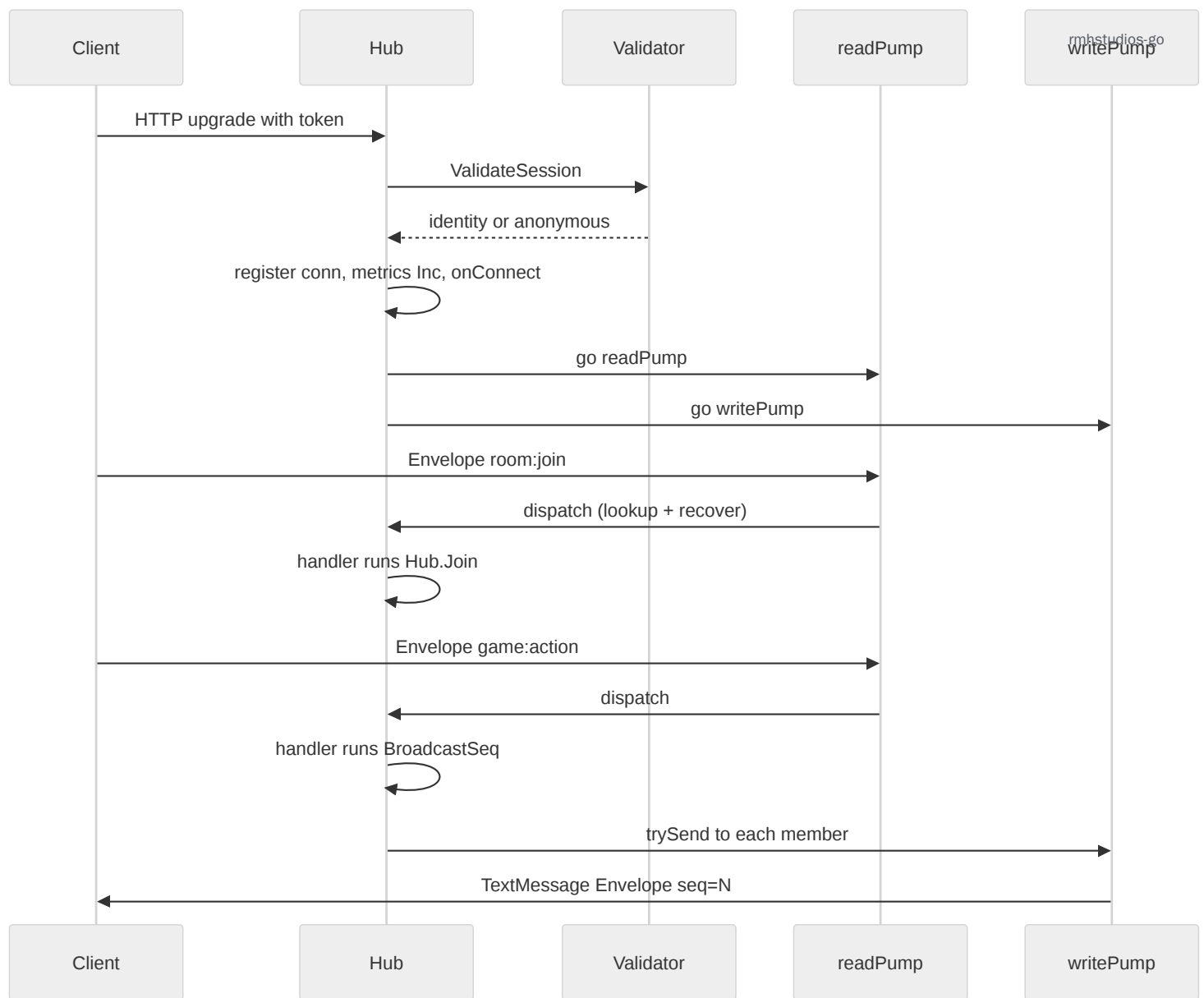


Figure: connect, then a join and a seq-stamped mutation, through the read pump, dispatch, and back out via the write pump.

On any disconnect, `Hub.remove` closes the loop: it leaves every room the connection joined (reaping now-empty rooms and their bus subscriptions), removes the connection from the registry, decrements the active-connection gauge, and fires `onDisconnect` — the hook where a service arms a `GraceTimers` entry to defer player removal until the reconnect window expires.

Status. The Go side of the realtime framework — protocol, hub, conn, room, grace timers, and cross-replica bus fan-out — is implemented and unit-tested (`realtime_test.go` covers envelope round-trip, seq monotonicity under 1000-way concurrency, and grace-timer cancel/reschedule). The corrected channel-shutdown design is in place. The one outstanding migration deliverable is the TypeScript client adapter that bridges the Socket.IO v4 wire format to this JSON-over-raw-WebSocket transport.

7 Service: Gateway (Edge / BFF)

The `gateway` service is the single edge / backend-for-frontend (BFF) that fronts the whole rmhstudios platform. It listens on `PORT_WEB` (default `7005`) and is the one ingress target the Helm chart from PR #121 routes to: it reverse-proxies the React SSR / Nitro app and the Go realtime services (WebSocket pass-through included), turns a Better Auth session into a trusted identity header set, optionally serves built static assets, and exposes `/health` + `/metrics`. It is implemented in `internal/gateway/proxy.go`, `internal/gateway/auth_middleware.go`, and wired in `cmd/gateway/main.go`.

7005

PORT_WEB EDGE

4

WS PREFIX ROUTES

1

SSR CATCH-ALL

X-Rmh-*

TRUST BOUNDARY

Role: the single edge / BFF

In the Node deployment each browser request could fan out to several processes: the React SSR app, the unified Socket.IO games server, and the rmhbox / rmhtube / rmhmusic realtime services. The gateway collapses that fan-out into one `http.Handler`. Everything the platform exposes to the public network arrives here first, which makes the gateway the natural place to put two cross-cutting concerns the individual services should not each re-implement: upstream routing and the authentication trust boundary.

The package doc comment states the contract directly: the gateway is “the single ingress target the Helm chart (PR #121) routes to: it fronts the React SSR/Nitro app and reverse-proxies the WebSocket path prefixes to the Go realtime services.” Because it is the only thing the chart needs to publish, the rest of the Go services can stay `ClusterIP`-internal and trust the identity headers the gateway asserts.

The default upstream targets mirror the Compose / Helm service names and the `PORT_*` family the Node services already used, so the gateway drops in without renaming anything:

```
const (
    defaultWebUpstream      = "http://web:3000"
    defaultGamehubUpstream = "http://gamehub:7001"
    defaultRmhboxUpstream  = "http://rmhbox:7676"
    defaultRmhtubeUpstream = "http://rmhtube:7003"
    defaultRmhmusicUpstream = "http://rmhmusic:7002"
)
```

Longest-prefix reverse-proxy router

The router selects an upstream by **longest-matching path prefix**, falling back to the web (SSR) upstream for everything no Go service owns. Each upstream is a `prefix` string paired with a parsed `*url.URL` target and a `*httputil.ReverseProxy`. `NewRouter` builds the web catch-all first (prefix `""`), then the four Go realtime routes:

```
routes := []struct {
    prefix string
    target string
}{
    {"/socket/", cfg.GamehubUpstream},
    {"/rmhbox-ws/", cfg.RmhboxUpstream},
    {"/rmhtube-ws/", cfg.RmhtubeUpstream},
    {"/rmhmusic-ws/", cfg.RmhmusicUpstream},
}
```

Each upstream is configurable at runtime through its `*_UPSTREAM` environment variable (`WEB_UPSTREAM`, `GAMEHUB_UPSTREAM`, `RMHBOX_UPSTREAM`, `RMHTUBE_UPSTREAM`, `RMHMUSIC_UPSTREAM`), resolved by `LoadConfig` against the defaults above. The realtime hubs

serve these *exact* prefixes (for example the rmhbox hub registers `ServeWS` on `/rmhbox-ws/`), so the gateway proxies the full path through unchanged — there is deliberately no prefix stripping.

rmhstudios-go

After registration the prefixed routes are sorted longest-first so selection is deterministic and “most specific wins”:

```
sort.Slice(r.prefixed, func(i, j int) bool {
    return len(r.prefixed[i].prefix) > len(r.prefixed[j].prefix)
})
```

The hot path is `pick`, which walks the sorted prefixes and returns the first match, or the web catch-all when nothing matches:

```
func (r *Router) pick(path string) *upstream {
    for _, u := range r.prefixed {
        if hasPrefix(path, u.prefix) {
            return u
        }
    }
    return r.web
}
```

Path prefix	Upstream (default)	Env override	Kind
/socket/	http://gamehub:7001	GAMEHUB_UPSTREAM	WS
/rmhbox-ws/	http://rmhbox:7676	RMHBOX_UPSTREAM	WS
/rmhtube-ws/	http://rmhtube:7003	RMHTUBE_UPSTREAM	WS
/rmhmusic-ws/	http://rmhmusic:7002	RMHMUSIC_UPSTREAM	WS
/ (catch-all)	http://web:3000	WEB_UPSTREAM	SSR / HTTP

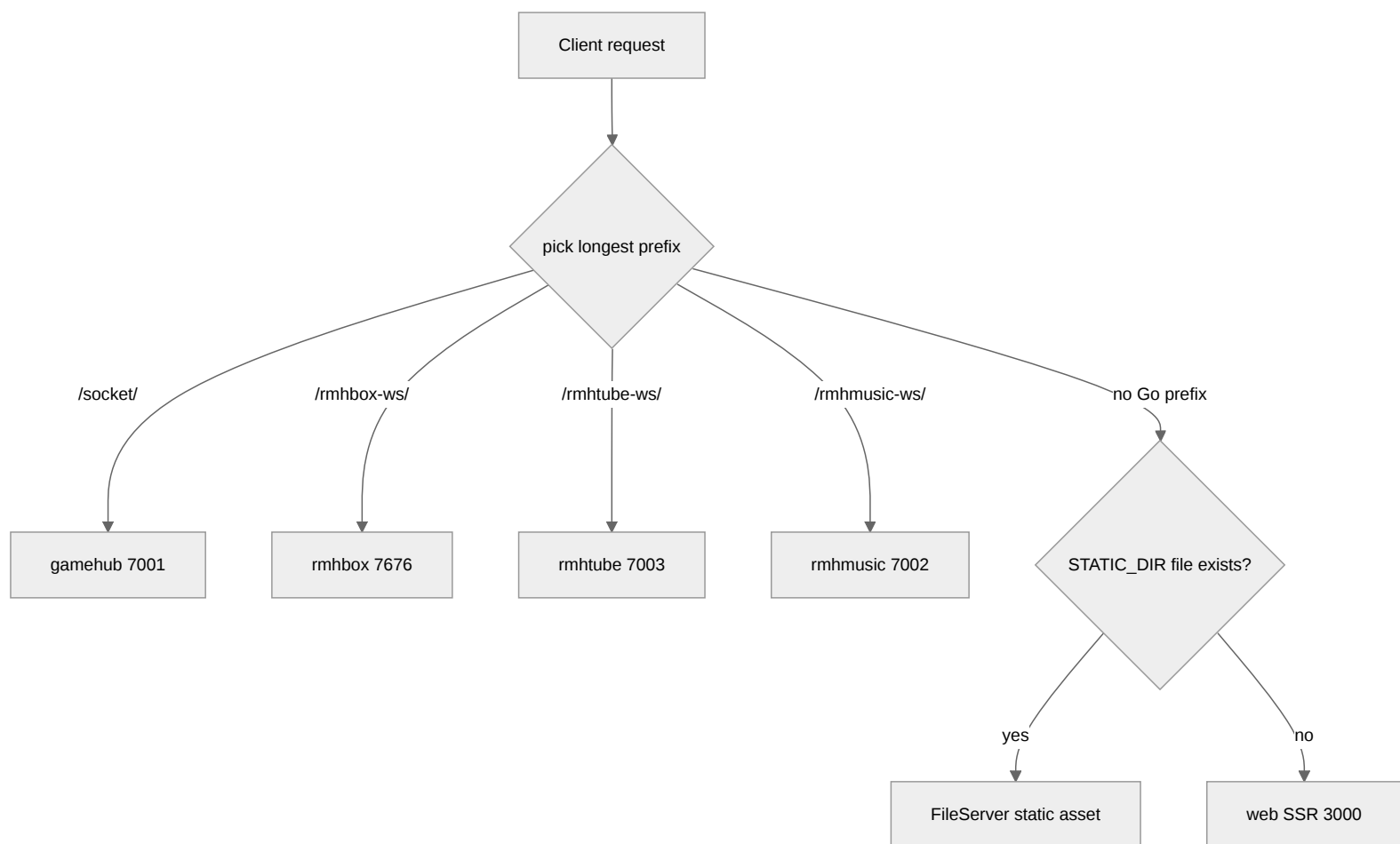


Figure 7.1: routing decision — Go prefixes win first, then static files, then the SSR catch-all.

WebSocket upgrade pass-through

rmhstudios-go

The realtime routes all carry WebSocket traffic, and the gateway passes the upgrade through transparently. The key is *not* doing anything clever: `newProxy` builds a plain `httputil.NewSingleHostReverseProxy` and leaves the default Director in place. Go 1.23's `httputil.ReverseProxy` detects a WebSocket upgrade request, switches into bidirectional-copy mode on a `101 Switching Protocols` response, and manages the hop-by-hop `Connection / Upgrade` headers itself.

The code comment is explicit that this is intentional: “We deliberately do NOT install a Director that strips the Connection/Upgrade hop-by-hop headers... Leaving the default Director in place is exactly what makes WebSocket pass-through work.” A custom Director that scrubbed those headers would silently break the upgrade.

The only customization is an `ErrorHandler` that logs the upstream target and path and returns a `502 Bad Gateway` (“upstream unavailable”) rather than letting the proxy emit its default error.

Auth trust boundary

Because the gateway is the only public ingress, it is also the trust boundary for identity. `AuthMiddleware` (in `auth_middleware.go`) wraps the proxy router so every proxied request carries a validated identity header set — or none, for anonymous traffic. It does two things, in order.

1. Strip every inbound `X-Rmh-*` header

Before any validation, the middleware deletes *all* client-supplied headers under the `X-Rmh-` prefix, canonicalizing each name first:

```
for name := range r.Header {
    if strings.HasPrefix(http.CanonicalHeaderKey(name), trustedHeaderPrefix) {
        r.Header.Del(name)
    }
}
```

This prefix-wide strip is a hardening step that came out of senior review. Deleting only the three headers the gateway injects would still let a client smuggle a future `X-Rmh-*` header that some later upstream learns to trust. Stripping the entire prefix means “neither today's headers nor any future `X-Rmh-*` header can be forged by a client,” as the constant's doc comment puts it.

2. Validate the session and inject trusted identity

If a session token is present (extracted by `httpx.SessionToken`) and it validates against the `SessionValidator`, the middleware injects three trusted headers toward the upstream:

```
if id, err := v.ValidateSession(r.Context(), token); err == nil {
    r.Header.Set(HeaderUserID, id.UserID) // X-Rmh-User-Id
    r.Header.Set(HeaderName, id.Name)     // X-Rmh-User-Name
    if id.IsAdmin {
        r.Header.Set(HeaderIsAdmin, "true") // X-Rmh-Is-Admin
    } else {
        r.Header.Set(HeaderIsAdmin, "false")
    }
}
```

The middleware is **non-blocking**: an absent or invalid token simply passes through as anonymous (the headers stay stripped), because the SSR app renders its own auth pages and the realtime hubs run soft-auth. The `SessionValidator` is a narrow interface (`ValidateSession(ctx, token) (auth.Identity, error)`) — the real implementation is `*auth.Validator` backed by the DB pool, while tests supply a fake, so the middleware is exercisable with `httptest` alone.

For routes that *must* be authenticated, the package offers the opt-in `RequireAuth` helper, which returns a `401` JSON error when no gateway-asserted `X-Rmh-User-Id` is present. It is an explicit seam for future Go API routes; the proxy catch-all does not use it today.

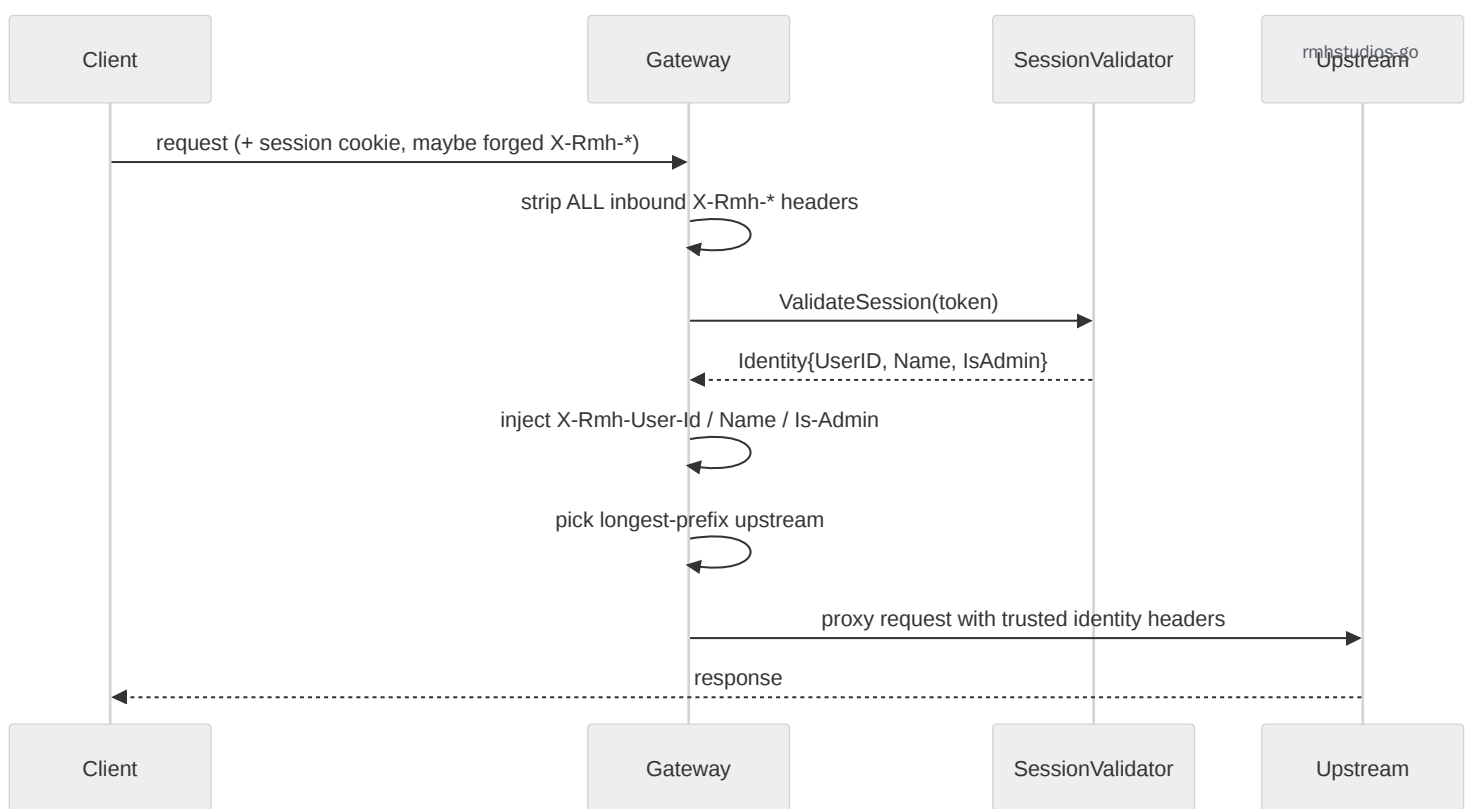


Figure 7.2: an authenticated request through the gateway to an upstream — strip, validate, inject, route.

Static asset serving

The gateway can optionally serve built React assets directly. When `STATIC_DIR` is set and points at an existing directory, `NewRouter` installs an `http.FileServer` over it; otherwise it logs a warning and ignores the value. In `ServeHTTP` the static server is consulted *only after* the Go prefixes and *only for real files*:

```

func (r *Router) ServeHTTP(w http.ResponseWriter, req *http.Request) {
    if u := r.pick(req.URL.Path); u != r.web {
        u.proxy.ServeHTTP(w, req)
        return
    }
    if r.static != nil && r.staticAssetExists(req.URL.Path) {
        r.static.ServeHTTP(w, req)
        return
    }
    r.web.proxy.ServeHTTP(w, req)
}

```

`staticAssetExists` opens the path under the static dir and returns true only for a regular (non-directory) file. This short-circuit-only-for-real-files rule is deliberate: unknown routes — SSR pages and client-side routes — must still fall through to the web upstream instead of 404ing from the file server.

Service wiring (`cmd/gateway/main.go`)

The entrypoint loads common config, opens the DB pool that backs the auth validator, builds the router, and composes the auth middleware in front of it:

```

router, _ := gateway.NewRouter(gateway.LoadConfig(), logger)
proxied := gateway.AuthMiddleware(v)(router)

mux := http.NewServeMux()
mux.HandleFunc("/health", httpx.Health("gateway", nil))
mux.Handle("/metrics", metrics.Handler())
mux.Handle("/", proxied) // catch-all -> auth-wrapped proxy router

addr := ":" + config.GetString("PORT_WEB", "7005")

```

Only `/health` and `/metrics` are gateway-local; everything else is handed to the auth-wrapped proxy router, so every proxied request carries the trusted identity headers (or none). The server is run with a 30-second graceful shutdown via `httpx.NewServer`.

Tests

The gateway is covered at the two seams that matter:

- **Prefix selection.** `pick` is unit-tested for correct longest-prefix routing — each `/socket/`, `/rmhbox-ws/`, `/rmhtube-ws/`, `/rmhmusic-ws/` path resolves to its own upstream, and unmatched paths fall to the web catch-all.
- **Header inject / strip.** The auth middleware is exercised with a fake `SessionValidator` over `httptest`: a valid token injects `X-Rmh-User-Id / Name / Is-Admin`, while a forged inbound `X-Rmh-*` header is stripped and never reaches the next handler — the property the trust boundary exists to guarantee.

The fake-validator design means these tests need neither a database nor a live websocket, so they run fast and under the race detector alongside the rest of the suite.

The `gamehub` service is the Go port of the unified Socket.IO games server (`server/socket-server`): one WebSocket runtime hosting every mini-game relay behind a single shared `realtime.Hub` on the path `/socket/`, listening on `SOCKET_PORT` (default `7001`). Auth is **soft** — anonymous connections are allowed, matching the legacy soft-auth middleware. Kowloon Knockout is ported faithfully and standalone (`kk.go`); the remaining games are represented by the generic `RelayGroup` framework (`relay.go`) plus a clearly-marked, compiling registry of stubs (`manager.go`).

7001

SOCKET_PORT

/socket/

WS PATH

soft

AUTH

KK

FAITHFUL PORT

Role: one WebSocket runtime for every game

The Node deployment ran a single Socket.IO process (`server/socket-server`) whose `index.ts` registered every game's handlers on connect and called every game's disconnect handler on disconnect. `gamehub` reproduces that shape on top of the shared `realtime.Hub`. The endpoint (`cmd/gamehub/main.go`) builds the hub with `RequireAuth: false` — soft-auth, anonymous allowed — and serves it on `/socket/` alongside `/health` and `/metrics`:

```
hub := realtime.NewHub(ctx, realtime.Options{
    Origin:      host,
    Validator:   validator,
    AllowOrigins: config.GetCSV("SOCKET_CORS_ORIGIN"), // empty => allow all
    RequireAuth: false,                               // soft-auth: anonymous allowed
})
mgr := gamehub.NewManager(hub, logger)
mgr.Register()
mux.HandleFunc("/socket/", hub.ServeWS)
addr := ":" + config.GetString("SOCKET_PORT", "7001")
```

The session DB is treated as *optional*. The legacy server allowed connections even when auth was unavailable, so a failed DB open here degrades to fully-anonymous (`"db unavailable, running anonymous-only"`) rather than being fatal.

The connection registry

Host-authoritative relays must deliver a frame to a *specific* peer (the host, or the guest) rather than broadcast to a room. The hub dispatches handlers per-`Conn` but exposes no public “send to a connection id” lookup, so the package keeps its own `connRegistry` mapping `connID -> send callback` (`relay.go`). The `Manager` owns one shared registry and feeds it from a single `OnConnect` / `OnDisconnect` pair, fanning disconnect cleanup out to every game (mirroring `index.ts`):

```
m.hub.OnConnect(func(c *realtime.Conn) {
    conn := c
    m.reg.add(conn.ID, func(e realtime.Envelope) { conn.Send(e) })
})
m.hub.OnDisconnect(func(c *realtime.Conn) {
    m.kk.Cleanup(c.ID)
    for _, g := range m.relays { g.Cleanup(c.ID) }
    m.reg.remove(c.ID)
})
```

Keeping the sender behind a `func(realtime.Envelope)` callback rather than a live `*realtime.Conn` is what makes the relay cores unit-testable: tests register plain string ids with an in-memory capture callback, no websocket required.

Kowloon Knockout: faithful 1v1 host-authoritative relay

rmhstudios-go

Kowloon Knockout (`kk.go`) is a faithful Go port of `server/socket-server/handlers/kowloon-knockout.ts` . It is a simple 1v1 host-authoritative relay: the host creates a room, a guest joins, then `input` frames (guest -> host) and `game_state` frames (host -> guest) are relayed; a rematch handshake re-arms the match. It is ported standalone rather than via `RelayGroup` because of the rematch handshake and fighter-class bookkeeping, which are kept verbatim against the source of truth.

State

`KKManager` holds a `rooms` map (`code -> *KKRoom`), a `connToRoom` reverse index (`connID -> code`), and a `sync.Mutex` guarding both, plus the shared `connRegistry` . A `KKRoom` mirrors the TS interface: the host and (initially empty) guest connection ids, each side's fighter class, and the pending rematch-handshake state (`HostReady` / `GuestReady` and `PendingHostClass` / `PendingGuestClass`). The default fighter is `stone_tiger` .

The exact `kk:*` events

The event constants are kept 1:1 with the legacy C2S/S2C maps:

Inbound (client -> server)	Outbound (server -> client)	Meaning
<code>kk:create_room</code>	<code>kk:room_created</code> (to host, <code>{code}</code>)	host opens a room
<code>kk:join_room</code>	<code>kk:room_joined</code> (to both, <code>{hostClass, guestClass, isHost}</code>)	guest seats; match armed
<code>kk:input</code> (guest)	<code>kk:input</code> (relayed to host)	guest input relay
<code>kk:game_state</code> (host)	<code>kk:game_state</code> (relayed to guest)	host authoritative state
<code>kk:fighter_ready</code>	<code>kk:opponent_ready</code> , then <code>kk:room_joined</code> on both ready	rematch handshake
<code>kk:leave</code>	<code>kk:opponent_disconnected</code> (to peer)	explicit leave
(hub disconnect)	<code>kk:opponent_disconnected</code> (to peer)	implicit teardown
—	<code>kk:error</code> (<code>{message}</code>)	“Room not found” / “Room is full”

Relay direction is enforced

The relays are strictly directional and seat-checked. `Input` only relays guest -> host (it returns early unless `room.GuestConnID == connID`); `GameState` only relays host -> guest (requires `room.HostConnID == connID` and a seated guest). Relayed frames reuse the inbound payload bytes verbatim (`realtime.Envelope{Event: kkInput, Payload: payload}`), so the gateway / hub never re-serializes game data.

Rematch handshake

`FighterReady` records the caller's pending fighter selection, sets that seat's ready flag, and notifies the opponent with `kk:opponent_ready` . When *both* seats are ready it commits the pending classes onto the room, clears the ready/pending state, and re-arms the match by emitting a fresh `kk:room_joined` to both players. Resetting the flags is what prevents a later single-side ready from spuriously re-triggering a rematch.

Room codes

Codes are 6 characters from the alphabet `ABCDEFGHIJKLMNOPQRSTUVWXYZ23456789` (matching the socket-server config), generated with `crypto/rand` instead of `Math.random` . `uniqueCode` ports `generateUniqueCode` : up to 20 attempts, falling back to the last candidate rather than panicking — collisions in the 32⁶ space are astronomically unlikely, so it degrades gracefully where the legacy code threw.

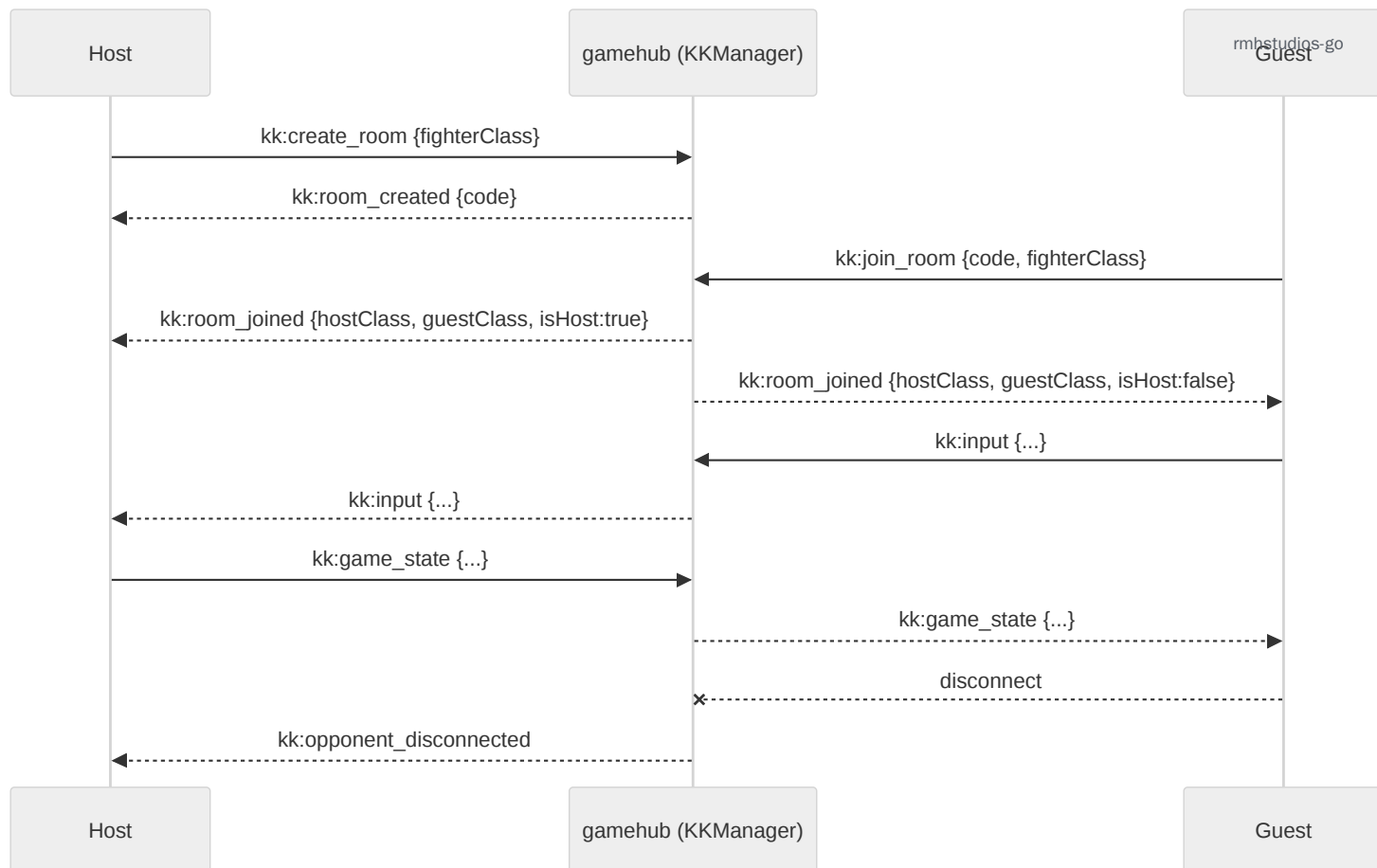


Figure 8.1: a Kowloon Knockout match — create, join, directional relay, teardown.

The generic RelayGroup framework

The legacy Node server hand-rolled the same 1v1 “host creates / guest joins / relay input+state” shape across several handlers. Rather than copy that boilerplate per game, `RelayGroup` (`relay.go`) captures it once and lets each game register onto the hub with its own event names. A `RelayConfig` declares the full wire surface — the inbound `CreateRoom` / `JoinRoom` / `Input` / `State` / `Leave` events and the outbound `RoomCreated` / `RoomJoined` / `Disconnected` / `Errorf` events — with every name fully qualified so the mapping to the legacy `Socket.IO` constants stays explicit and greppable.

`RelayGroup` implements only the common create/join/input/state/leave flow over a two-seat `RelayRoom` (`Code` , `Host` , optional `Guest`), with the same `mutex` + `rooms` / `connToRoom` structure and the same directional relay and seat checks as KK. Multiple games can run as separate `RelayGroup` instances sharing one `connRegistry` and hub. Games that need extra semantics — like KK’s rematch handshake — are ported standalone instead.

Scope — what is faithful vs framework+stub. Kowloon Knockout is a **faithful, verbatim** port with full game semantics and a lifecycle test suite. The other games are **framework + stub**: `RelayGroup` is real, reusable, and tested-by-construction, but the per-game registrations are intentionally not wired with real event names or rules yet. `buildRelayRegistry` returns an empty slice today, so only KK is exposed on the wire; each remaining game is documented as a precise plug-in point.

`manager.go` enumerates the remaining games as a `TODO(migration)` registry — `Slice It`, `Neon Driftway`, `Synapse Storm`, `Blackjack` (noted as multi-seat, needing a richer model than the two-seat `RelayRoom`), plus `Holdem`, `Baccarat`, `Roulette`, `Lights Out`, `Doctrine`, `RMH Type/Study`, `Altair`, and `RMH Music`. A commented example shows exactly how a ported relay registers:

```
// sliceIt := NewRelayGroup(RelayConfig{
//     Prefix:      "slice",
//     CreateRoom:   "slice:create_room",
//     JoinRoom:     "slice:join_room",
//     Input:        "slice:input",
//     State:        "slice:game_state",
//     Leave:        "slice:leave",
//     RoomCreated:  "slice:room_created",
//     RoomJoined:   "slice:room_joined",
//     Disconnected: "slice:opponent_disconnected",
//     Errorf:       "slice:error",
// }, reg, logger)
// return []*RelayGroup{sliceIt}
```

The registry returns `nil` on purpose: this keeps the service compiling and behaving correctly (only KK live) so no half-ported game ships with real event names while its rules are still missing.

The Kowloon Knockout lifecycle test

`kk_test.go` exercises the KK core directly on connection-id strings plus a `capture` sink that records the envelopes a fake connection receives — no live websocket. `newKKBarnes` wires a `KKManager` around a registry with two fake conns (`host-conn` , `guest-conn`). The suite covers the full lifecycle:

- `TestKKCreateAndJoin` — create yields `kk:room_created` with a non-empty code; join delivers `kk:room_joined` to both seats with the right `isHost` flag and the committed `hostClass` / `guestClass` (`iron_crane` / `jade_fox`).
- `TestKKJoinErrors` — an unknown code returns “Room not found”; a third connection joining an occupied room returns “Room is full”.
- `TestKKRelay` — guest input relays to host and host `game_state` relays to guest, while the reverse directions are asserted to *not* relay (directionality enforced).
- `TestKKRematchHandshake` — one side ready notifies the opponent but does not start a rematch; both ready commits the pending classes (`new_host` / `new_guest`) and re-emits `kk:room_joined` ; and the handshake flags are confirmed reset so a later single-side ready does not re-trigger.
- `TestKKDisconnectNotifiesOpponentAndCleansUp` — host cleanup sends `kk:opponent_disconnected` to the guest, empties both the `rooms` and `connToRoom` maps, and confirms a subsequent stale input is a no-op.

Because the relay cores resolve peers through the registry callback rather than a live `*realtime.Conn` , this entire lifecycle runs in-memory and under the race detector, giving high-confidence coverage of the one game that is fully ported.

The `rmhbox` service is the Go port of the Node `server/rmhbox` Jackbox-style party-game platform: short multiplayer minigames played from a phone or a Discord Activity, with a shared host-driven lobby. It serves the realtime hub at `/rmhbox-ws/` on port `7676`, built on the shared `realtime.Hub` (replacing Socket.IO) rather than on a bespoke transport. This section documents the lobby manager and its room model, the per-lobby lifecycle FSM and game coordinator, the pausable timer/state-sync layer, the minigame framework, the per-player scoped state that prevents info-leaks, persistence, and the concurrency model — and is honest about what is fully ported (one minigame) versus scaffolded (eight).

7676 WS PORT	10 FSM STATES	9 REGISTERED GAMES	1 / 8 PORTED / STUBBED	10s HEARTBEAT SYNC	120s DISCONNECT GRACE
------------------------	-------------------------	---------------------------------	-------------------------------------	---------------------------------	------------------------------------

Purpose & endpoint

One `rmhbox.Manager` wires together five subsystems onto a single `realtime.Hub`: the `LobbyManager` (lobby CRUD, host controls, garbage collection), the `GameCoordinator` (the per-lobby minigame lifecycle FSM), the `StateSync` service (heartbeats and phase timers), the minigame framework + registry, and the persistence `Repo`. The `cmd/rmhbox/main.go` endpoint is deliberately thin — it opens an *optional* database (a failed open degrades to in-memory play with persistence disabled), constructs the hub, then registers handlers and starts background goroutines.

```
hub := realtime.NewHub(ctx, realtime.Options{
    Origin: host, Logger: logger, Metrics: metrics, Validator: validator,
    Bus: bus, AllowOrigins: config.GetCSV("SOCKET_CORS_ORIGIN"), RequireAuth: false,
})
mgr := rmhbox.NewManager(hub, database, logger)
mgr.Register() // bind connect/disconnect + every inbound rmhbox:* handler
mgr.Start()    // launch heartbeat + lobby GC goroutines
defer mgr.Stop() // tear down goroutines and cancel all grace/disband timers

mux.HandleFunc("/rmhbox-ws/", hub.ServeWS) // addr := ":" + RMHBOX_PORT (7676)
```

Every participating action requires an identity. The Manager guards each handler with `requireUser`, which sends an `UNAUTHENTICATED` error and bails for anonymous connections, so the hub itself runs with `RequireAuth: false` but the application layer enforces identity per event. Inbound events are bound from the envelope into typed payload structs and delegated to the appropriate subsystem; small adapters (`lobbyOnly`, `hostTarget`, `away`) cut boilerplate for the common `{lobbyId}` / `{lobbyId, targetUserId}` shapes.

Faithful-port note. Socket.IO's sub-rooms (`lobby:{id}`, `:players`, `:spectators`, `:player:{userId}`) are modelled as logical member sets on the `Lobby` and broadcast through the hub's single `rmhbox:{id}` room, filtering per-recipient where the Node code targeted a sub-room. Sequenced `GAME_ACTION` mutations go through `hub.BroadcastSeq` so seq numbering matches the legacy delta protocol. Every minigame handler call is wrapped in `recover()` for fault isolation, porting the try/catch-around-every-handler discipline.

LobbyManager & the room model

The `LobbyManager` owns all lobbies plus the same indices the Node manager kept. It is the single source of truth for membership and the only thing that talks to the hub for broadcasts.

Index	Type	Purpose	rmhstudios-go
lobbies	map[string]*Lobby	room code → lobby	
userToLobby	map[string]string	resolve which lobby a user is in	
channelToLobby	map[string]string	Discord channel key → lobby	
userConn	map[string]*realtime.Conn	live conn per user (the :player:{id} target)	
connUser	map[string]string	connID → user, for disconnect mapping	
pendingJoin	map[string][]string	mid-game spectators awaiting join_next_subround	
graceTimers	*realtime.GraceTimers	per-user disconnect grace + keyed preload/ingame timers	
disbandTimers	*realtime.GraceTimers	per-lobby post-session disband delay	

Room codes are four characters from a 32-char ambiguity-free alphabet (`ABCDEFGHIJKLMNOPQRSTUVWXYZ23456789`), drawn from `rand.Intn` (unbiased rejection sampling) and retried up to `maxCodeAttempts` times for uniqueness. The lobby creator becomes host, joins as a ready player, and the lobby starts in `WAITING` .

Broadcast routing

Two distinct broadcast paths exist. `broadcastAction` emits a *sequenced* `rmhbox:game:action` envelope via `hub.BroadcastSeq` — the hub stamps a monotonic per-room `seq` so clients can detect gaps and reorder deltas. Non-mutating events (`settings_opened` , `countdown` , `round_results` , `disbanded`) go through the non-sequenced `broadcastRaw` . Targeted sends resolve the user's live conn through `userConn` ; `broadcastToPlayers` / `broadcastToSpectators` iterate the logical member set and fan out individual sends.

Test seam. Every broadcast/send routes through an optional function field (`broadcastSeqFn` , `broadcastFn` , `sendFn`). When set (in tests) they capture envelopes without a live WebSocket; `testSeq` mirrors the hub's atomic per-room `seq` stamping so ordering behaviour is verifiable. In production these are nil and calls hit the hub directly.

The lobby:* event surface

Inbound event	Handler	Notes
<code>rmhbox:lobby:create</code>	<code>CreateLobby</code>	replies <code>lobby:created</code> with scoped state
<code>rmhbox:lobby:join</code>	<code>JoinLobby</code>	player or <code>asSpectator</code> ; honors JIP policy
<code>rmhbox:lobby:leave</code>	<code>LeaveLobby</code>	cancels grace timer
<code>rmhbox:lobby:kick</code> / <code>transfer_host</code>	<code>KickPlayer</code> / <code>TransferHost</code>	host-only
<code>rmhbox:lobby:update_settings</code>	<code>UpdateSettings</code>	host-only, <code>WAITING</code> only
<code>rmhbox:lobby:toggle_ready</code>	<code>ToggleReady</code>	non-host; can trigger <code>AUTO_START_TRIGGERED</code>
<code>rmhbox:lobby:promote_spectator</code> / <code>request_promotion</code>	<code>PromoteSpectator</code>	between rounds only
<code>rmhbox:lobby:browse</code>	<code>BrowseLobbies</code>	public lobbies, cursor-paginated
<code>rmhbox:lobby:end_session</code>	<code>EndSession</code>	→ <code>SESSION_RESULTS</code> + disband timer
<code>rmhbox:lobby:chat</code>	<code>onChat</code>	sanitized, ring-buffered (100 msgs)
<code>rmhbox:game:pick</code>	<code>PickGame</code>	host pre-selects a game (or <code>__vote__</code>)

Grace, succession, disband & GC

On disconnect, a *player* is marked disconnected and unready and given a 120-second grace timer; a *spectator* is removed immediately (spectators have no grace). Reconnecting cancels the grace timer. If a host leaves, `succeedHost` promotes the

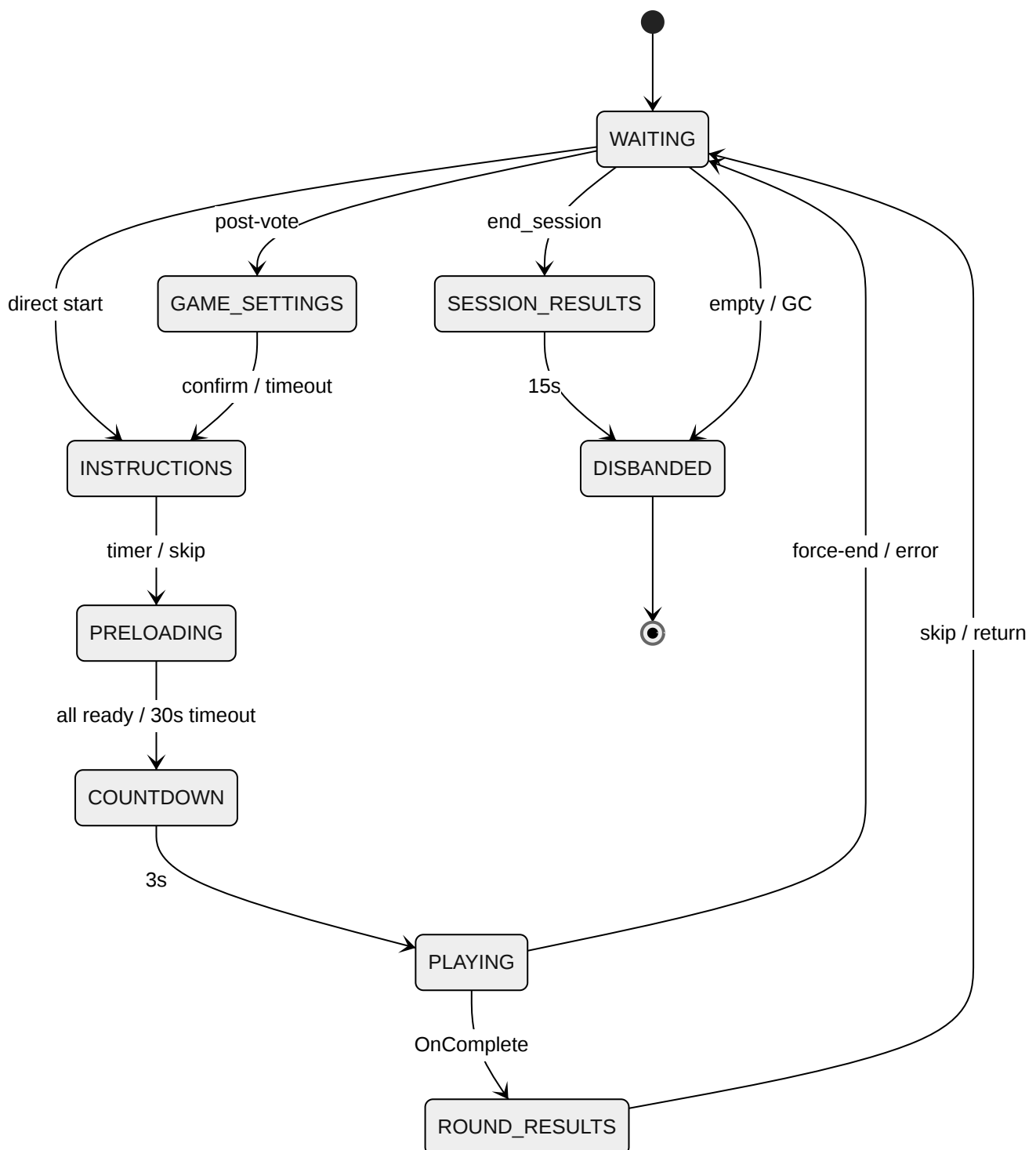
earliest-joined remaining player (or, failing that, promotes a spectator into a player slot); if nobody remains, the lobby disbands. A 60-second GC ticker disbands lobbies that are idle in `WAITING` past 15 min, fully disconnected past 30 min, or empty past 2 min.

Review fix — grace closures re-resolve the live lobby. A grace/preload/ingame timer closure captures a `*Lobby` that may be stale by the time it fires (the lobby could have been disbanded and replaced). Every such closure now re-resolves by id and bails if the live lobby differs: `cur := m.GetLobby(l.ID); if cur == nil || cur != l { return }`. Likewise `removePlayer` only clears `userToLobby[userID]` if it still points at *this* lobby, so a delayed grace expiry can't evict a user who has since rejoined a different lobby.

Lobby FSM

The coordinator drives a single forward path through the lobby's `LobbyState`. The happy path from a host pressing "Start" to results is `WAITING → (GAME_SETTINGS) → INSTRUCTIONS → PRELOADING → COUNTDOWN → PLAYING → ROUND_RESULTS → WAITING`.

`GAME_SETTINGS` is entered only in post-vote mode (a direct host start skips straight to `INSTRUCTIONS`). `VOTING`, `SESSION_RESULTS` and `DISBANDED` are branch/terminal states.



State changes are always paired: a sequenced `STATE_CHANGED` action carries the canonical state, and a per-phase raw event carries phase payload (`game:instructions` , `game:preload_start` , `game:countdown` , `game:started` , `game:round_results`). After each transition the coordinator calls `broadcastFullSync` so every member receives a fresh scoped snapshot at phase boundaries.

Two phases are gated rather than purely time-driven. `PRELOADING` waits for every connected player to emit `rmhbox:game:ready_to_render` ; the coordinator tracks readiness in `lifecycleState.readyPlayers` and advances to `COUNTDOWN` as soon as all are ready, with a 30-second `preload` grace timer as a force-proceed fallback. The host can also short-circuit any pre-play phase via `force_skip` , or abort entirely with `force_end` .

GameCoordinator & a game round

The `GameCoordinator` is the FSM engine. It holds a per-lobby `lifecycleState` (the active `TimerHandle` , the preload readiness set, and an optional phase-cancel func) plus a `spectatorTargets` map. When `startPlaying` runs it snapshots the player set, instantiates the minigame handler from the `serverRegistry` , and builds a `MinigameContext` — the handler's entire outward communication surface (broadcast/send closures, host-id getter, `OnComplete` / `OnError` callbacks).

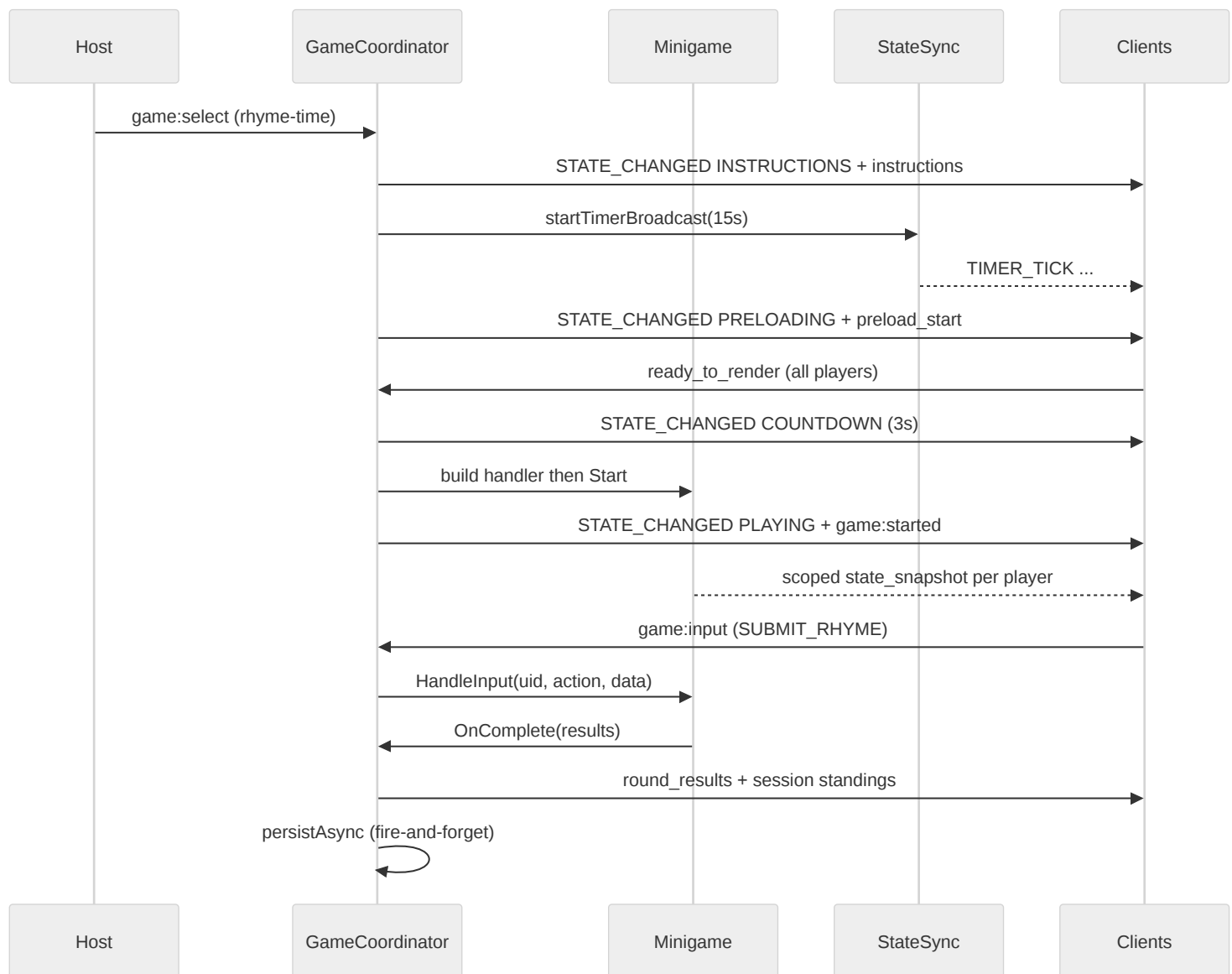


Figure 9.2: one game round end-to-end. Input routing and the `OnComplete` callback are both wrapped in `recover()` .

Player input is routed only while `PLAYING` , only from actual players (spectator input is silently dropped), and only into a non-nil handler — then dispatched under a panic recover so one bad handler can't crash the goroutine:

```
func (gc *GameCoordinator) OnInput(userID, lobbyID, action string, data map[string]any) {
    // ... resolve lobby, verify PLAYING + handler present, verify isPlayer ...
    func() {
        defer func() {
            if rec := recover(); rec != nil {
                gc.logger.Error("game_input_error", "lobbyId", lobbyID,
                    "userId", userID, "panic", rec)
            }
        }()
        handler.HandleInput(userID, action, data)
    }()
}
```

Host controls map onto the FSM directly. `OnForceSkip` advances the current non-playing phase (instructions → preload, preload → countdown, countdown → playing, results → waiting). `OnForceEnd` tears down the handler with `safeCleanup` and returns to `WAITING`. `OnPauseTimer` toggles either the lifecycle `TimerHandle` (for instructions/countdown/results) or the handler's own phase timer (during play). On game completion the coordinator applies round scores cumulatively, records a `ServerMatchSummary` in the lobby's match history, builds live session standings, transitions to `ROUND_RESULTS` with an infinite (host-advanced) timer ring, and kicks off fire-and-forget persistence.

StateSync & the TimerHandle data-race fix

`StateSync` has two jobs. First, a 10-second heartbeat that walks every lobby and, for those in `PLAYING`, pushes a fresh *per-player scoped* full snapshot to every connected member — so a client that missed deltas self-heals every 10s. Second, the pausable 1-second `TIMER_TICK` broadcasts that drive the phase rings: `startTimerBroadcast` emits a `TIMER_START`, then ticks `TIMER_TICK` each second until the countdown crosses below zero, at which point it fires `onComplete`. `broadcastInfiniteTimer` shows a full ring with no ticking for host-advanced results.

Review fix — the TimerHandle stop-channel data race (found by `-race`). The broadcast goroutine reads `h.stop` without the lock. `Cancel` originally nil'd `t.stop`, which raced against that lockless read. The fix: the goroutine *captures* the channel once before it starts (set before the goroutine, never reassigned — happens-before via goroutine creation, so reading it needs no lock), and `Cancel` closes it exactly once behind a `cancelled` guard and *never* nils it.

```
// Cancel stops the timer broadcast.
func (t *TimerHandle) Cancel() {
    t.mu.Lock()
    if t.cancelled { t.mu.Unlock(); return }
    t.cancelled = true
    // Closing the channel is the signal; the `cancelled` guard makes this
    // close-once. We deliberately do NOT nil t.stop – the broadcast goroutine
    // reads it without the lock, so mutating it here would be a data race.
    if t.stop != nil { close(t.stop) }
    t.mu.Unlock()
}

// in startTimerBroadcast:
stop := h.stop           // capture once; never reassigned -> lockless read is safe
go func() {
    for {
        select {
        case <-stop:      return
        case <-t.C:       /* decrement under h.mu, broadcast TIMER_TICK */
        }
    }
}()
```

Minigame framework

The Node abstract `BaseMinigame` class becomes a Go interface (`Minigame`) plus an embeddable helper (`BaseGame`) supplying shared lifecycle plumbing: tracked pausable one-shot timers, the pausable phase-timer broadcast, default no-op hooks, and

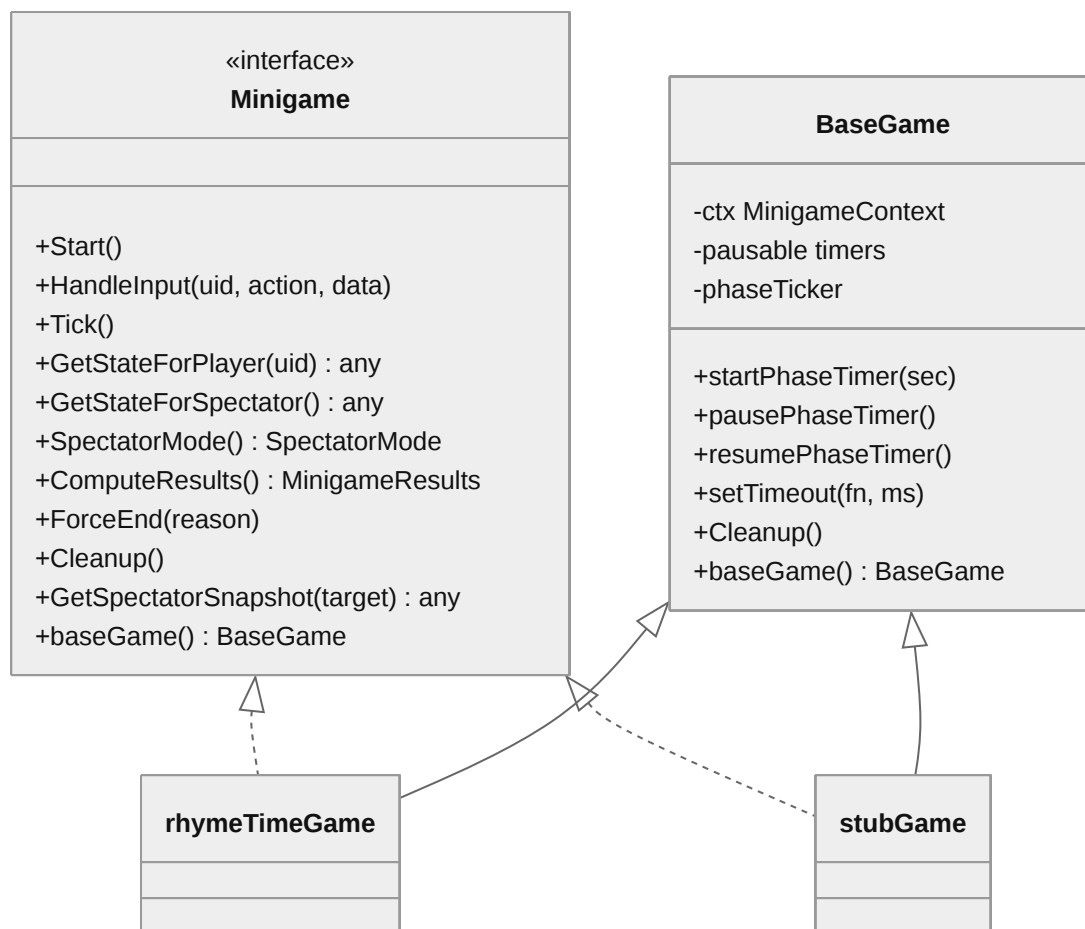


Figure 9.3: the minigame contract. `BaseGame` supplies `baseGame()`, so the compiler guarantees every handler exposes its embedded base for generic pause/resume.

Review fix — `baseGame()` replaces a concrete type-switch. The coordinator needs the embedded `*BaseGame` to drive phase pause/resume generically. An earlier version used a type-switch over each concrete game, which had to be kept in sync as games were added. Now the `Minigame` interface declares an unexported `baseGame() *BaseGame` method that `BaseGame` itself implements, so every embedder satisfies it automatically and `asBaseGame(h)` is a single, total call — the compiler enforces coverage.

The phase-timer pause is non-trivial because in-flight one-shot timers must also freeze. `pausePhaseTimer` snapshots each unfired pausable timer's remaining delay and stops it; `resumePhaseTimer` re-arms each with its saved remainder via `time.AfterFunc`. Every fired callback runs through `fire`, which dedupes (`pt.fired`) and recovers panics into `ctx.OnError`. `BaseGame` is concurrency-safe by design: the host may pause/resume from one goroutine while game timers fire on others, all guarded by `b.mu`.

Rhyme Time — the one fully-ported game

`rhymeTimeGame` is the faithful port of `server/rmhbox/minigames/rhyme-time/handler.ts`. Players get a root word each round and submit rhymes within a time limit; scoring rewards rarity (fewer players found the word), multi-syllable rhymes, and speed (first to a non-unique word), with a penalty for known non-rhyming words. Each round runs a four-phase sub-FSM `ROUND_START` → `INPUT` → `SCORING` → `INTERMISSION` across three rounds, driven by `setTimeout` chains and `startPhaseTimer` broadcasts. End-of-game `computeAwards` produces superlatives (Wordsmith, Diamond in the Rough, Syllable Surfer, Quick Draw, Overachiever).

Scope: ported vs stubbed.

- Fully ported (1):** `rhyme-time` — full phase FSM, input validation, rarity/speed/multi-syllable scoring, per-player state masking, and awards. *Caveat:* the Node handler validated rhymes against the CMU pronouncing dictionary (the `rhyming-part` package). To stay dependency-free the Go port substitutes a deterministic phonetic-suffix heuristic (`rtRhymeKey`) over a small bundled word set; the `structure`, scoring, masking and awards are faithful, the phonetic accuracy is approximate.
- Compiling stubs (8):** `undercover-agent`, `category-crash`, `wiki-race`, `wit-war`, `fact-or-friction`, `undercover-editor`, `minimalist-masterpiece`, `emoji-cinema`. Each is a `stubGame` that auto-completes after 1s with all players ranked at zero, so the

The 9 games are registered with correct static metadata (min/max players, join-in-progress policy, spectator mode) in
minigameRegistry and serverRegistry ; only the gameplay handlers differ in fidelity.

Per-player scoped state (info-leak prevention)

The single exit point for all lobby state is LobbyManager.buildClientState(l, userID) . It strips internal fields and, critically, scopes the active minigame state by the recipient's role: a player gets GetStateForPlayer(userID) (which includes only *their own* private data), while a spectator gets GetStateForSpectator() (no private submissions). For Rhyme Time this means mySubmissions is populated only for the owning player and is empty for spectators during the INPUT phase — the load-bearing distinction that prevents one player seeing another's in-progress answers. Competitive spectators may follow a chosen player via GetSpectatorSnapshot , which for competitive-individual games returns the target player's scoped state.

```
if myRole == "player" {
    gameState = safeStateForPlayer(l.CurrentGame.Handler, userID)
} else {
    gameState = safeStateForSpectator(l.CurrentGame.Handler)
}
```

Both safeStateForPlayer and safeStateForSpectator wrap the handler call in recover() , defaulting to an empty map on panic. Host-only data (kick/transfer/settings, the ban/kick controls) is enforced at the handler layer — non-host callers receive a NOT_HOST error and the mutation is rejected before any broadcast.

Persistence & leaderboard

Persistence is **fire-and-forget**: on game complete the coordinator spawns a goroutine (panic-recovered) that calls repo.PersistMatch ; a DB error is logged but never blocks or fails the game flow. When the database is absent (repo == nil) the whole path is a no-op and leaderboard reads return empty. PersistMatch writes a rmhbox_match row and one rmhbox_match_player row per player inside a single transaction.

Atomic minigameStats merge — fixes a Node concurrency hazard. The Node code read a player's profile, merged the new match into the minigameStats JSONB in application code, then wrote it back — a find-then-update with no lock, so two matches finishing for the same user could clobber each other. The Go port performs the read-modify-write *atomically* under a row lock inside the transaction:

```
SELECT "id","minigameStats","currentWinStreak","bestWinStreak"
FROM "rmhbox_profile" WHERE "userId"=$1 FOR UPDATE -- row locked for the tx
-- then mergeMinigameStats(...) in Go, then UPDATE ... in place
```

The pure merge function mergeMinigameStats (games played, wins, best score, total score, total/average rank) is extracted so the math is unit-tested directly without a database. The leaderboard read maps the caller's metric onto a fixed whitelist of quoted column literals ("totalScore" / "totalWins" / "totalGamesPlayed") before concatenation, so no user input ever reaches the ORDER BY — the query is injection-proof by construction.

Concurrency model

- **Per-lobby mutex.** Each Lobby carries its own sync.Mutex ; every read or write of lobby fields takes it. Many goroutines touch a lobby concurrently — timers, the GC ticker, the heartbeat, and every connection's read pump — so the lock is the lobby's single serialization point. The manager's own RWMutex guards the global indices.
- **Snapshot-then-broadcast.** Handlers consistently take the lock, copy out the minimal data they need (ids, names, a settings value), release the lock, and only *then* broadcast — broadcast helpers re-take the lock themselves, so locks are never held across a fan-out.
- **Grace closures re-resolve.** As noted, every deferred timer re-resolves its lobby by id and bails if it was disbanded/replaced (the review fix).

- **Panic-recover fault isolation.** Every minigame handler entry point (`Start` , `HandleInput` , `HandlePlayerDisconnect` , `ForceEnd` , the state getters, `Cleanup`) and the async persist goroutine are wrapped in `recover()` , routing failures to `ctx.OnError / handleGameError` or a log line rather than crashing the service.

rmhstudios-go

Tests

`rmhbox_test.go` drives the manager through a capturing test rig (no sockets) and covers the load-bearing behaviors:

Test	Verifies
<code>TestFSMTransitions</code>	full forward path WAITING → INSTRUCTIONS → PRELOADING → COUNTDOWN → PLAYING → WAITING via select / skip / ready_to_render / force-end, plus the <code>STATE_CHANGED</code> sequence
<code>TestRoundResultsTransitionOnComplete</code>	<code>handleGameComplete</code> → <code>ROUND_RESULTS</code> , cumulative scoring, match-history recording
<code>TestSeqOrderedGameActions</code>	<code>GAME_ACTION</code> envelopes carry monotonic <code>seq</code> from 1
<code>TestSeqMonotonicUnderConcurrency</code>	100 concurrent broadcasts yield 100 unique, non-duplicated seqs
<code>TestScopedStateFiltering</code>	each player sees only their own submissions; spectator sees none; a follower sees the target's scoped state
<code>TestBuildClientStateScopesByRole</code>	<code>buildClientState</code> tags <code>myRole</code> and never leaks submissions into the spectator's <code>publicState</code>
<code>TestMinigameStatsMerge</code>	read-modify-write math for existing & fresh entries, with the prior entry untouched
<code>TestGameCompleteFiresPersistence</code>	the fire-and-forget goroutine reaches the (fake) repo with the correct match record

The suite compiles, passes `go vet` , and passes under the race detector — the latter being how the `TimerHandle` stop-channel race was caught.

The `rmhtube` service is the Go port of the standalone RmhTube watch-party server (`server/rmhtube/*`). It hosts synchronized video rooms: a group of viewers share one playback head, a media queue, and a chat. One leader's player state is authoritative and the server fans the projected position out to everyone on a heartbeat. It serves the realtime hub at `/rmhtube-ws/` (default port `7003` , `RMHTUBE_PORT`) plus `/health` and `/metrics` , and restores its active rooms from the database before it starts listening so a restart does not vaporize a session. The package lives in `internal/rmhtube/` (events, types, manager, room, sync, queue, chat, snapshot, repo, utils) and is wired in `cmd/rmhtube/main.go` .

7003

RMHTUBE_PORT

~1sHOST-STATE
REPORT**2s**

SYNC HEARTBEAT

120sDISCONNECT
GRACE**3**

DB TABLES

Service shape and startup

The binary is a single WebSocket runtime. `main.go` loads the common config, opens the database *optionally* (a failed open degrades to a fully in-memory service, exactly as the legacy server started anyway when DB restore failed), constructs the shared `realtime.Hub` with `RequireAuth: false` (anonymous viewers are allowed), registers the manager's handlers, and — critically — restores active rooms *before* the listener accepts connections:

```
mgr := rmhtube.NewManager(ctx, hub, repo, logger, metrics)
mgr.Register()
defer mgr.Stop()

// Restore active rooms before listening
// (mirrors restoreRoomsFromDb().then(listen)).
if err := mgr.RestoreRoomsFromDB(); err != nil {
    logger.Error("db_restore_failed", "error", err) // start anyway
}

mux.HandleFunc("/rmhtube-ws/", hub.ServeWS)
addr := ":" + config.GetString("RMHTUBE_PORT", "7003")
```

Because auth is optional, identity falls back gracefully. `userOf` returns the authenticated account id when present and the per-connection id otherwise, so the room machinery functions for guests; `nameOf` defaults to `"Guest"` .

Wire protocol: events, snapshot, and seq deltas

The hub speaks a JSON envelope whose `Event` field carries the exact strings ported 1:1 from `lib/rmhtube/events.ts` , so the existing TypeScript client adapter maps cleanly onto the Go server. Events are grouped into room lifecycle, video sync, queue, reactions, and chat. A representative slice of the client → server (C2S) and server → client (S2C) constants:

```
// Client -> Server (excerpt)
c2sRoomCreate    = "rmhtube:room:create"
c2sRoomJoin     = "rmhtube:room:join"
c2sSyncHostState = "rmhtube:sync:host_state"
c2sSyncPlay     = "rmhtube:sync:play"
c2sSyncSeek     = "rmhtube:sync:seek"
c2sSyncSetSpeed = "rmhtube:sync:set_speed"
c2sQueueAdd     = "rmhtube:queue:add"
c2sQueueVoteSkip = "rmhtube:queue:vote_skip"
c2sQueueShuffle = "rmhtube:queue:shuffle"

// Server -> Client (excerpt)
s2cRoomStateSnapshot = "rmhtube:room:state_snapshot"
s2cRoomAction        = "rmhtube:room:action"
s2cSyncState         = "rmhtube:sync:state"
s2cSyncMediaChanged  = "rmhtube:sync:media_changed"
```

The protocol is **snapshot + seq-numbered delta**. On join the server sends a full `room:state_snapshot` built by `buildClientState` ; every subsequent mutation goes out as a `room:action` envelope carrying a monotonically increasing per-room sequence number. The single broadcast primitive is `broadcastAction` , which bumps the room's `Seq` under the room lock and fans the envelope to the whole hub room:

```
// broadcastAction bumps the room seq and fans S2C.ROOM_ACTION
// {type,payload,seq,timestamp} to the whole room. The caller MUST
// hold the room lock (so seq stays consistent with snapshots).
func (m *Manager) broadcastAction(r *room, actionType string, payload any) {
    r.Seq++
    m.hub.Broadcast(r.ID, realtime.MustEnvelope(s2cRoomAction, map[string]any{
        "type":    actionType,
        "payload":  payload,
        "seq":      r.Seq,
        "timestamp": nowMs(),
    }))
}
```

Holding the room lock while incrementing `Seq` is what keeps the seq on a snapshot consistent with the seqs on the deltas that follow it: a client can discard any delta whose seq it has already applied and detect gaps. A unit test (`TestBroadcastActionSeqMonotonic`) drives 200 actions and asserts the seq lands at exactly 200.

Leader-authoritative video synchronization

This is the heart of the service and a faithful port of `server/rmhtube/sync-engine.ts` . The model: one member is the *leader* (initially the host). The leader's local player is the source of truth. The leader's client reports its player state roughly once a second via `SYNC_HOST_STATE` ; the server stores it as the canonical `videoState` but does **not** rebroadcast it. Instead, a 2-second heartbeat broadcasts the *projected* state to every multi-member room that has something playing. Explicit leader actions (play / pause / seek / set-speed) broadcast immediately.

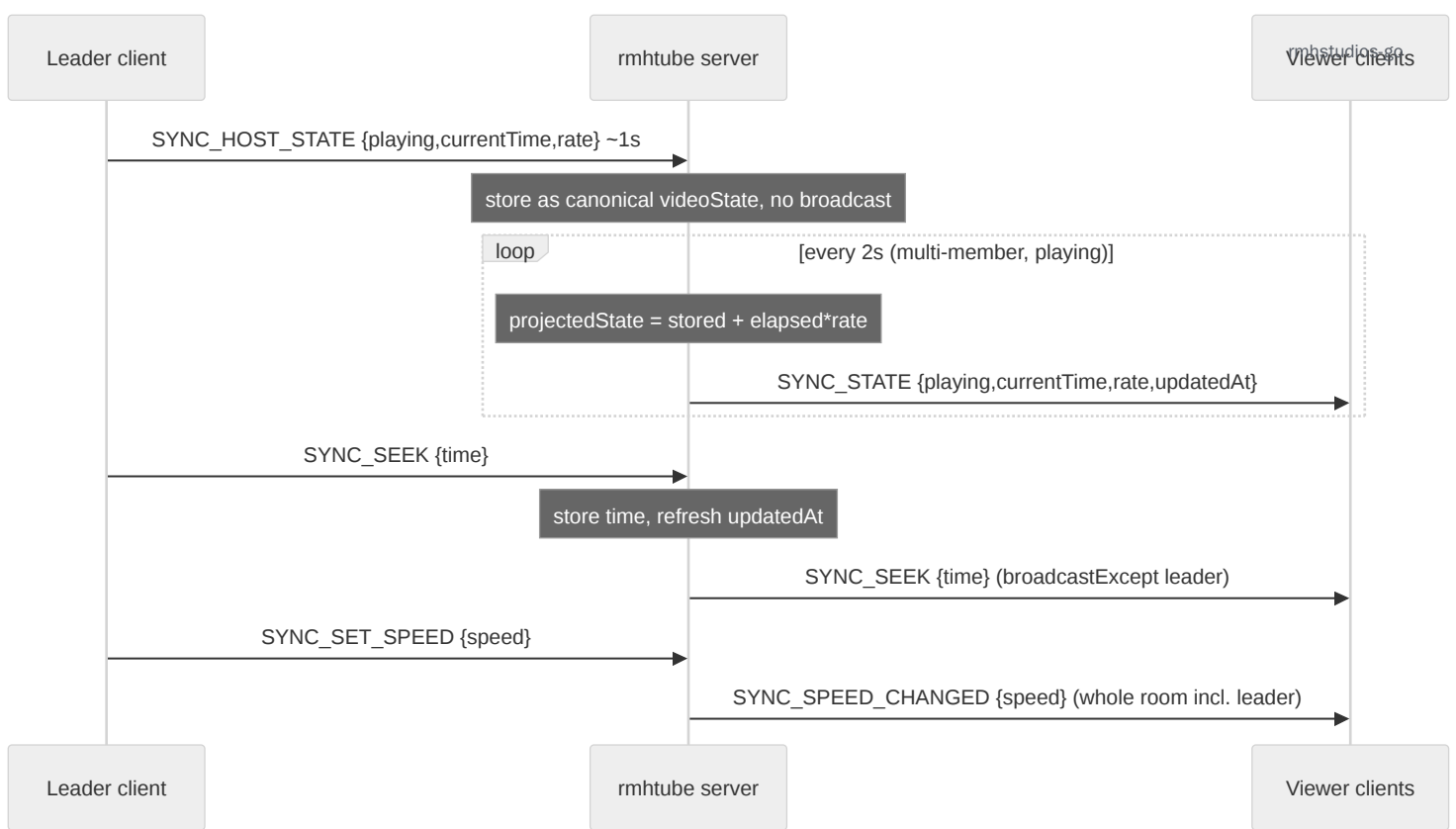


Figure 10.1: leader-authoritative video sync. The leader reports state; the server projects it forward and heartbeats it to viewers.

The projection is the key computation. When playing, the live position is the stored position plus the wall-clock seconds elapsed since `updatedAt`, scaled by playback rate — so a viewer receiving a heartbeat sees where the leader *actually is*, not where the leader was one report ago. Paused state is returned unchanged (the position is frozen), and a clock-skew report from the future is clamped so a viewer can never be rewind:

```

func projectedState(v videoState, now int64) videoState {
    out := v
    if v.Playing {
        elapsed := float64(now-v.UpdatedAt) / 1000.0
        if elapsed < 0 {
            elapsed = 0
        }
        out.CurrentTime = v.CurrentTime + elapsed*v.PlaybackRate
        out.UpdatedAt = now
    }
    return out
}

```

Three unit tests pin this behavior: a 1× projection 5s later yields `10 + 5 = 15`; a 2× projection 4s later yields `10 + 8 = 18`; a paused video stays at 42; and a negative elapsed does not rewind below 10. Authority is enforced by `LeaderRoom`, which returns the user's room only if `r.LeaderUserID == userID` — every sync handler short-circuits for a non-leader. The handlers store under the lock, then release before any websocket send (websocket writes never happen under the lock):

```

func (m *Manager) onSeek(c *realtime.Conn, e realtime.Envelope) {
    var p struct{ Time float64 `json:"time"` }
    if err := e.Bind(&p); err != nil { return }
    ent, r, ok := m.LeaderRoom(userOf(c))
    if !ok { return }
    r.VideoState.CurrentTime = p.Time
    r.VideoState.UpdatedAt = nowMs()
    r.LastActivity = nowMs()
    roomID := r.ID
    ent.mu.Unlock()
    m.broadcastExcept(roomID, c.ID, s2cSyncSeek, map[string]any{"time": p.Time})
}

```

Play / pause / seek use `broadcastExcept` (the leader applies them locally, so it is excluded), while `SET_SPEED` goes to the whole room — matching the TS `io.to` vs `socket.to` distinction. The 2s heartbeat tick (`tickHeartbeat`) snapshots the registry, then per room broadcasts `projectedState` only when `CurrentItem != nil` and there is more than one member (no point synchronizing an audience of one). When media changes, `onMediaChanged` resets the `videoState` to a clean paused state at `t=0`, `rate=1` and emits `SYNC_MEDIA_CHANGED`.

Rooms, queue, and chat

A room owns the full watch-party state: members, queue, current item, video state, chat ring buffer, skip votes, played history, bans, and invite links. Room creation generates a 6-character code from an unambiguous alphabet (retrying up to 10 times on collision), installs the creator as host + leader + first member, joins the hub room, persists the row fire-and-forget, and sends the creator both `room:created` and a full snapshot.

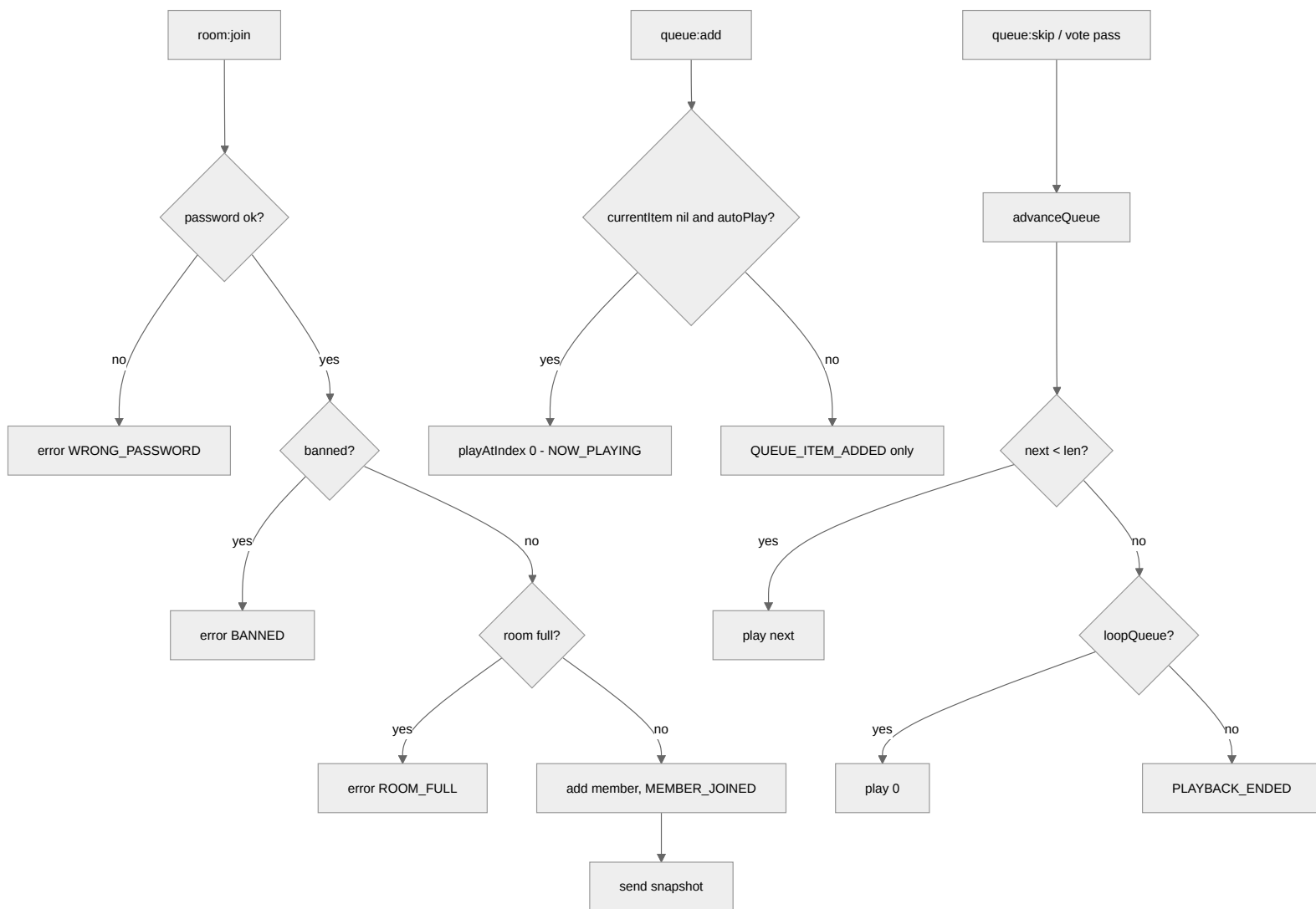


Figure 10.2: room join gate and the queue / playback flow.

The queue (port of `media-queue.ts`) supports add, remove, reorder, play-item, skip, vote-skip, queue-voting, and shuffle. Permissions are role-gated: adding requires host or `AllowMemberQueue`; reorder / play-item / skip / shuffle require the leader; removing your own item is allowed. Adding the first item with `AutoPlay` on auto-starts playback. Vote-skip computes a ceiling-of-half threshold against the *active* member count and advances when met:

```

r.SkipVotes[userID] = struct{}{}
active := r.activeCount()
votesNeeded := (active + 1) / 2 // ceil(active/2)
voters := setToSlice(r.SkipVotes)
m.broadcastAction(r, "VOTE_SKIP_UPDATED", map[string]any{
    "voters": voters, "votesNeeded": votesNeeded, "totalMembers": active,
})
if len(r.SkipVotes) >= votesNeeded {
    m.broadcastAction(r, "VOTE_SKIP_PASSED", map[string]any{})
    m.advanceQueue(r)
}

```

Reorder uses `moveItem`, which clamps the target position into range and reindexes positions; tests cover the move, the high/low clamps, and the no-op-on-missing case. Shuffle is a Fisher–Yates over the items *after* the current index (so the now-playing item is never displaced). Chat (port of `chat-handler.ts`) sanitizes and truncates content to 300 chars, resolves replies, validates mentions against current members, appends to a 200-message ring buffer, and supports typing indicators (with a 3s auto-expiring timer), per-message emoji reactions (toggle add/remove), and host-only pinning. Reactions and typing fan out with `broadcastExcept` so the sender does not echo their own event.

Concurrency model

The brief is “per-room mutex; many goroutines,” and the manager implements it precisely. A top-level `registryMu` guards the global maps (`rooms`, `userRoomIndex`, `userConns`); each `roomEntry` carries its own mutex, and every read/write of a room's state happens while holding it. The discipline is: **acquire the room lock, mutate, collect the data to send, release, then broadcast** — so websocket writes and DB calls never run under the lock. Grace and GC timers run on their own goroutines and re-acquire locks as needed.

```

type roomEntry struct {
    mu sync.Mutex
    r *room
}

```

Disconnect grace. On disconnect the member is marked offline and a 120s grace timer (via the shared `realtime.GraceTimers`) is armed; if the member reconnects within the window, `rejoin` cancels the timer and sends a fresh snapshot. Otherwise the member is removed. Empty rooms are scheduled for disband after a 5-minute timeout, and a 60-second GC ticker sweeps for rooms that are empty-past-timeout or idle-past-30-minutes as a safety net.

Review fixes carried in the port. Two correctness fixes from review are visible in the code. First, `removeMember` prunes the departing user's votes — it deletes them from `SkipVotes` and from every `QueueVotes` set — so vote thresholds (computed against `activeCount`) are never measured against ghosts:

```

delete(r.SkipVotes, userID)
for itemID, voters := range r.QueueVotes {
    delete(voters, userID)
    if len(voters) == 0 {
        delete(r.QueueVotes, itemID)
    }
}

```

Second, `disbandRoom` holds the room lock across the *entire* teardown (broadcast, index cleanup, and registry delete) so a concurrent `onRoomJoin` cannot add a member to a room being deleted. Correspondingly `onRoomJoin` re-verifies the room is still registered after it acquires `ent.mu`, because a concurrent disband may have removed it from the registry while the join was blocked on the lock — adding a member to a disbanded entry would strand it.

Persistence behind a Repo

Unlike the games hub (which keeps its core DB-free), rmhtube persists, so the room logic depends only on a `Repo` interface — the room / sync / queue / chat handlers are exercisable in tests with the in-memory `NopRepo{} . All Repo calls are made fire-and-`

forget through a `persist` helper that logs failures and counts a metric, so a slow or failing database never blocks a real-time mutation (faithful to the TS `.catch(...)` pattern around every Prisma call).

rmhstudios-go

Concern	Repo method(s)	Table
Create / close room	CreateRoom , CloseRoom	rmhtube_room
Host / settings	UpdateHost , UpdateSettings	rmhtube_room
Membership	MemberJoin , MemberLeave	rmhtube_room_member
Queue	QueueAdd , QueueRemove , QueuePlayed , QueuePositions	rmhtube_queue_item
Restore	RestoreActiveRooms , LoadRoom	all three (join)

The production `PgxRepo` runs raw pgx against the Prisma-managed, quoted-identifier schema (column names kept verbatim with the TS DDL). Startup restore selects every not-closed room, hydrates its live members (joined to `"user"` for name/image), its unplayed queue (ordered by position), and recent chat, and skips rooms with no live members — mirroring `restoreRoomsFromDb`. Restored members start *disconnected* and reconnect over the websocket. A join miss for a room not in memory triggers a single on-demand `LoadRoom`. Membership uses an upsert on `(roomId,userId)` that clears `leftAt` on re-join, and position rewrites run in a transaction.

YouTube URL parsing is pure string ops

`utils.go` ports `lib/rmhtube/utils.ts` with no network access whatsoever. `detectMediaType` classifies a URL as `youtube` / `twitch` / `direct` (or `""` for unsupported) by host and a file-extension regex; `extractYouTubeID` pulls the id from `youtu.be/<id>` or the `?v=` query parameter; and the thumbnail URL is templated against `img.youtube.com`. There is no YouTube Data API call — title resolution that the TS version stubbed (`"YouTube Video (<id>)"`) is reproduced exactly. Room codes use `crypto/rand` over the unambiguous alphabet; opaque ids use a nanoid analog.

Scope: what is faithful vs simplified. The load-bearing watch-party core — leader-authoritative sync and its projection, the queue (add/remove/reorder/play/skip/vote-skip/shuffle/loop), chat with replies/mentions/reactions/typing/pinning, the snapshot+seq-delta protocol, the disconnect-grace + GC lifecycle, and room/member/queue persistence and restore — is a faithful, behavior-for-behavior port. The niche Phase-4 social features are intentionally simplified: the *entry gate* for bans is kept real (a banned user is rejected at join with `BANNED`), but the surrounding ban / unban / invite-link bookkeeping is in-memory only and not persisted, and the invite codes are generated but not enforced as a join path. Chat persistence is wired as an optional `chatWriter` extension a Repo may implement rather than part of the required interface — the in-memory ring buffer remains authoritative for live rooms either way. None of this is invented behavior; it is the deliberate faithful-core / simplified-periphery split called out in the source comments.

The `rmhmusic` service is the Go port of the Node `rmhmusic` feature: synchronized group music-listening rooms where the host controls playback and members share a queue and chat. In Node this lived *inside* the unified socket-server behind the `rmhmusic:` event prefix; in the Go world it is extracted into its own standalone service. It serves the realtime hub at `/rmhmusic-ws/` (default port `7002`, `RMHMUSIC_PORT`) plus `/health` and `/metrics`. Unlike `rmhtube`, it sets `RequireAuth: true` on the hub — only authenticated users may connect — which makes the database mandatory (it backs both session validation and the one persisted row). The package lives in `internal/rmhmusic/` (events, types, manager, room sync, queue, chat, repo, util) and is wired in `cmd/rmhmusic/main.go`.

7002

RMHMUSIC_PORT

auth

REQUIREAUTH=TRUE

5s

SYNC HEARTBEAT

30sDISCONNECT
GRACE**1**

DB ROW PERSISTED

Service shape and the auth requirement

The binary is a single authenticated WebSocket runtime. Because auth is mandatory, `main.go` opens the database with `db.Open` and `Fatal`s on failure (no in-memory degrade path, unlike `rmhtube`), then constructs the hub with `RequireAuth: true` and a session validator:

```
database, err := db.Open(ctx, cfg.DatabaseURL, cfg.DBPoolSize)
if err != nil { logger.Fatal("db", "error", err) }

v := auth.NewValidator(database.Pool)
hub := realtime.NewHub(ctx, realtime.Options{
    Validator: v,
    RequireAuth: true, // rmhmusic requires auth
    // ...
})
mgr := rmhmusic.NewManager(hub, logger, rmhmusic.NewPgRepo(database))
mgr.Register()
mgr.Start(ctx) // GC reaper + 5s playback heartbeat
```

Because every connection is authenticated, `c.UserID()` is always a real account id — there is no guest fallback to reason about. The Go port composes the four Node managers (RoomManager, SyncEngine, QueueManager, ChatHandler), which in Node shared one `RoomManager` instance, onto a single `Manager` struct that holds the shared state directly.

Snapshot + seq-delta protocol

Event names are ported verbatim from `lib/rmhmusic/events.ts`, carrying the `rmhmusic:` prefix so the existing client adapter speaks them byte-for-byte. As with `rmhtube`, the protocol is **full snapshot on join + seq-numbered deltas thereafter**. On join the server sends a `room:state_snapshot` from `toClientState`; every mutation then goes out as a `room:action` envelope with a per-room monotonic sequence. The delta types are a compact set:

```
actionMemberJoined    = "MEMBER_JOINED"
actionMemberLeft      = "MEMBER_LEFT"
actionMemberDisconnected = "MEMBER_DISCONNECTED"
actionHostTransferred = "HOST_TRANSFERRED"
actionChatMessage     = "CHAT_MESSAGE"
actionNowPlaying      = "NOW_PLAYING"
actionQueueItemAdded  = "QUEUE_ITEM_ADDED"
```

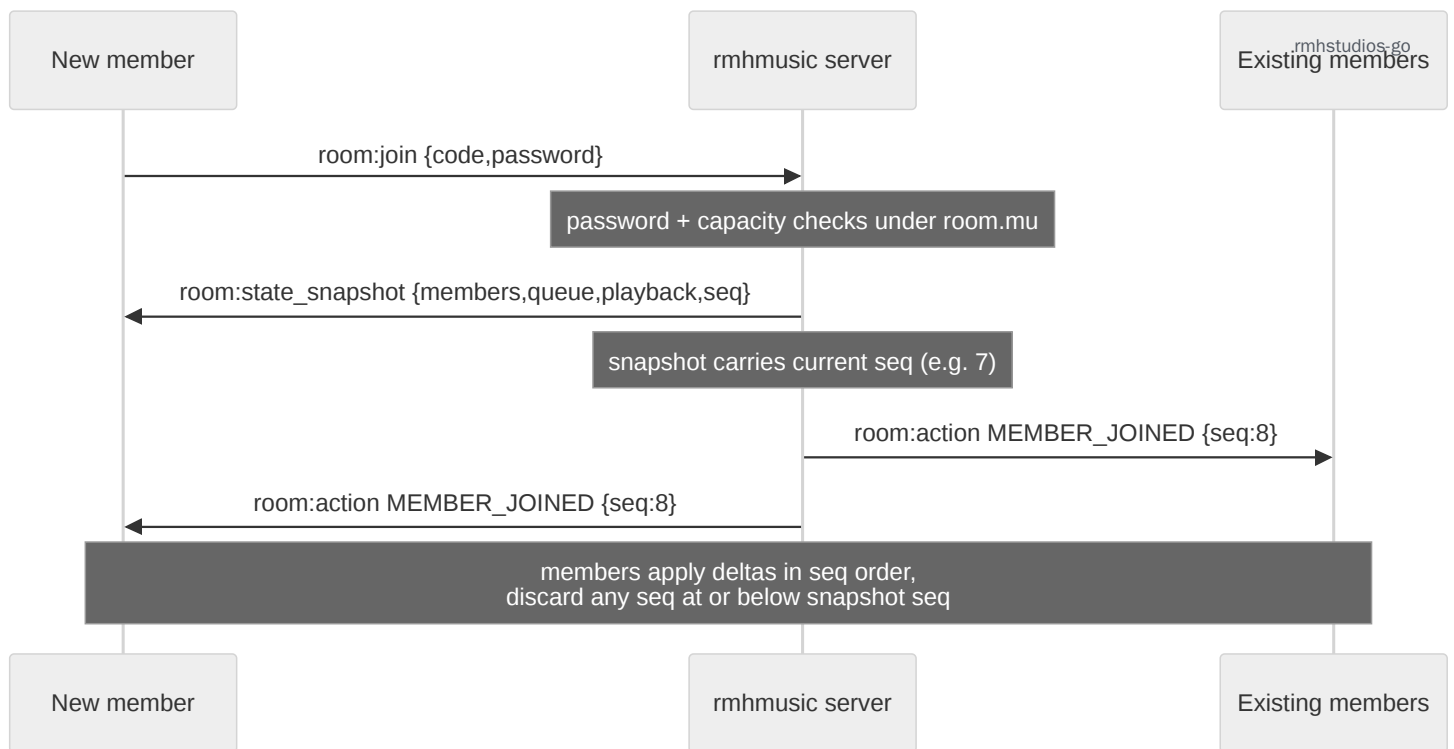


Figure 11.1: snapshot + delta join. The joiner gets a full snapshot stamped with the current seq, then everyone receives the MEMBER_JOINED delta at the next seq.

The broadcast primitive `broadcastAction` increments `room.Seq` under `room.mu` and broadcasts the `{type,payload,seq,timestamp}` envelope:

```

func (m *Manager) broadcastAction(room *ServerRoom, typ string, payload any) {
    room.Seq++
    room.LastActivityAt = m.now()
    m.hub.Broadcast(room.ID, realtime.MustEnvelope(s2cRoomAction, map[string]any{
        "type":    typ,
        "payload":  payload,
        "seq":      room.Seq,
        "timestamp": m.now(),
    })))
}

```

Seq-stamping consistency (per review). The doc comment is explicit that the *inner* payload seq — the room's own counter, what the Node delta carried and what snapshots report — is the single authoritative sequence. This is why `broadcastAction` uses plain `hub.Broadcast` and **not** a `BroadcastSeq` helper: a second, envelope-level seq stamp would be redundant and could diverge from the inner one. This deliberately aligns `rmhmusic` with `rmhtube`. `TestSeqOrderedDeltaNumbering` drives three actions and asserts both the room counter and the snapshot's reported `seq` land at 3.

Host-only playback and drift correction

Playback control is a faithful port of `server/rmhmusic/sync-engine.ts` and is strictly **host-only**: a non-host `play` is rejected with `NOT_HOST`, while non-host `pause` / `seek` / `skip` are silently ignored, exactly as the Node `SyncEngine` behaved. The ephemeral playback head is a small struct — track URI, position in milliseconds, a playing flag, and the `updatedAt` stamp that anchors drift correction:

```

type Playback struct {
    TrackURI    string `json:"trackUri"`
    PositionMs  int64  `json:"positionMs"`
    IsPlaying   bool   `json:"isPlaying"`
    UpdatedAt   int64  `json:"updatedAt"`
}

```

A 5-second heartbeat (`runHeartbeat`) emits `sync:heartbeat` to each *playing* room carrying the drift-corrected position. The correction is the same idea as `rmhtube`'s but in integer milliseconds and with an implicit 1× rate: while playing, the projected position is the stored position advanced by the wall-clock elapsed since `updatedAt` ; while paused, the stored value is returned unchanged:

rmhstudios.go

```
func projectedPositionMs(p Playback, nowMs int64) int64 {
    if !p.IsPlaying {
        return p.PositionMs
    }
    return p.PositionMs + (nowMs - p.UpdatedAt)
}
```

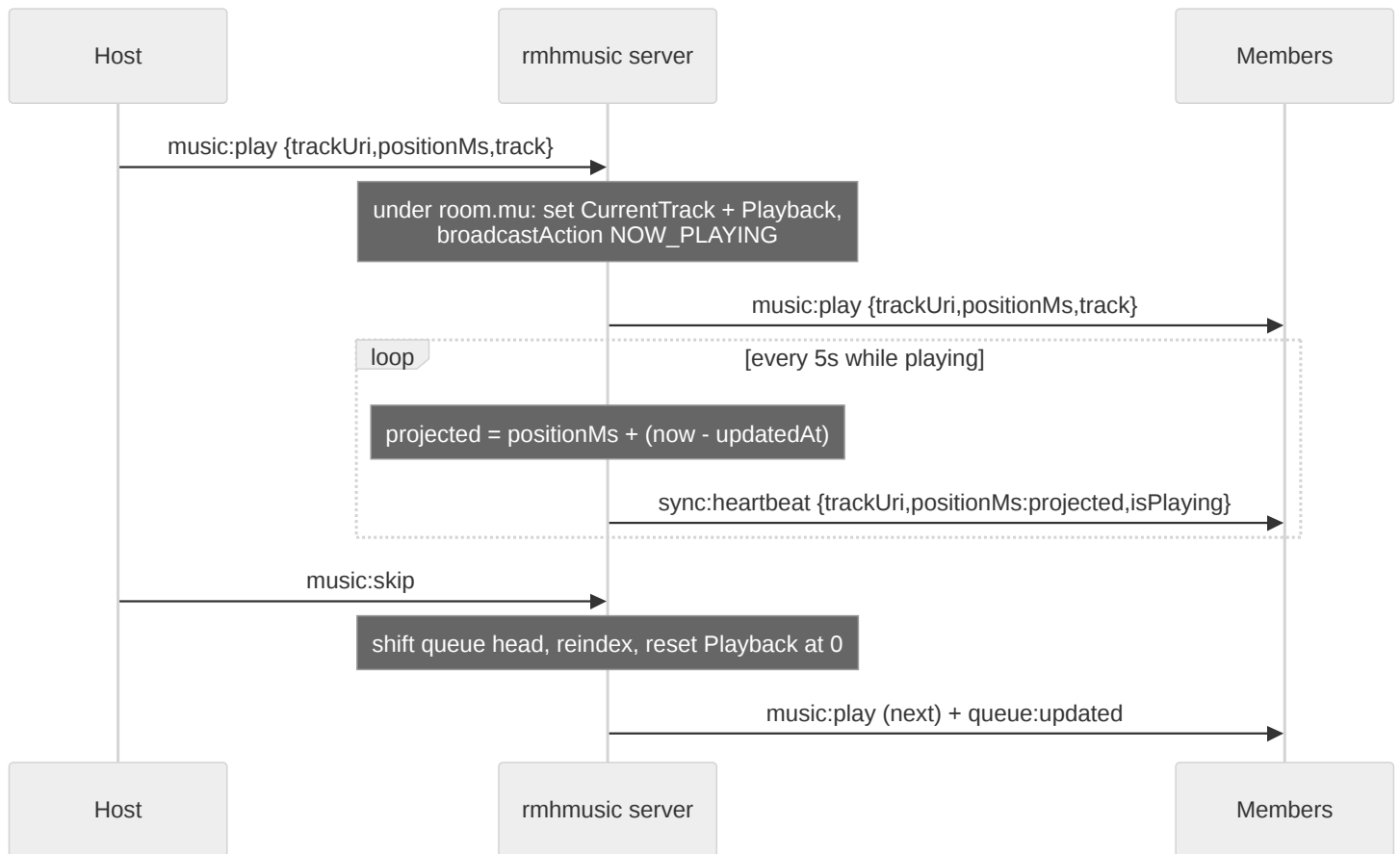


Figure 11.2: host-only playback and the drift-corrected heartbeat.

Three subcases of `TestProjectedPositionMs` pin the math: playing advances by elapsed (`1000 + 3500`), zero-elapsed returns the stored position, and paused returns the stored position regardless of how much time passed. The `onPlay` handler shows the locking discipline that prevents out-of-order track changes: the playback mutation and the `NOW_PLAYING` delta are stamped in one critical section (so a concurrent host control cannot interleave between the state change and the broadcast), and the plain `music:play` fan-out is emitted only after the unlock:

```
room.mu.Lock()
track := p.Track
room.CurrentTrack = &track
room.Playback = Playback{
    TrackURI: p.TrackURI, PositionMs: p.PositionMs,
    IsPlaying: true, UpdatedAt: m.now(),
}
m.broadcastAction(room, actionNowPlaying, map[string]any{"track": track})
room.mu.Unlock()

m.broadcast(room.ID, s2cMusicPlay, map[string]any{
    "trackUri": p.TrackURI, "positionMs": p.PositionMs, "track": track,
})
```

`onSkip` pops the queue head, reindexes, resets the playback head to position 0 for the next track (or clears it and emits `track_changed: nil` when the queue is empty), then broadcasts `music:play` and `queue:updated` .

rmhstudios-go

Queue and chat

The queue (port of `queue-manager.ts`) is deliberately *not* host-gated — any member may add, remove, or reorder, matching the Node `QueueManager`. Items are Spotify tracks (URI, title, artist, album art, duration, preview URL). Add appends to a 50-item-capped queue, broadcasts `queue:updated` with the full queue, then a `QUEUE_ITEM_ADDED` delta. Remove and reorder splice the slice and `reindex` positions 0..n-1. Chat (port of `chat-handler.ts`) sanitizes and truncates content to 300 chars, appends to a 100-message ring buffer, and broadcasts a `CHAT_MESSAGE` delta — a compact subset of `rmhtube`'s chat (no replies, mentions, reactions, or pinning here, faithful to the simpler Node service).

Host auto-reassignment and the memberOrder list

When the host leaves a non-empty room, the host role is auto-reassigned to the *first remaining member in insertion order*. In Node this fell out naturally from `room.members.values().next().value` over an insertion-ordered JS `Map` . Go maps randomize iteration order, so the port maintains an *explicit* `memberOrder []string` alongside the members map; every join appends to it and every removal filters the user out, and `firstMember` walks it:

```
// firstMember returns the first member in insertion order (host
// reassignment target), matching room.members.values().next().value
// over an insertion-ordered JS Map.
func (r *ServerRoom) firstMember() *ServerMember {
    for _, uid := range r.memberOrder {
        if mm, ok := r.Members[uid]; ok {
            return mm
        }
    }
    return nil
}
```

The reassignment happens inside `removeMember` : if the room is now empty it is deleted immediately; otherwise `MEMBER_LEFT` is broadcast, and when the departing user was the host the new host is set and `HOST_TRANSFERRED` announced:

```
if hostLeft {
    if newHost := room.firstMember(); newHost != nil {
        room.HostUserID = newHost.UserID
        m.broadcastAction(room, actionHostTransferred, map[string]any{
            "newHostUserId": newHost.UserID, "newHostUserName": newHost.UserName,
        })
    }
}
```

`TestHostAutoReassignOnHostLeave` builds a room ordered `[host, bob, amy]` , removes `host` , and asserts the new host is `bob` (the first remaining member) and the room survives with two members. A companion test confirms a solo room is reaped immediately on last-member leave.

Concurrency and lifecycle

The manager holds a top-level `sync.RWMutex` over its index maps (`rooms` , `connToRoom` , `userToConn`); each `ServerRoom` additionally carries its own `mu` , and all mutation of a room's fields happens under it — so events on different rooms never contend and events on the same room serialize. `broadcastAction` and `toClientState` both document that the caller must hold `room.mu` .

Disconnect grace. On disconnect the member is marked offline, a `MEMBER_DISCONNECTED` delta is broadcast, and a 30-second grace timer (shared `realtime.GraceTimers`) is armed; a reconnect within the window cancels it. The disconnect handler guards against a stale connection replacing a newer one by checking `member.ConnID == c.ID` before acting, and only clears `userToConn` if the disconnecting conn is still the user's active one. A 60-second GC ticker reaps rooms that have been empty for more than

Persistence: one row, faithfully

This is the sharpest contrast with rmhtube. The `Repo` interface has a single method, `CreateRoom`, and that is the *only* database write the service ever makes. Members, queue, playback, and chat live entirely in memory and are intentionally never written — replicating the Node service exactly. The write is fire-and-forget: a failure is logged, never surfaced to the client.

```
// Repo persists the durable part of a room. NOTE (faithful to Node):
// the original code only persists the room ROW on create -- members,
// queue, playback and chat live entirely in memory.
type Repo interface {
    CreateRoom(ctx context.Context, r RoomRow) error
}
```

The persisted row is the `rmh_music_room` table (`id`, `code`, `name`, `hostId`, `isPublic`, `password`, `maxMembers`, `createdAt`, `updatedAt`), inserted by `PgRepo` with quoted, camelCase column names matching the Prisma / better-auth schema convention. `NopRepo{}` is the no-op used in tests so the room logic — which is fully in-memory and works regardless — is exercisable without a database. Room ids use a 21-char nanoid-style alphabet (`crypto/rand` -backed); join codes use the 6-char unambiguous alphabet.

Aspect	rmhtube	rmhmusic
WS path / port	<code>/rmhtube-ws/</code> · 7003	<code>/rmhmusic-ws/</code> · 7002
Auth	optional (guests allowed)	required
Authority model	leader-authoritative	host-only playback
Heartbeat	2s (host reports ~1s)	5s
Disconnect grace	120s	30s
Persistence	room + members + queue (3 tables)	room row only (1 table)
Restore on startup	yes	no

Scope: faithful to the simpler Node service. rmhmusic is a faithful port of a deliberately leaner feature than rmhtube. The full core — host-only playback with drift-corrected heartbeat, the open (non-host) queue, chat, the snapshot+seq-delta protocol, host auto-reassignment via the explicit `memberOrder`, and the grace + GC lifecycle — is ported behavior-for- behavior. The single-row persistence is not a shortcut but a faithful replication of the Node service, which also kept members / queue / playback / chat purely in-memory; consequently there is no startup room restore (a restart drops live rooms, as it did in Node). The only intentional cross-service alignment beyond the Node original is the seq-stamping consistency note above, which standardizes the inner-payload-seq convention with rmhtube. Nothing here is invented.

12 Service: Doctrine Worker

The **doctrine-worker** is a headless background scheduler — a faithful port of `server/doctrine-worker/index.ts` — that drives the Doctrine Engine's daily content. It owns three independent jobs (puzzle generation, Sahur activation, weekly reputation decay) and exposes no client-facing HTTP at all, only `/health` and `/metrics`. Its defining property is **bit-exact determinism**: the PRNG and seeding logic are ported so that the puzzle the worker records for a date is the same puzzle the web app will regenerate from that seed on demand.

3

SCHEDULED JOBS

5+1

PUZZLE MODES / DAY

0

CLIENT PORTS

bit-exact

PRNG PARITY

Service shape: a port with no client surface

The Node original was a Node process that ran three timers off `setTimeout / setInterval`. The Go port keeps exactly that structure but expresses each timer as a goroutine ticker draining into a shared stop channel. `cmd/doctrine-worker/main.go` wires the common foundation (config, DB wait-for-reachable, telemetry) and then launches the worker; the only HTTP it serves is the operational pair on `METRICS_ADDR`:

```
metrics := telemetry.New("doctrine-worker")
go func() {
    mux := http.NewServeMux()
    mux.HandleFunc("/health", httpx.Health("doctrine-worker", nil))
    mux.Handle("/metrics", metrics.Handler())
    if err := http.ListenAndServe(cfg.MetricsAddr, mux); err != nil {
        // A bind failure leaves the pod unprobeable (k8s liveness hits
        // /health here), so fail loudly rather than run blind.
        logger.Fatal("metrics server", "error", err)
    }
}()

w := doctrine.New(database, logger, metrics)
w.Start(ctx) // launches goroutine tickers
httpx.WaitForSignal() // block until SIGINT/SIGTERM
cancel()
w.Stop()
```

There is no listener on a service port. The Doctrine Engine's read/write API lives in the web app; this worker only *produces* rows the app consumes. That keeps the worker trivially horizontally-isolated — it is a pure side-effecting scheduler.

The three jobs

`Worker.Start` runs `generateDailyPuzzles` once immediately (the startup catch-up run), then spawns three loop goroutines. Each has its own cadence and its own gating rule, faithful to the Node `startup()`:

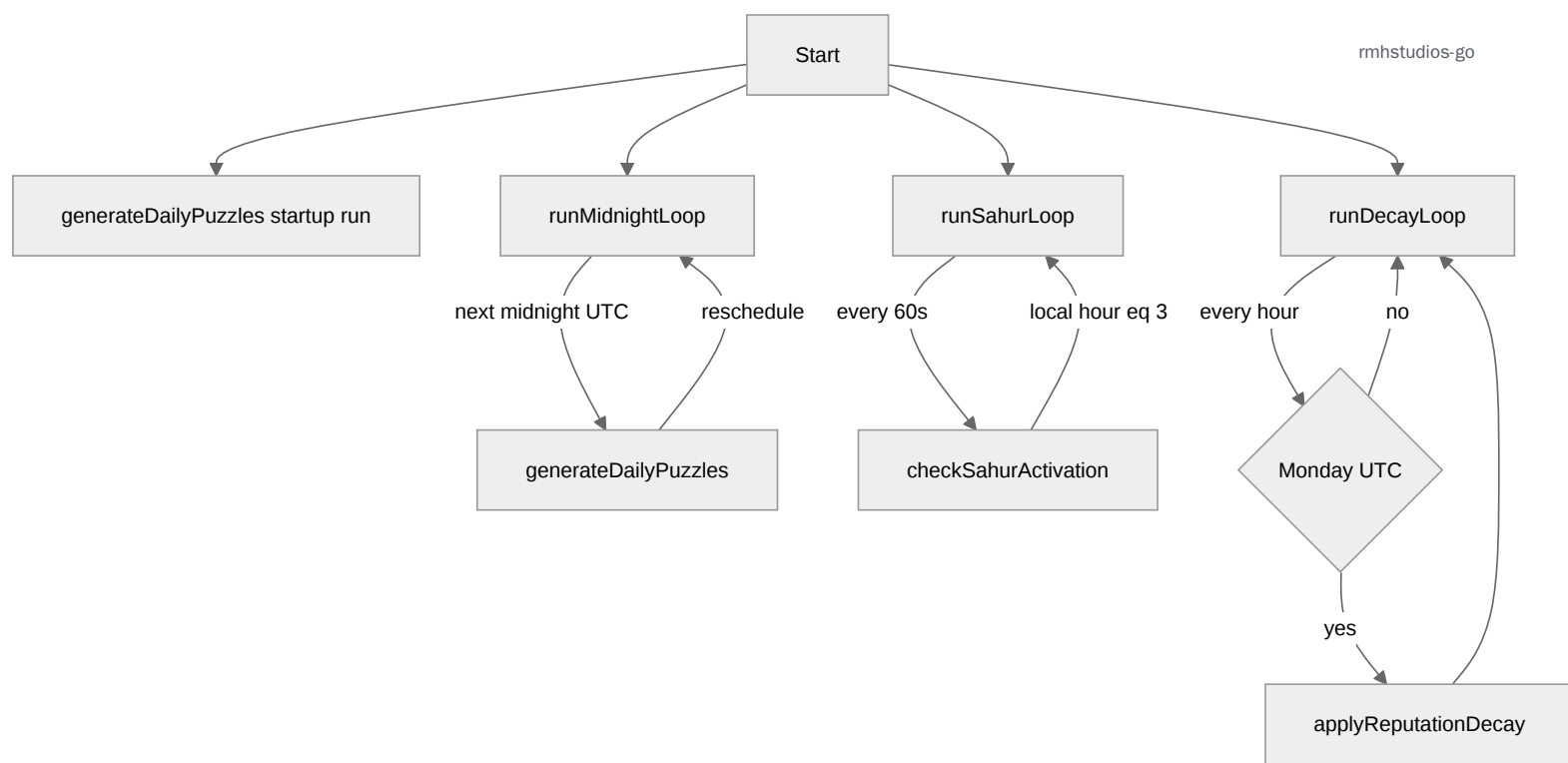


Figure 12.1: the three scheduled jobs and their gating.

Job	Cadence	Gate	Effect
generateDailyPuzzles	startup + each midnight UTC (self-rescheduling)	skip modes whose row exists	insert 5 mode puzzles + SAHUR_SPECIAL for tomorrow
checkSahurActivation	every 60 s	per-timezone local hour == 3	insert one sahur_session per (today, tz)
applyReputationDecay	every hour	Mondays only (UTC)	-5% XP for users inactive > 7 days, +ledger row

Self-rescheduling midnight loop

The Node code recursively re-armed a `setTimeout(schedulePuzzles, ...)` at each midnight. The Go port computes the duration to the next UTC midnight, arms a one-shot timer, runs the job, then loops to recompute — the same recursive self-rescheduling, but leak-free under cancellation:

```

func (w *Worker) runMidnightLoop(ctx context.Context) {
    defer w.wg.Done()
    for {
        d := millisUntilMidnightUTC(time.Now().UTC())
        timer := time.NewTimer(d)
        select {
        case <-ctx.Done():
            timer.Stop(); return
        case <-w.stop:
            timer.Stop(); return
        case <-timer.C:
            w.runJob(ctx, "daily_puzzles", w.generateDailyPuzzles)
        }
    }
}

```

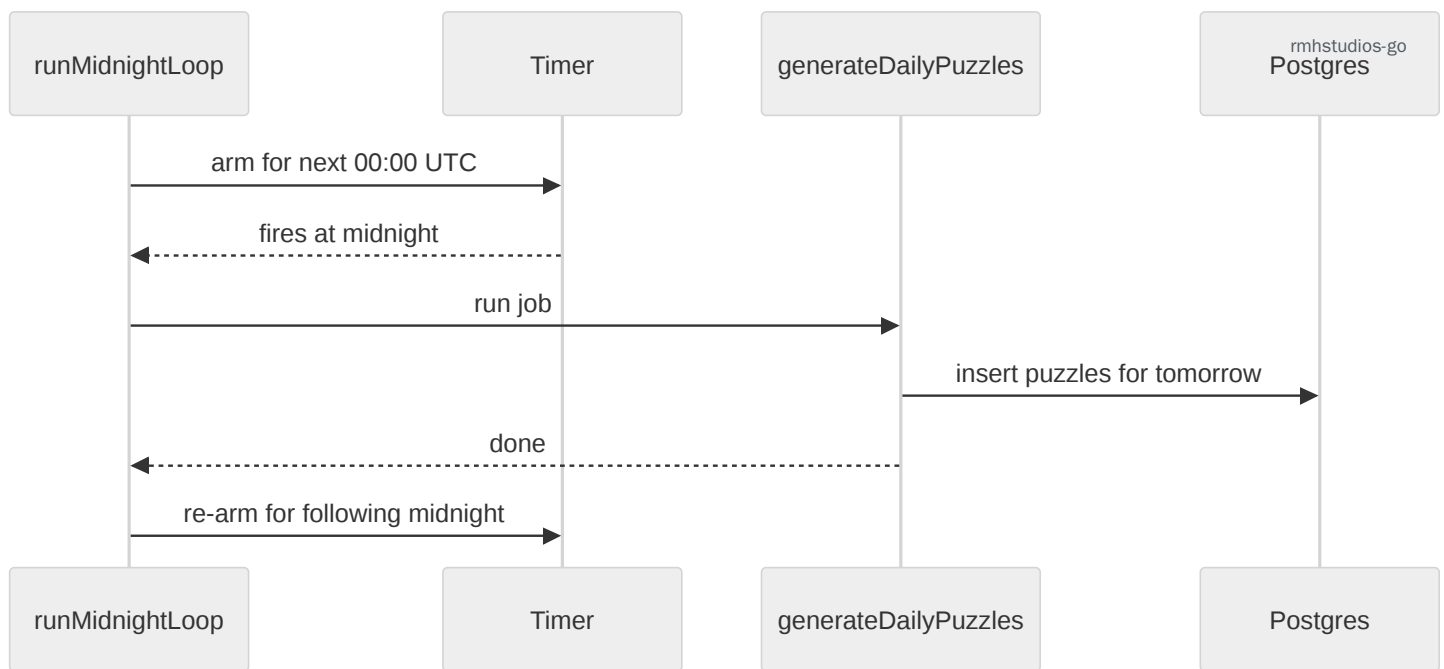


Figure 12.2: the midnight reschedule cycle.

The decay loop is the inverse: it ticks hourly but the Monday gate lives in the scheduler, not the job, matching the Node `setInterval(... if (isMonday()) ..., 3_600_000)`:

```

case <-ticker.C:
  if time.Now().UTC().Weekday() == time.Monday {
    w.runJob(ctx, "reputation_decay", w.applyReputationDecay)
  }

```

Determinism highlight: a bit-exact PRNG port

This is the heart of the worker and the reason the port had to be exact. The web app and the worker must agree on puzzle content: the worker stores a `seed`, and the app later regenerates the full puzzle (board layout, clue ordering, etc.) by feeding that same seed into the *same* PRNG. If the Go PRNG diverged from the original Node `mulberry32` by even one bit, the worker's stored seed would reconstruct a different puzzle in the browser.

The challenge is that JavaScript's number semantics — `| 0` truncation, `Math.imul` signed 32-bit multiply, `>>>` unsigned shift — must be reproduced in Go's typed integer world. The port does this with deliberate `int32 / uint32` conversions:

```

func Mulberry32(seed int) func() float64 {
  s := int32(seed) // 'seed | 0' truncates to signed 32-bit.
  return func() float64 {
    s = s + int32(0x6D2B79F5) // wrapping 32-bit add
    us := uint32(s)
    // t = Math.imul(s ^ (s >>> 15), 1 | s)
    t := imul(int32(us^(us>>>15)), int32(uint32(1)|us))
    // t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t
    ut := uint32(t)
    t = (t + imul(int32(ut^(ut>>>7)), int32(uint32(61)|ut))) ^ t
    // return ((t ^ (t >>> 14)) >>> 0) / 4294967296
    ut = uint32(t)
    return float64(ut^(ut>>>14)) / 4294967296.0
  }
}

func imul(a, b int32) int32 { return int32(uint32(a) * uint32(b)) }

```

The seed itself comes from a string hash over `"<date>:<mode>:doctrine"` using the classic `(hash << 5) - hash + c` rolling hash with 32-bit wrap, then `Math.abs`. Mode names are lowercased at the call site to match the Node `mode.toLowerCase()`:

```
func GetSeedForDate(date, mode string) int {
    var hash int32
    input := date + ":" + mode + ":doctrine"
    for i := 0; i < len(input); i++ {
        hash = (hash<<5 - hash + int32(input[i]))
    }
    if hash < 0 {
        return int(-int64(hash)) // guard against MinInt32
    }
    return int(hash)
}
```

Golden-vector tested. `seed_test.go` locks both functions to reference values produced by running the actual Node code. For 2026-06-19 the seeds must be `alibi=489762987`, `spectrum=1864840003`, `outcast=1313135021`, `chainlink=49484481`, `impostor=1589193561`, `sahur_special=1827053367`; `Mulberry32(0)`'s first three draws must equal `0.26642920868471265`, `0.00032974570058286`, `0.22327202744781971` (tolerance `1e-15`, i.e. float-literal rounding only). These are not approximate checks — the integer arithmetic is identical, so the outputs are identical.

Difficulty by weekday

Difficulty is a fixed map over the UTC weekday (0 = Sunday, matching JS `Date.getUTCDay`), with a `?? 3` default. `SAHUR_SPECIAL` derives from the base via `min(5, base+1)`:

```
func DifficultyForWeekday(weekday int) int {
    switch weekday {
    case 1: return 1 // Mon
    case 2: return 2 // Tue
    case 3: return 3 // Wed
    case 4: return 3 // Thu
    case 5: return 4 // Fri
    case 6: return 4 // Sat
    case 0: return 5 // Sun
    default: return 3
    }
}

func SahurDifficulty(base int) int { return int(math.Min(5, float64(base+1))) }
```

The five regular modes are an ordered set; `SAHUR_SPECIAL` is intentionally generated separately (it is not in the `Modes` slice), exactly as in the source:

```
var Modes = []string{"ALIBI", "SPECTRUM", "OUTCAST", "CHAINLINK", "IMPOSTOR"}
const SahurSpecialMode = "SAHUR_SPECIAL"
```

Each generation run targets *tomorrow* (now + 1 day, sliced to `YYYY-MM-DD`), computes one difficulty for the day, and for each mode checks existence before inserting — so re-running (startup after a restart) is idempotent. The persisted JSONB mirrors the Node shape exactly: `{ "generated": true, "seed": <seed> }`.

Per-timezone Sahur activation

Every 60 seconds, `checkSahurActivation` pulls the distinct non-null `doctrineTimezone` values from the `user` table and, for each timezone whose *local* hour is currently 3 AM, opens a `sahur_session` for that (today, timezone) pair if one does not exist. Unknown timezones return `false`, matching the Node try/catch fallback:

```
func isSahurHour(timezone string, t time.Time) bool {
    loc, err := time.LoadLocation(timezone)
    if err != nil {
        return false
    }
    return t.In(loc).Hour() == 3
}
```

rmhstudios-go

Weekly reputation decay

On Mondays, decay finds reputations with `lastActiveAt < now-7d` and `totalXp > 0`, multiplies XP by `1 - 0.05` (Go integer truncation equals `Math.floor` for non-negatives), updates the row, and writes a `WEEKLY_DECAY` ledger entry recording the (negative) delta:

```
newXp := int(float64(rep.totalXp) * (1 - weeklyDecayRate))
delta := newXp - rep.totalXp
// UPDATE doctrine_reputation SET totalXp ...
// INSERT doctrine_reputation_ledger (action=WEEKLY_DECAY, xpDelta=delta, ...)
```

Scope — UTC vs local cutoff. The Node code computed `oneWeekAgo` with a local `Date`. The Go port deliberately uses UTC (`time.Now().UTC().AddDate(0,0,-7)`) so the whole worker is consistent with its UTC-based scheduling. In practice this only shifts the 7-day boundary by the host's offset; for a worker that runs in a UTC container the behavior is identical.

Tables written

Table	Key columns	Notes
doctrine_puzzle	id, mode, date, seed, data (JSONB), difficulty, resetsAt, isSahur, createdAt	uniqueness enforced on (mode, date) via existence check before insert
doctrine_sahur_session	id, dateKey, timezone, activatedAt, participantCount	one per (dateKey, timezone); count defaults to 0
doctrine_reputation	id, totalXp, lastActiveAt	read for decay candidates, XP updated in place
doctrine_reputation_ledger	id, reputationId, action, xpDelta, reason, metadata, createdAt	metadata left NULL for decay rows

Scope — UUID primary keys vs cuid. The Prisma schema issued `cuid()` IDs. The Go worker generates `uuid.NewString()` for every inserted row. Both are opaque, collision-resistant string PKs and the columns are plain text, so nothing in the schema or the consuming app depends on the format — but it is an honest substitution worth noting: rows created by the Go worker are visibly UUIDs, rows created by the legacy Node path are cuids.

Observability

Every job run increments `JobRuns{job, ok|error}` via the `runJob` wrapper (replacing the Node `.catch(console.error)` pattern), and every query increments `DBQueries{ok|error}`. Combined with `/health`, this gives k8s liveness and Prometheus enough to alert on a stalled scheduler even though the worker has no request traffic of its own.

The **recap** service ports `server/recap/index.ts` — the Lights Out daily-recap runner. Every five minutes it finds Discord channels whose recap is due, builds a rich embed summarising that guild's players for the recap date (medals, ratings, ranking), posts it, and clears the schedule. It also holds a Discord gateway connection so the bot shows as online. It serves `/health` and `/metrics` on `RECAP_PORT` (default `7004`).

5 min POLL INTERVAL	7004 RECAP_PORT	2 INTERFACES (REPO, POSTER)	always CLEARS ON POST
-------------------------------	---------------------------	--	---------------------------------

The 5-minute due-recap loop

`CHECK_INTERVAL_MS = 5 * 60 * 1000` becomes `checkInterval = 5 * time.Minute`. `Start` opens the gateway, runs one recap pass immediately (mirroring the Node `processDueRecaps()` kick before the interval), then ticks. Each tick calls `processDueRecaps`, which fetches every due channel and processes them one at a time, logging — but not aborting on — per-guild failures.

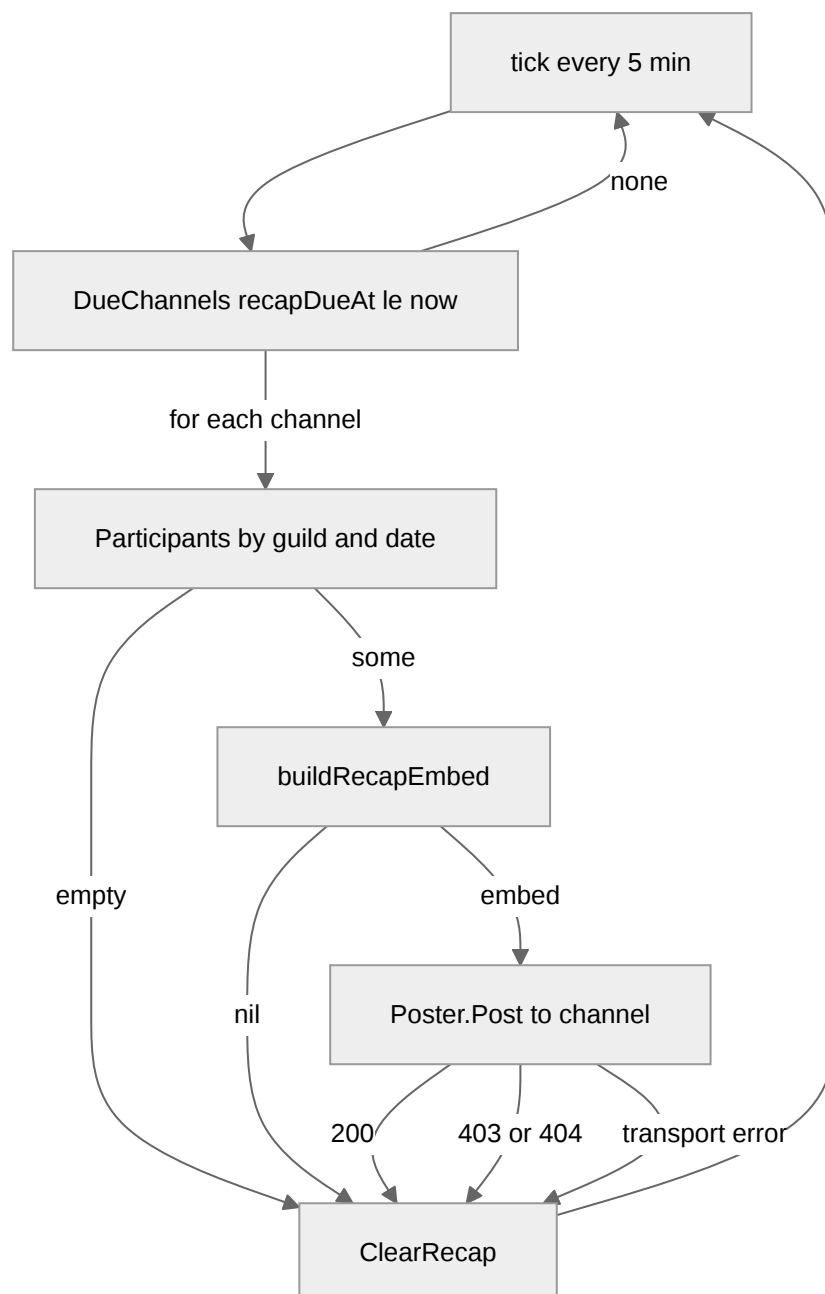


Figure 13.1: the due-recap loop — the schedule is cleared on every path, including access errors.

The "always clear" behavior is the subtle, faithful part. Whether the post succeeds, the bot is forbidden (403), the channel is gone (404), or the transport itself errors, the recap schedule for that channel is cleared so it never retries forever. ^{rmhstudios-go}

```
status, postErr := r.poster.Post(ctx, ch.ChannelID, msg)
switch {
case postErr != nil:
    r.log.Warn("recap post error", "guildId", ch.GuildID, "error", postErr)
case status == 403 || status == 404:
    r.log.Info("bot lacks access - clearing recap", "guildId", ch.GuildID, "status", status)
default:
    r.log.Info("posted recap", "guildId", ch.GuildID, "channelId", ch.ChannelID)
}
// Always clear the recap schedule (faithful to the Node comment).
return r.repo.ClearRecap(ctx, ch.ID)
```

The due-channel query is the literal port of the Node predicate (`activity='lights-out'` with a non-null, past `recapDueAt` and a non-null `recapDateKey`):

```
SELECT "id", "guildId", "channelId", "recapDateKey"
FROM "discord_activity_channel"
WHERE "activity" = 'lights-out'
AND "recapDueAt" IS NOT NULL
AND "recapDueAt" <= $1
AND "recapDateKey" IS NOT NULL
```

Building the embed: medals, ratings, ranking

Participants come from `discord_daily_participant` ordered by `moves ASC NULLS LAST` (completers with the fewest moves first; NULL-move players last), matching Prisma's `orderBy moves:'asc'` on Postgres. `buildRecapEmbed` then reproduces the Node embed exactly: the top three completers get 🥇🥈🥉, fourth onward get a plain bullet, each line shows the rating emoji (default 🤖) and optional rating label, and a "did not finish" line counts the players still `playing`:

```
for i, p := range completed {
    medal := "" // plain bullet for 4th+
    if i < len(medals) {
        medal = medals[i] // 🥇🥈🥉
    }
    emoji := "\U0001F4A1" // 🤖 default
    if p.RatingEmoji != nil && *p.RatingEmoji != "" {
        emoji = *p.RatingEmoji
    }
    line := fmt.Sprintf("%s **%s** - %s %d move%s", medal, p.Username, emoji, moves, plural(moves))
    if p.RatingLabel != nil && *p.RatingLabel != "" {
        line += fmt.Sprintf(" (%s)", *p.RatingLabel)
    }
    lines = append(lines, line)
}
```

The description is assembled with blank separators then run through `filterNonEmpty`, which removes *all* empty strings — the exact behavior of the Node `.filter(Boolean)` over the whole array. The embed color is the amber `0xf59e0b` from the source, and when an `APP_ID` is configured a "Play today's puzzle" link is appended pointing at `discord.com/activities/<APP_ID>`. The leaderboard image URL is rebuilt the same way the Node runner did, from `SITE_URL` plus the guild and date:

```
siteURL + "/api/discord/activity-image?type=leaderboard&guildId=" +
guildID + "&dateKey=" + dateKey + "&recap=1"
```

`buildRecapEmbed` returns `nil` for an empty participant list, so the caller just clears the schedule — the Node `if (participants.length === 0)` early-out. Pluralization is the faithful `n != 1 ? 's' : ''` rule (the test asserts "1 move" but never "1 moves").

Porting the Lights Out puzzle math

rmhstudios-go

The recap header shows the day's shape and the optimal move count, which means the runner must recompute the puzzle from the date. `lightsout.go` ports just the pieces needed for `(shapeLabel, optimalMoves)`: the Mulberry32 RNG, the `SHAPES` table (order matters — the seed indexes into it), `generatePuzzle` (difficulty scaling, the 100-attempt retry loop, deterministic fallback), and a brute-force `solvePuzzle` for the minimum solution:

```
func computePuzzleMeta(dateKey string) puzzleMeta {
    y, m, d, ok := parseDateKey(dateKey)
    if !ok {
        return puzzleMeta{shapeLabel: dateKey, optimal: -1}
    }
    seed := getDateSeed(y, m, d) // y*10000 + m*100 + d
    shape := getDailyShape(seed) // seed % len(shapes)
    label := getShapeLabel(shape)
    puzzle := generatePuzzle(createSeededRng(seed), shape)
    optimal := getOptimalMoves(puzzle, shape) // -1 == omit "Optimal" line
    return puzzleMeta{shapeLabel: label, optimal: optimal}
}
```

Scope — only the recap-facing helpers are ported. The interactive play helpers (cell toggling for live play, share/persistence) are deliberately out of scope: the runner only needs to *describe* the day's puzzle, not let anyone play it. `recap_test.go` verifies `computePuzzleMeta` is deterministic and that the seeded RNG reproduces its sequence (same seed → identical draws in `[0,1)`).

Gateway presence: discordgo replaces hand-rolled opcodes

This is the clearest improvement over the Node runner. The original hand-rolled the Discord gateway: op-10 hello, op-1 heartbeat on a `setInterval`, op-2 identify, op-7 reconnect, op-9 invalid-session, and a 30-second reconnect timer it had to maintain itself. The Go port hands all of that to `discordgo`:

```
session, err = discordgo.New("Bot " + botToken)
session.Identify.Intents = discordgo.IntentsNone // index.ts uses intents: 0
session.Identify.Presence = discordgo.GatewayStatusUpdate{
    Status: "online",
    Game:   discordgo.Activity{Name: "RMHBox", Type: discordgo.ActivityTypeGame},
}
// ... later, in Runner.Start:
if err := r.session.Open(); err != nil { ... }
```

Improvement. `session.Open()` manages the handshake, heartbeats, reconnect, and RESUME. The op-10/op-1/op-7/op-9 logic, the heartbeat interval, and the manual reconnect timer that `index.ts` maintained are gone. Posting uses the same library's REST path (`ChannelMessageSendComplex`), and the production `Poster` recovers the HTTP status from a `*discordgo.RESTError` so 403/404 are treated as non-fatal skips — preserving the exact Node semantics with far less code. When no bot token is set, the runner still serves health/metrics and simply skips posting (the Node `if (!BOT_TOKEN)` guard, minus the exit).

Testing with fakes

The orchestration sits behind two small interfaces so it tests without a real DB or Discord. `Repo` covers `DueChannels`, `Participants`, and `ClearRecap`; `Poster` wraps the single send. The tests assert the three behaviors that matter:

Test	Asserts
RankingAndMedals	🏆🏆🏆 then plain bullet, default 💡 vs supplied emoji, rating-label suffix, "N did not finish", play link, image URL
PostsAndClears	one post to the channel, then the channel cleared
NoParticipants_ClearsWithoutPosting	no post, but schedule still cleared
ForbiddenStillClears	a 403 from the poster still clears the schedule

Per-job outcomes feed `JobRuns{recap, ok|error}` ; DB calls feed `DBQueries{ok|error}` . With the gateway connection double-purposed as the bot's online presence, a single process covers both the scheduled posting and the always-online requirement the Node runner served.

The **vibe-worker** renders gallery thumbnails for AI-generated "vibe pages" — arbitrary interactive HTML (esbuild-bundled React, canvas, three.js from esm.sh). It ports `server/vibe-worker/index.ts` and `lib/rmhvibe/vibe-screenshot.server.ts` : every 10 seconds it picks up to five stale pages, renders each headlessly in Chromium, writes a downscaled PNG to the shared `db/vibe-thumbs/<slug>.png` volume, and clears the stale flag with optimistic concurrency. There is no client HTTP surface — only `/health` and `/metrics` on `METRICS_ADDR` .

10 s

POLL INTERVAL

5

PAGES PER BATCH

10 s

PER-RENDER TIMEOUT

640px

THUMBNAIL WIDTH

Poll → render → optimistic clear

The worker decouples headless Chromium from the web app's request path. Its loop runs one pass immediately on start (the Node `void processStale()` kick) then ticks every `pollInterval` . Each pass selects the oldest stale pages, renders each, and clears the flag only if the page hasn't changed underneath the render:

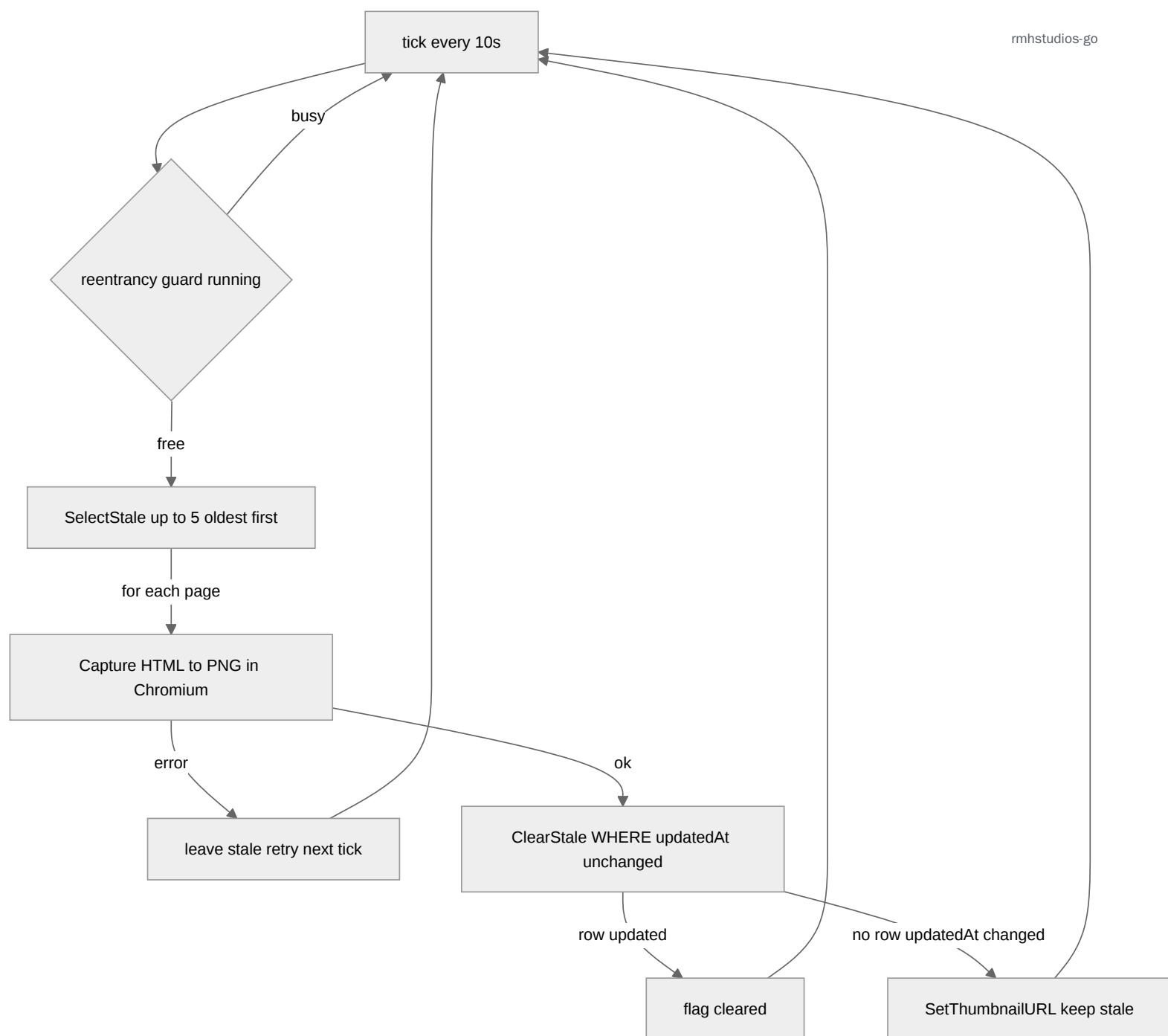


Figure 14.1: poll → render → optimistic clear, with the mid-render change fallback.

Reentrancy guard

A batch of five Chromium renders can easily outlast the 10-second tick, so the worker carries a `running` flag (the Node `running` boolean) guarded by a mutex. A tick that fires while a batch is in flight is a no-op:

```

func (w *Worker) processStale(ctx context.Context) {
    w.mu.Lock()
    if w.running {
        w.mu.Unlock()
        return // overlapping tick - skip
    }
    w.running = true
    w.mu.Unlock()
    defer func() { w.mu.Lock(); w.running = false; w.mu.Unlock() }()
    ...
}

```

The select query is the literal port of the Prisma `findMany({ where: thumbnailStale, orderBy: updatedAt asc, take: 5 })`:

```
SELECT "id", "slug", "html", "updatedAt"
  FROM "vibe_page"
 WHERE "thumbnailStale" = true
 ORDER BY "updatedAt" ASC
 LIMIT $1
```

rmhstudios-go

Optimistic concurrency clear

The page's HTML can be edited by a user *while* we are rendering the previous version. The worker reads `updatedAt` with the row and, after capturing, only clears the stale flag when `updatedAt` is still unchanged — a compare-and-set in SQL:

```
UPDATE "vibe_page"
  SET "thumbnailStale" = false, "thumbnailUrl" = $1
 WHERE "id" = $2 AND "updatedAt" = $3 -- expected updatedAt
```

If no row was affected, the content changed mid-render. The worker still records the freshly-captured URL (so the gallery shows *something*) via `SetThumbnailURL` but leaves the flag set so the newest content re-renders next tick:

```
cleared, err := w.repo.ClearStale(ctx, page.ID, url, page.UpdatedAt)
if cleared {
    return true // clean render
}
// Page changed mid-render: record URL, keep it stale.
if err := w.repo.SetThumbnailURL(ctx, page.ID, url); err != nil {
    return false
}
return true
```

Batch outcome is recorded honestly: `ok` when the batch was empty or every render succeeded, `error` when at least one was attempted and none succeeded, and `partial` when some succeeded and some failed — feeding `JobRuns{vibe_thumbnails, ok|partial|error}`.

Headless capture with chromedp

The `Capturer` interface hides the rendering so the loop can be tested with a fake. The production implementation drives headless Chromium via `chromedp` with container-safe flags mirroring the Node launch args:

```
err := chromedp.Run(runCtx,
    chromedp.EmulateViewport(viewportWidth, viewportHeight, chromedp.EmulateScale(thumbDeviceScale)),
    chromedp.Navigate(dataURL),
    chromedp.Sleep(settleDelay), // modules + first paint + intro animations
    chromedp.CaptureScreenshot(&png),
)
```

Scope — chromedp replaces Playwright. The Node worker drove Chromium through Playwright. The Go port uses `chromedp` (the CDP client) to the same end: navigate, settle, screenshot. The container-safe flags (`NoSandbox` , `DisableGPU` , `disable-dev-shm-usage` , `headless`) mirror the Playwright launch args; `execPath` honors `PLAYWRIGHT_CHROMIUM_EXECUTABLE_PATH` in production and is empty locally so `chromedp` discovers an installed Chrome.

Runtime requirement. A Chromium/Chrome binary *must* be present at runtime — there is no fallback renderer. The production Dockerfile installs Chromium and points `PLAYWRIGHT_CHROMIUM_EXECUTABLE_PATH` at it; without a browser, every capture fails and pages stay stale.

Review fix: base64-encoded data URL

The page is rendered from a `data:` URL rather than a temp file, so the worker never touches the filesystem for input. The HTML is arbitrary and routinely contains `#` (a fragment delimiter) and `%` (a percent-escape introducer), both of which truncate or mis-decode in a plain `data:` URL. The payload is therefore base64-encoded so Chromium decodes the exact bytes:

Review fix. Base64 makes the URL opaque, eliminating the `# / %` truncation class of bugs entirely — a correctness improvement over a naive inline data URL.

A real per-render timeout

Animation-heavy pages may never reach network-idle, and a sub-resource that never finishes could hang a render forever. Each capture runs under a real deadline (`captureTimeout = 10s`), comfortably above the `settleDelay` of 1.5s so a render is never cut off mid-settle. On timeout the run is cancelled and the page is left stale to retry — the batch is never blocked indefinitely:

```
runCtx, cancelRun := context.WithTimeout(taskCtx, captureTimeout)
defer cancelRun()
```

Downscaling: Chromium device scale instead of sharp

Scope — substitution for sharp. The Node code downscaled the raw screenshot with the `sharp` image library (resize to `thumbWidth` , PNG compression level 9). With no sharp-equivalent in the allowed dependency set, the Go port instead asks Chromium itself to rasterize smaller: it sets a fractional device scale factor (`thumbWidth/viewportWidth`) on the emulated viewport. The layout viewport stays 1200×750 so pages lay out identically, but the PNG comes out ~640px wide. This achieves the faithful intent — a small gallery PNG — with chromedp's own scaling rather than a post-capture resize.

```
const thumbDeviceScale = float64(thumbWidth) / float64(viewportWidth)
// EmulateViewport(1200, 750, EmulateScale(thumbDeviceScale))
```

On success the PNG is written to `<thumbDir>/<slug>.png` and the worker returns a cache-busted URL the web app serves at `/api/vibe/thumb/{slug}?v=<unixMillis>` .

Repo behind an interface; tested with fakes

Both the renderer (`Capturer`) and the data layer (`Repo`) are interfaces; the production `PGRepo` and `ChromedpCapturer` are injected, and `worker_test.go` supplies fakes that model the optimistic-concurrency behavior:

Test	Asserts
<code>HappyPath_ClearsFlagOptimistically</code>	both pages captured and cleared with the right URLs; no mid-render fallback used
<code>ChangedMidRender_KeepsStaleButRecordsURL</code>	when <code>updatedAt</code> advanced, <code>ClearStale</code> misses and <code>SetThumbnailURL</code> records the URL
<code>CaptureFailure_LeavesPageUntouched</code>	a capture error touches neither the cleared nor the set path
<code>ReentrancyGuard</code>	with <code>running=true</code> , a tick performs no <code>SelectStale</code> at all

The fake `Capturer` can be told to fail per-slug, and the fake `Repo` tracks a current `updatedAt` per id so `ClearStale` only "succeeds" on an exact match — exercising the compare-and-set without a database. `Close()` on the production capturer is a no-op: a fresh browser is allocated per capture and torn down by deferred cancels, so there is no shared browser to release.

The `discord-bot` service is the Go port of `server/discord-bot/*.ts` : a long-running Discord *gateway* bot (no client-facing HTTP) built on `bwmarrin/discordgo` . It powers two features — the `/chat` AI persona ("Alex Wu"), which is faithfully ported end-to-end, and the `/rmhbot` agentic website editor, which is ported as an honest *scaffold*. This section documents the gateway lifecycle, command registration, interaction routing, the DeepSeek client, the chat embed-packing logic, the rmhbot agentic tool loop, and the security guards — quoting the real code throughout.

1 GATEWAY INTENT (GUILDS)	4 SLASH COMMANDS	6000 CHAR EMBED BUDGET	40 MAX TOOL ROUNDS	5 min SESSION AUTO- UNLOCK	9 UNIT TESTS
---	-------------------------------	-------------------------------------	---------------------------------	---	------------------------

Scope at a glance. `/chat` (persona, DeepSeek client, embed packing, DB session persistence) is a **faithful 1:1 port**. `/rmhbot` is **deliberately scaffolded**: its command/interaction structure, session model, locking, path-traversal & safety guards, git operations and tool-loop skeleton are real and tested, but `run_typecheck` , `createPullRequest` , `search_code` , and attachment ingestion are explicit stubs. The separation is called out inline below and never hidden.

Gateway lifecycle & bootstrap

Unlike every other service in this report, the bot exposes *no* client port. It connects outbound to Discord's gateway and serves only `/health` and `/metrics` on `MetricsAddr` . The `main()` entrypoint (`cmd/discord-bot/main.go`) follows the FOUNDATION worker skeleton: resolve config from env, wait for the database, start the metrics server, construct services, then block on `Run(ctx)` until a signal cancels the context.

```
// cmd/discord-bot/main.go - metrics-only HTTP; bind failure is fatal.
go func() {
    mux := http.NewServeMux()
    mux.HandleFunc("/health", httpx.Health("discord-bot", nil))
    mux.Handle("/metrics", metrics.Handler())
    if err := http.ListenAndServe(cfg.MetricsAddr, mux); err != nil {
        logger.Fatal("metrics server", "error", err) // unprobeable => k8s restart
    }
}()

botCfg := discordbot.Config{
    Token:      firstNonEmpty(config.GetString("DISCORD_BOT_TOKEN", ""),
                               config.GetString("DISCORD_ACTIVITY_BOT_TOKEN", "")),
    DevGuildID: config.GetString("DISCORD_DEV_GUILD_ID", ""),
    OwnerID:    config.GetString("OWNER_ID", ""),
    DeepSeekKey: config.GetString("DEEPSEEK_API_KEY", ""),
    DeepSeekMod: config.GetString("DEEPSEEK_MODEL", "deepseek-chat"),
    WorktreesDir: config.GetString("RMHBOT_WORKTREES_DIR", ""),
    GithubToken: config.GetString("GITHUB_TOKEN", ""),
}
```

The token resolution mirrors the TS code: `DISCORD_BOT_TOKEN` wins, falling back to `DISCORD_ACTIVITY_BOT_TOKEN` . The DB is used for `/chat` session persistence; the chat handler tolerates a nil DB but `main` still waits for reachability like the other workers.

The session is created with the **Guilds intent only** — the bot reacts to interactions (slash commands, buttons, modals), which do not require message-content or member privileged intents:

```
// bot.go - New()
session, err := discordgo.New("Bot " + cfg.Token)
session.Identify.Intents = discordgo.IntentsGuilds
b := &Bot{cfg: cfg, logger: logger, session: session, chat: chat, rmhbot: rmhbot}
session.AddHandler(b.onReady)
session.AddHandler(b.onInteraction)
```

`Run` records the lifecycle context, opens the gateway, blocks on `ctx.Done()`, then closes the session cleanly — graceful shutdown:

```
func (b *Bot) Run(ctx context.Context) error {
    b.ctxMu.Lock(); b.lifecycleCtx = ctx; b.ctxMu.Unlock()
    if err := b.session.Open(); err != nil { return fmt.Errorf("open gateway: %w", err) }
    b.logger.Info("discord gateway opened")
    <-ctx.Done()
    b.logger.Info("shutdown_start")
    if err := b.session.Close(); err != nil { b.logger.Warn("session close", "error", err) }
    b.logger.Info("shutdown_complete")
    return nil
}
```

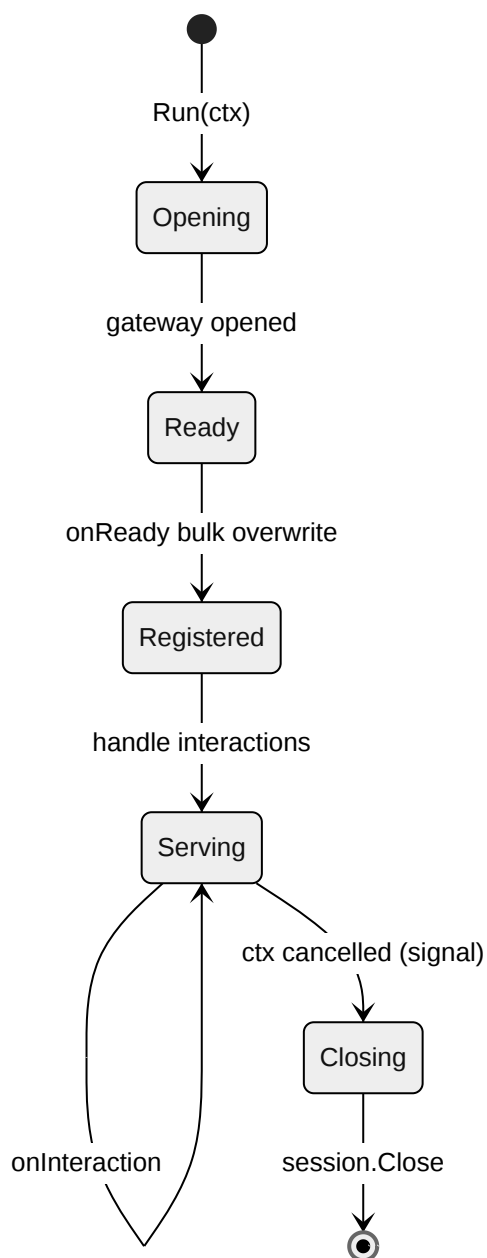


Figure 15.1: the bot's worker lifecycle, driven by the discordgo session.

Command registration on ready

On the `READY` event the bot bulk-overwrites its slash commands. Registration is **guild-scoped** when `DISCORD_DEV_GUILD_ID` is set (instant propagation for development) and **global** otherwise. A subtle but important detail is preserved from the TS: a bulk overwrite cannot delete Discord's *Entry Point* command (type 4, used by Activities) and will fail with error `50240` if you try. So the bot first fetches existing commands and re-appends any Entry Point command to the desired set:

```
// bot.go - onReady()
guildID := b.cfg.DevGuildID // "" => global
desired := slashCommands()

existing, err := s.ApplicationCommands(s.State.User.ID, guildID)
if err == nil {
    for _, c := range existing {
        if int(c.Type) == 4 { // PrimaryEntryPoint - bulk overwrite cannot remove it
            desired = append(desired, c)
        }
    }
}
if _, err := s.ApplicationCommandBulkOverwrite(s.State.User.ID, guildID, desired); err != nil {
    b.logger.Error("command_registration_failed", "error", err)
    return
}
```

The four commands declared by `slashCommands()` :

Command	Option	Handler
<code>/chat</code>	<code>message</code> (required)	<code>chat.HandleChat(..., isNew=true)</code>
<code>/rmhbot</code>	<code>request</code> (required)	<code>rmhbot.HandleRmhbotCommand(..., isNew=true)</code>
<code>/rmhbot-continue</code>	<code>request</code> (required)	<code>rmhbot.HandleRmhbotCommand(..., isNew=false)</code>
<code>/rmhbot-push</code>	<code>title</code> (optional)	<code>rmhbot.HandlePush(...)</code>

The TS `/rmhbot` command also accepted an `attachment` option. Attachment ingestion is not ported, so the option is intentionally omitted rather than declared and ignored.

Interaction routing & owner-gating

All interactions enter through `onInteraction` . A **review-fix** here is that each interaction runs under a context *derived from the bot's lifecycle context* with a per-interaction timeout, rather than being detached via `context.Background()` . This bounds in-flight agent loops and DeepSeek calls and cancels them on shutdown:

```
const interactionTimeout = 10 * time.Minute // generous: rmhbot loop runs many rounds

func (b *Bot) onInteraction(s *discordgo.Session, i *discordgo.InteractionCreate) {
    b.ctxMu.RLock(); parent := b.lifecycleCtx; b.ctxMu.RUnlock()
    if parent == nil { parent = context.Background() } // handler may fire before Run records ctx
    ctx, cancel := context.WithTimeout(parent, interactionTimeout)
    defer cancel()
    switch i.Type {
    case discordgo.InteractionApplicationCommand: b.routeCommand(ctx, s, i)
    case discordgo.InteractionMessageComponent:   b.routeComponent(ctx, s, i)
    case discordgo.InteractionModalSubmit:       b.routeModal(ctx, s, i)
    }
}
```

Buttons and modals carry a custom ID of the form `action:ownerId` , split by `splitCustomID` . Every component and modal handler is **owner-gated**: the interacting user must equal the embedded owner id, or be the configured `OWNER_ID` admin override. Otherwise an ephemeral denial is sent:

```
func (b *Bot) ownerOK(s, i, userID, ownerID, denyMsg string) bool {
    if userID == ownerID { return true }
    if b.cfg.OwnerID != "" && userID == b.cfg.OwnerID { return true } // admin override
    _ = s.InteractionRespond(i.Interaction, &discordgo.InteractionResponse{
        Type: discordgo.InteractionResponseChannelMessageWithSource,
        Data: &discordgo.InteractionResponseData{Content: denyMsg,
            Flags: discordgo.MessageFlagsEphemeral},
    })
    return false
}
```

The button/modal actions form two flows: `rmhbot_continue` / `chat_continue` buttons open a paragraph-input modal (`showModal`), whose submit (`rmhbot_continue_modal` / `chat_continue_modal`) re-enters the same handler with `isNew=false` ; the `rmhbot_push` button calls `HandlePush` directly. The user id is read via `interactionUser` , which handles both guild (`i.Member.User`) and DM (`i.User`) shapes.

The DeepSeek client

The TS code used the `openai` SDK pointed at DeepSeek's OpenAI-compatible API. Per the migration's no-SDK rule, the Go port re-implements a minimal client over `net/http` against `https://api.deepseek.com` , model from `DEEPSEEK_MODEL` (default `deepseek-chat`). It is **non-streaming** — a single blocking POST to `/v1/chat/completions` , rendered once — a deliberate simplification of the TS live-streaming embed edits.

```
// deepseek.go - single non-streamed completion round.
func (c *DeepSeekClient) complete(ctx, messages []ChatMessage, tools []Tool) (*completionResult, error) {
    if c.apiKey == "" { return nil, fmt.Errorf("DEEPSEEK_API_KEY is not set") }
    raw, _ := json.Marshal(chatCompletionRequest{
        Model: c.model, Messages: messages, Stream: false, Tools: tools})
    req, _ := http.NewRequestWithContext(ctx, http.MethodPost,
        c.baseURL+"/v1/chat/completions", bytes.NewReader(raw))
    req.Header.Set("Content-Type", "application/json")
    req.Header.Set("Authorization", "Bearer "+c.apiKey)
    resp, err := c.http.Do(req) // http.Client{Timeout: 60s}
    ...
    body, _ := io.ReadAll(io.LimitReader(resp.Body, 4<<20)) // 4 MiB cap
    if resp.StatusCode < 200 || resp.StatusCode >= 300 {
        return nil, fmt.Errorf("deepseek HTTP %d: %s", resp.StatusCode, string(body))
    }
    ...
}
```

The message and tool-call structs (`ChatMessage` , `ToolCall` , `ToolCallFunc` , `Tool`) mirror the OpenAI schema. Tool-call fields are `omitempty` so a plain `/chat` message serializes identically to the original TS payload, while the `rmhbot` loop can round-trip `tool_calls` / `tool_call_id` . `Chat()` is a thin wrapper for the persona (one round, no tools, returns the reply text).

/chat — the Alex Wu persona (faithful port)

The `/chat` handler is the faithful piece. The "Alex Wu" system persona is copied **verbatim** from `chat-handler.ts` (a 21-year-old UMN CS student / Wells Fargo intern, boba-obsessed, AAVE blaccent, vibe-coding tech bro). On a new `/chat` the history is reset to just the system prompt; the continue modal loads the existing session.

Session persistence

Sessions live in an in-memory cache keyed by Discord user id, backed by the `discord_chat_session` table via raw `pgx` . `loadSession` returns cache → DB → fresh persona-seeded session; on any DB error (including no rows) it keeps the fresh session, matching the TS `.catch(() => null)` . `saveSession` upserts by `discordUserId` :

```
// chat.go - saveSession upsert (best-effort, like the TS .catch(() => {})).
_, err = s.db.Pool.Exec(ctx,
`INSERT INTO "discord_chat_session"
  ("discordUserId","username","history","lastMessageId","updatedAt")
VALUES ($1,$2,$3,$4,$5)
ON CONFLICT ("discordUserId") DO UPDATE SET
  "username"=EXCLUDED."username", "history"=EXCLUDED."history",
  "lastMessageId"=EXCLUDED."lastMessageId", "updatedAt"=EXCLUDED."updatedAt",
session.userID, session.username, historyJSON, lastMessageID, time.Now().UTC())
```

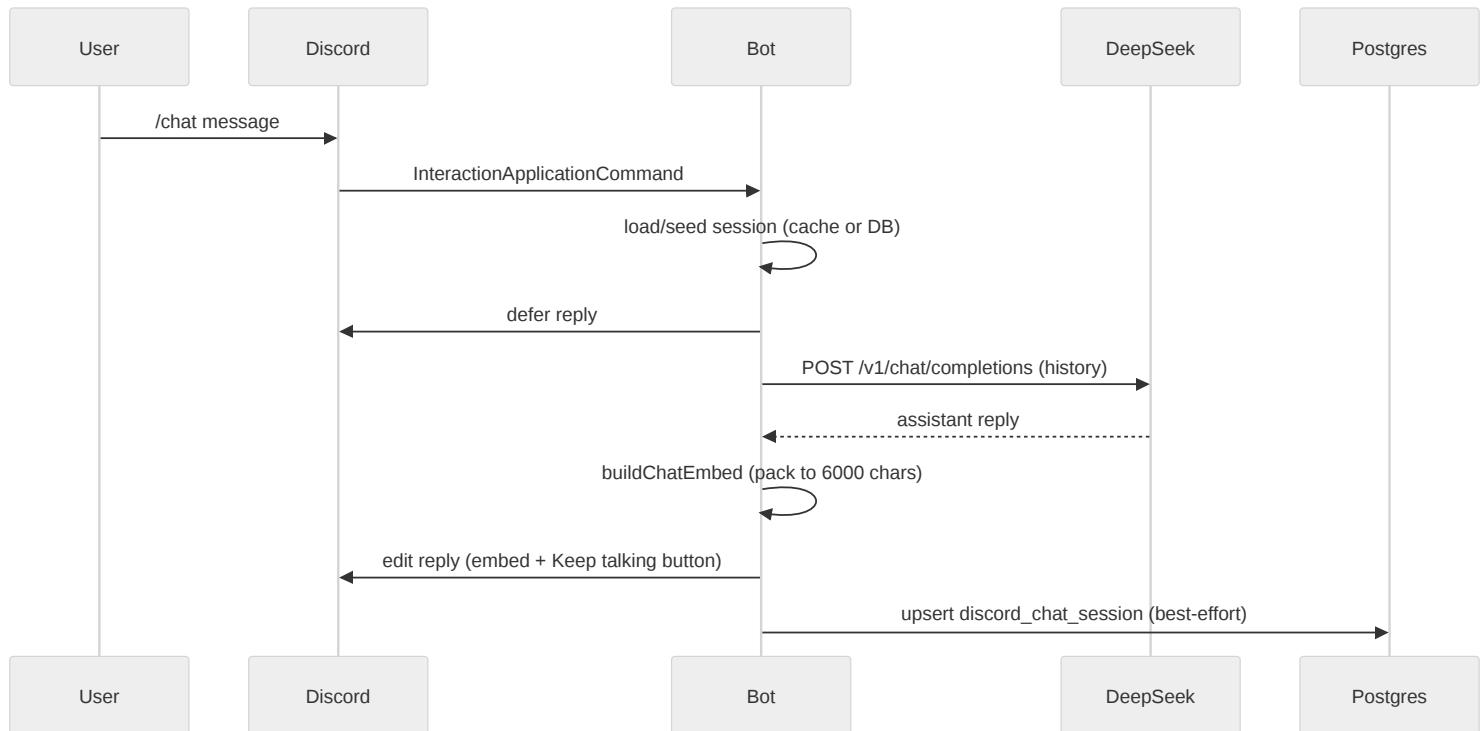


Figure 15.2: the /chat request path. The handler defers first because model calls take seconds.

The 6000-char embed packing (ported 1:1)

Discord caps an embed at 6000 characters total across title, footer, and field names+values, with ≤ 25 fields and ≤ 1024 chars per field value. `buildChatEmbed` ports the TS budget-packing logic exactly: it subtracts title+footer from the 6000 budget, gives the echoed user prompt first claim, then splits the reply across as many fields as fit, using `alexFieldName` ("👉 Alex") for the first and a zero-width space for continuations. If it runs out of budget or fields, the last field is ellipsis-marked:

```
// chat.go - reply field-packing loop (runes, not bytes).
remaining := []rune(replyText)
if len(remaining) == 0 { remaining = []rune("(no response)") }
for len(remaining) > 0 {
  if len(fields) >= maxFields { truncated = true; break }
  name := alexFieldName
  if replyFieldCount > 0 { name = continuationFieldName } // zero-width space
  valueBudget := min(fieldValueMax, budget-runeLen(name))
  if valueBudget <= 0 { truncated = true; break }
  take := valueBudget
  if take > len(remaining) { take = len(remaining) }
  value := string(remaining[:take])
  fields = append(fields, embedField{Name: name, Value: value})
  budget -= runeLen(name) + runeLen(value)
  remaining = remaining[take:]
  replyFieldCount++
}
```

All arithmetic is done in **runes** rather than bytes, so the emoji-heavy persona never splits a multi-byte character mid-rune — a Go-specific correctness improvement over the JS `.length` semantics it emulates.

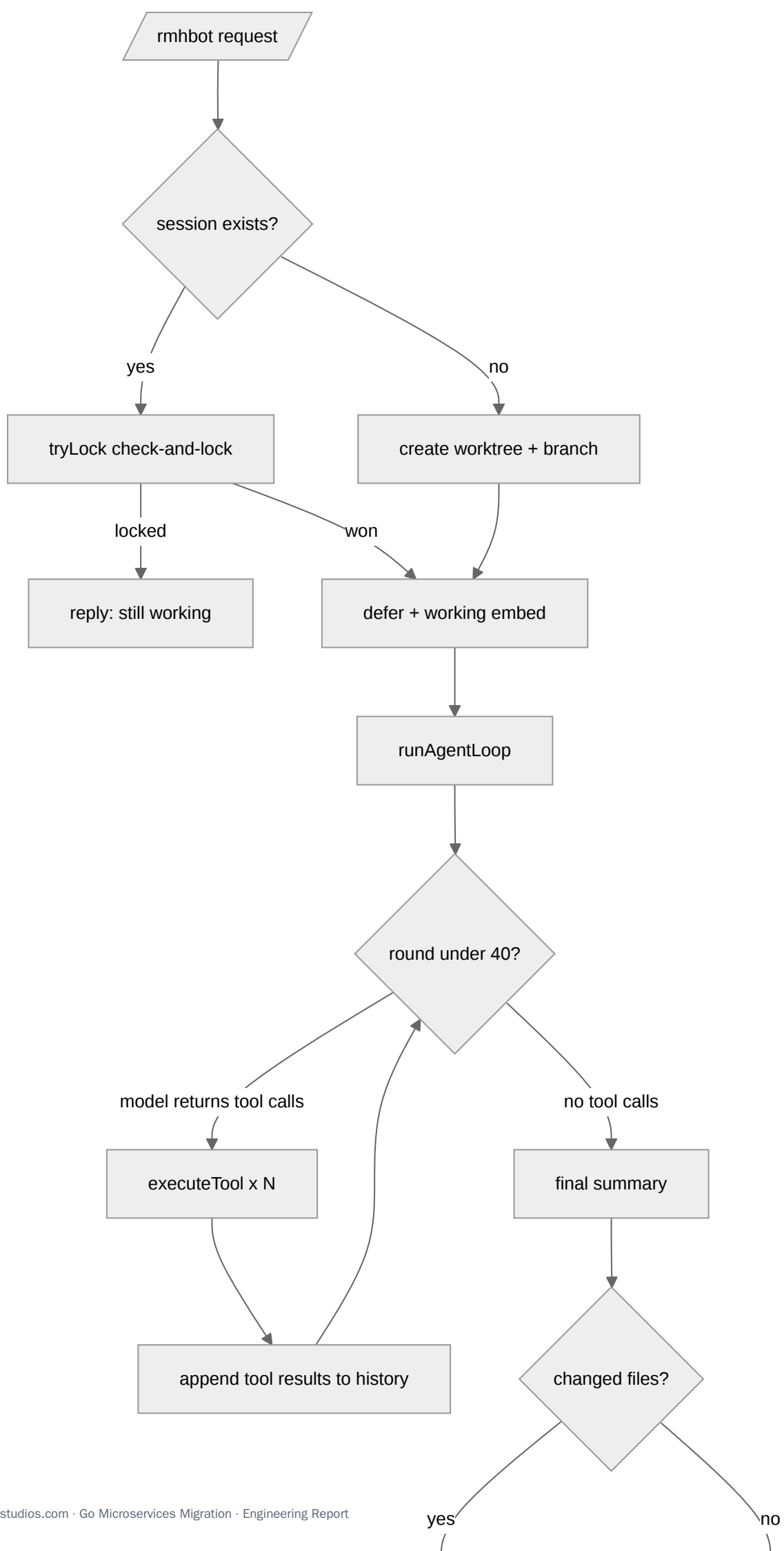
What is real: command/interaction structure; the in-memory `conversationState` (branch, worktree path, history, `deletedFilesThisSession`, `lockedAt`); 5-minute auto-unlock with an injectable clock; per-session locking that fixes a TOCTOU race; the `safePath` traversal guard and safety guards; git ops via `os/exec`; the `MAX_TOOL_ROUNDS = 40` loop; real `read_file` / `list_directory` / `write_file` / `delete_file` tools. **What is stubbed:** `run_typecheck` (needs the JS toolchain), `createPullRequest` (needs the GitHub REST call), `search_code` (needs grep wiring), recursive listing, attachment ingestion, and the live progress reporter.

A session is created on `/rmhbot` by cloning the target repo into a worktree and checking out a fresh branch. The repo is env-driven: `resolveRepo()` parses `GITHUB_REPO` ("owner/name"), falling back to `stickms/rmhstudios.com`. The branch name is deterministic under the injectable clock: `rmhbot-<sanitized-username>-<unix>`.

The agentic tool loop

`runAgentLoop` drives up to `maxToolRounds = 40` non-streamed DeepSeek rounds. Each round, if the model returns tool calls, the bot records the assistant turn, dispatches each call through `executeTool`, truncates the result to `maxToolResultLen = 8000`, and round-trips it back as a `role:"tool"` message. A round with no tool calls is the final answer:

```
// rmhbot.go - runAgentLoop core.
for round := 0; round < maxToolRounds; round++ {
    res, callErr := s.deepseek.complete(ctx, state.history, rmhbotTools())
    if callErr != nil { return "", changedFiles, callErr }
    if len(res.ToolCalls) > 0 {
        state.history = append(state.history, ChatMessage{
            Role: roleAssistant, Content: res.Content, ToolCalls: res.ToolCalls})
        for _, tc := range res.ToolCalls {
            var parsed map[string]any
            argsStr := tc.Function.Arguments; if argsStr == "" { argsStr = "{}" }
            result := ""
            if jErr := json.Unmarshal([]byte(argsStr), &parsed); jErr != nil {
                result = "Error: " + jErr.Error()
            } else {
                result = s.executeTool(ctx, tc.Function.Name, parsed, state, &changedFiles)
            }
            if len(result) > maxToolResultLen { result = result[:maxToolResultLen] + "\n[truncated]" }
            state.history = append(state.history, ChatMessage{
                Role: roleTool, ToolCallID: tc.ID, Content: result})
        }
        continue
    }
    summary = res.Content // no tool calls => done
    state.history = append(state.history, ChatMessage{Role: roleAssistant, Content: res.Content})
    return summary, changedFiles, nil
}
```



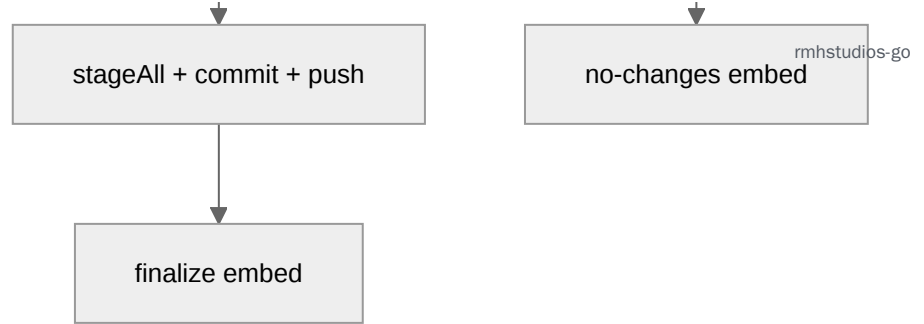


Figure 15.3: the /rmhbot agentic tool-loop and commit flow.

On a successful loop with changes, the bot stages all (`git add -A`), checks `status --porcelain` , commits with a `Co-Authored-By: RMHBot` trailer, and best-effort pushes the branch with `--force-with-lease` (mirroring the TS `.catch`). `/rmhbot-push` would then run typecheck and open a PR — both currently stubbed, so push fails closed on typecheck.

Per-session locking (TOCTOU review fix)

The original guard was an unsynchronized `exists && isSessionLocked` check, which let two concurrent interactions from the same user both pass. The Go port closes this with a per-session mutex (`conversationState.mu`) and a single check-and-lock critical section, `tryLock` . The map-level `s.mu` only guards the sessions map, not per-session state:

```
// rmhbot.go – atomic check-and-lock (closes the TOCTOU window).
func (s *RmhbotService) tryLock(state *conversationState) bool {
    state.mu.Lock(); defer state.mu.Unlock()
    if s.isSessionLockedLocked(state) { return false } // already locked
    now := s.clock.Now(); state.lockedAt = &now
    return true
}
```

Auto-unlock assumes a crashed prior run: a lock older than `sessionLockTTL = 5 * time.Minute` is cleared. The comparison is strict (`>`), so exactly 5 minutes is still locked — matching the TS:

```
func (s *RmhbotService) isSessionLockedLocked(state *conversationState) bool {
    if state.lockedAt == nil { return false }
    if s.clock.Now().Sub(*state.lockedAt) > sessionLockTTL { // auto-unlock
        state.lockedAt = nil
        return false
    }
    return true
}
```

Security guards

Path-traversal guard. Every filesystem tool resolves its path through `safePath` , ported 1:1 from `command-handler.ts` . It strips leading slashes, cleans/joins against the worktree base, and rejects any result that escapes the base. Crucially it uses a *separator-aware* prefix check, so a sibling like `/srv/clone-evil` (which shares the `/srv/clone` string prefix) is rejected:

```
func safePath(basePath, userPath string) (string, error) {
    normalized := leadingSlashes.ReplaceAllString(userPath, "")
    full := filepath.Clean(filepath.Join(basePath, normalized))
    base := filepath.Clean(basePath)
    if full != base && !strings.HasPrefix(full, base+string(filepath.Separator)) {
        return "", fmt.Errorf("Path traversal detected: %s", userPath)
    }
    return full, nil
}
```

Write/delete safety. Beyond traversal, writes are blocked on protected paths (`.env` , `server/discord-bot` , and the `.env.` / `server/discord-bot/` prefixes) and on content matching secret patterns (`DEEPSEEK_API_KEY` , `DISCORD_*TOKEN` , `GITHUB_TOKEN` ,

Git token handling caveat. `gitConfigArgs()` injects the `GITHUB_TOKEN` via an inline `url.https://x-access-token:<token>@github.com/.insteadOf` rewrite passed on the `git -c` command line. This is functional and scoped per-invocation (with `GIT_TERMINAL_PROMPT=0` to prevent interactive prompts), but the token appears in the process argument list, so it may be visible via `/proc` or process listings on the host while git runs. A hardened version should pass the credential via an `askpass` helper or a transient credential file instead.

Tests

- `discordbot_test.go` covers the three load-bearing, pure pieces — no Discord or DeepSeek mock needed:
- `safePath` **traversal guard** — allows in-tree paths (including leading-slash and `.`), rejects `../etc/passwd` , `app/../../etc/passwd` , `..` , and the sibling-prefix `../clone-evil/x` .
 - Embed packing** — a 20 000-char reply packs into multiple fields and never exceeds the 6000 total or the 1024/field cap; a 30 000-char reply ends in the ellipsis; a short reply yields exactly the You + Alex fields; an empty reply yields the `(no response)` placeholder.
 - 5-minute auto-unlock** — driven by a `fakeClock : nil` `lockedAt` is unlocked; 2 min stays locked; 5 min + 1 s auto-unlocks and clears `lockedAt` ; exactly 5 min stays locked (strict `>`). `TestMakeBranchName` asserts the deterministic sanitized branch `rmhbot-cool-user-1700000000` .

The faithfully-ported surface (gateway lifecycle, command registration, interaction routing & owner-gating, DeepSeek client, the full `/chat` persona with DB persistence and 1:1 embed packing) is complete and tested. The `/rmhbot` scaffold compiles, runs its real tools and git flow under the locking/safety guards, and marks every stub (`run_typecheck` , `createPullRequest` , `search_code`) with an explicit `TOD0(migration)` that fails closed rather than faking success.

The Go fleet shares one PostgreSQL database with the React SSR app — the same DSN, the same tables, the same Prisma-managed schema. Go does not own migrations and does not use an ORM; it reads and writes a well-defined subset of tables with hand-written, type-safe pgx queries. This section explains the data model boundary and how Go binds to it.

Who owns what

Prisma's schema has **98 models**. The Go services do not model all of them — they model only the tables backing the extracted services. Everything else (the social graph, feed, blog, the bulk of game tables, build galleries) remains the React app's territory, accessed through Prisma. The shared `session`, `user`, and `account` tables are the contact surface: Go reads them to authenticate; Better Auth (in JS) writes them.

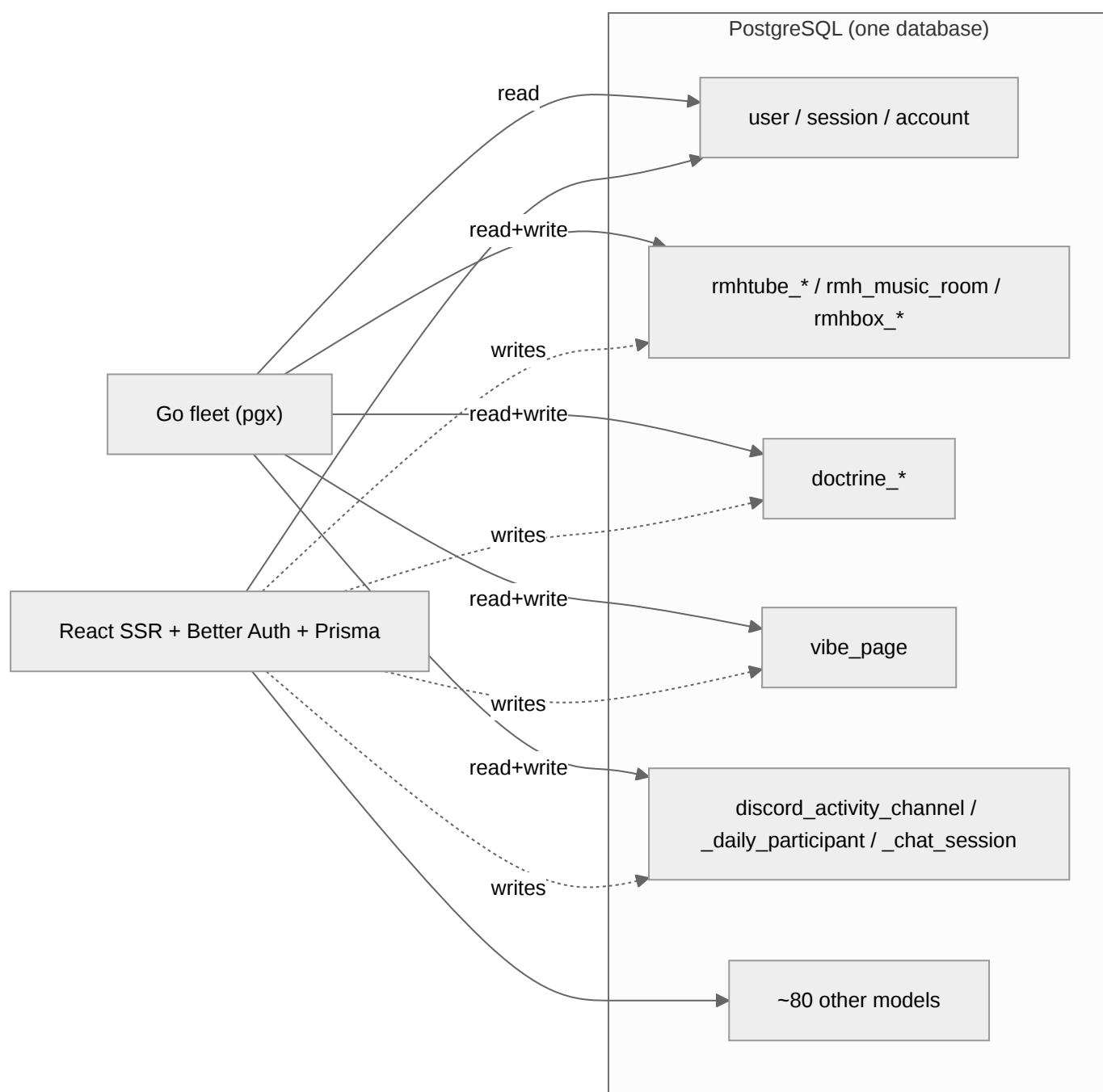


Figure 16.1 — Shared database, partitioned ownership. Go validates against auth tables and owns the operational tables of the services it runs.

The access layer: pgx, not an ORM

rmhstudios-go

All database access goes through one `pgxpool.Pool` built by `pkg/db` from the same `DATABASE_URL` the Node services use, with the pool size mirroring `SERVER_DB_POOL_SIZE`. This replaces both the Prisma client (used by the Node socket/worker services via `server/shared/prisma-client.ts`) and the ad-hoc raw `pg.Pool` clients embedded in the Node auth code — one uniform pool for the whole binary.

Each service keeps its SQL behind a small `Repo` interface so the business logic is unit-testable with a fake and the real implementation is a thin pgx layer. Column names and casing are taken verbatim from `prisma/migrations` so the quoted identifiers bind correctly.

```
// rmhtube: optimistic persistence behind a Repo interface
const q = `UPDATE "vibe_page"
  SET "thumbnailStale" = false, "thumbnailUrl" = $1
  WHERE "id" = $2 AND "updatedAt" = $3`
tag, err := pool.Exec(ctx, q, url, id, seenUpdatedAt) // 0 rows = lost the race
```

Typed models

`pkg/db/models.go` defines Go structs for the Go-owned tables, mapping Postgres types to Go: `TIMESTAMP(3)` → `time.Time` (`*time.Time` when nullable), `JSONB` → `json.RawMessage`, `DOUBLE PRECISION` → `float64`, `SMALLINT` → `int16`. Nullable scalars use pointers.

Domain	Tables (Go-owned)	Service
Identity	<code>user</code> , <code>session</code> , <code>account</code> , <code>verification</code>	all (validate)
Watch party	<code>rmhtube_room</code> , <code>rmhtube_room_member</code> , <code>rmhtube_queue_item</code>	rmhtube
Music rooms	<code>rmh_music_room</code>	rmhmusic
Party games	<code>rmhbox_profile</code> , <code>rmhbox_match</code> , <code>rmhbox_match_player</code>	rmhbox
Doctrine	<code>doctrine_puzzle</code> , <code>doctrine_reputation</code> , <code>doctrine_reputation_ledger</code> , <code>doctrine_sahur_session</code>	doctrine-worker
Vibe pages	<code>vibe_page</code>	vibe-worker
Discord	<code>discord_activity_channel</code> , <code>discord_daily_participant</code> , <code>discord_chat_session</code>	recap, discord-bot

Authentication against the shared schema

The migration's keystone insight is that authentication needs no Better Auth library on the Go side — it needs one SQL query. Both the original Node socket services and the Go fleet validate a session identically:

```
SELECT s."userId", s."expiresAt", u."name", u."image", u."isAdmin"
FROM "session" s JOIN "user" u ON u."id" = s."userId"
WHERE s."token" = $1 LIMIT 1      -- then reject if expiresAt < now()
```

The token arrives as a websocket query parameter, a Bearer header, or the `better-auth.session_token` cookie (the value before the dot). Discord-Activity clients present a Discord user id instead, mapped to a platform user through the `account` table (`providerId = 'discord'`). Issuance, refresh, and OAuth remain entirely in Better Auth.

Migrations stay with Prisma

Go writes rows; it never alters the schema. PR #121's Helm `pre-upgrade` migration hook remains the single authority for schema evolution. The Go services assume the schema is already migrated — exactly as the Node workers did — and fail fast on a missing column rather than attempting DDL. This keeps one source of truth for the schema and avoids two systems racing to migrate the same database.

Concurrency-safe writes

rmhstudios-go

Two write patterns deserve note. First, **optimistic concurrency** (vibe-worker): clear a flag only if the row's `updatedAt` is unchanged since it was read, so a concurrent edit is never clobbered. Second, **atomic read-modify-write** (rmhbox profile stats): the Node code read a JSONB blob, merged in memory, and wrote it back — a lost-update hazard under concurrent games. The Go port performs the merge inside a transaction with `SELECT ... FOR UPDATE`, closing the race. Both are detailed in their service sections.

Some concerns span every service: how identity flows, how the fleet scales, how it is observed, and how configuration and secrets are handled. These are solved once in the shared foundation and applied uniformly, which is the main payoff of the single-module design.

Authentication & the trust boundary

There is exactly one place identity is asserted: the gateway. It strips every inbound `X-Rmh-*` header (by prefix, so future headers can't be forged), validates the Better Auth session, and injects trusted headers downstream. Realtime services additionally validate the token themselves at websocket handshake (defense in depth, since a client connects to a WS prefix the gateway proxies). Both paths use the same `auth.Validator` and the same SQL.

A real DB/transport error during validation is logged and metered rather than silently treated as "anonymous" — so an auth-database outage is observable instead of quietly downgrading every user. (This distinction was added during senior review.)

Horizontal scaling & the backplane

The realtime services hold per-room state in memory, so a naive second replica would split a room across processes that can't see each other. The `events.Bus` abstraction solves this: a broadcast is delivered to local room members *and* published to a topic; every other replica's room subscription re-delivers it locally, skipping its own origin to avoid duplicates. Flipping between single- and multi-replica is one environment variable.

REDIS_URL	Bus	Realtime replicas
unset	<code>events.Local</code> (in-process)	1 (today's behavior)
set	<code>events.Redis</code> (pub/sub)	N, with sticky sessions at ingress

This is precisely the "socket.io Redis adapter + sticky sessions" work PR #121 named as Stage 1 and "the real first scaling investment." Here it is a first-class part of the foundation, language-agnostic in spirit and required by no service-specific code.

Observability

Every service constructs a per-service Prometheus registry via `pkg/telemetry` and exposes `/metrics` — on the client port for HTTP/WS services, on a dedicated metrics port for workers. Standard instruments ship out of the box: `rmh_active_connections` (gauge), `rmh_messages_total{event}`, `rmh_db_queries_total{outcome}`, and `rmh_job_runs_total{job,outcome}`. Structured JSON logs (`pkg/log`) match the Node logger's field shape so existing log tooling keeps working. This is the instrumentation half of PR #121's Stage 2; wiring dashboards/alerts is the remaining operational step.

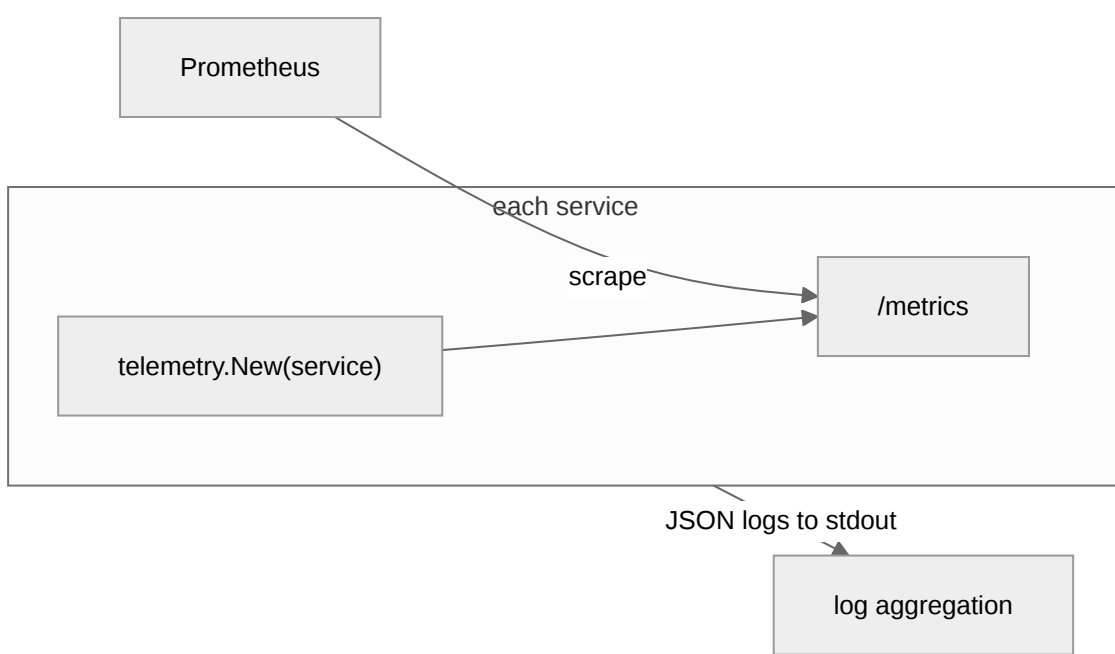


Figure 17.1 — Uniform metrics + structured logs across the fleet.

Configuration & secrets

All configuration is environment-driven through `pkg/config`, which reads the *same* variables the Node services used — `DATABASE_URL`, `SERVER_DB_POOL_SIZE`, the `PORT_*` family, the Discord and DeepSeek keys. In Kubernetes the values come from PR #121's `rmhstudios-secrets` Secret, mounted as `envFrom` on every Deployment; the Go chart adds no new secret and never puts secrets in git or Helm history. `REDIS_URL` and the gateway's `*_UPSTREAM` values are the only genuinely new variables, and both have sensible defaults.

Resilience patterns

- **Fault isolation:** every client-driven handler runs under `recover()`; a panic in one game or one connection cannot crash a service.
- **Graceful shutdown:** HTTP/WS services drain on SIGTERM within a grace period; workers stop their tickers and cancel their context; the discord-bot honors a 90s grace for in-flight AI work.
- **Fail-fast startup:** services ping the database on boot (or wait for it, for workers) and exit non-zero on misconfiguration, so a broken pod restarts rather than serving errors.
- **Backpressure:** slow websocket consumers are evicted rather than allowed to wedge a shared room broadcast.

Each service ships as its own minimal container, built from a single parameterized multi-stage Dockerfile. Static Go binaries on a bare Alpine base keep images small (the gateway is ~33 MB) and the attack surface tiny.

One Dockerfile, every service

Rather than nine near-identical Dockerfiles, there is one, parameterized by a `SERVICE` build arg (which `cmd/` to compile) and an optional `EXTRA_PKGS` arg for the few services that need OS packages at runtime. The build stage downloads modules with a cache mount, then compiles a fully static binary (`CGO_ENABLED=0` , `-trimpath -ldflags="-s -w"`). The runtime stage is Alpine with only what's required.

```
FROM golang:1.23-alpine AS build
ARG SERVICE
RUN --mount=type=cache,target=/go/pkg/mod go mod download
RUN CGO_ENABLED=0 GOOS=linux go build -trimpath -ldflags="-s -w" \
    -o /out/app ./cmd/${SERVICE}

FROM alpine:3.20 AS runtime
ARG EXTRA_PKGS=""
RUN apk add --no-cache ca-certificates tzdata ${EXTRA_PKGS} \
    && adduser -S app -G app
USER app
ENTRYPOINT ["/usr/local/bin/app"]
```

Per-service runtime needs

Service	EXTRA_PKGS	Why
most services	—	<code>ca-certificates</code> + <code>tzdata</code> from the base.
<code>doctrine-worker</code>	— (needs <code>tzdata</code> , in base)	<code>time.LoadLocation</code> per-timezone Sahur checks require the zoneinfo DB.
<code>vibe-worker</code>	<code>chromium nss freetype harfbuzz ttf-freefont</code>	Headless Chromium for thumbnail capture.
<code>discord-bot</code>	<code>git openssh-client</code>	RMHBot performs git clone/commit/push.

Two correctness details baked into the base image: `ca-certificates` (outbound TLS to Discord/DeepSeek) and `tzdata` (the `doctrine-worker`'s timezone math). Both were easy to forget and would have produced subtle runtime failures; they're in the base for every service.

Security posture

- **Non-root:** images run as an unprivileged `app` user.
- **Static binary:** no libc surface, no shell needed at runtime (except where `EXTRA_PKGS` adds one for git).
- **Minimal layers:** a single `apk add` , then the binary — small and auditable.
- **Reproducible:** `-trimpath` strips local paths; module download is cache-mounted and verified against `go.sum` .

Verified build

The Dockerfile is not theoretical: the gateway image was built and run during verification. It produced a **33 MB** image whose binary started, read its configuration, and exited cleanly with a structured error when `DATABASE_URL` was absent — confirming both stages work end to end.

```
$ docker build -f Dockerfile --build-arg SERVICE=gateway -t rmhstudios-go-gateway .  
$ docker images rmhstudios-go-gateway # 33.1MB  
$ docker run --rm rmhstudios-go-gateway  
{  
  "level": "ERROR",  
  "msg": "config",  
  "service": "gateway",  
  "error": "config: DATABASE_URL is required for service \"gateway\""}  
}
```

The fleet ships with two orchestration paths that mirror the existing project: a Docker Compose file for local/single-host parity with the Node stack, and a Helm chart that deliberately follows PR #121's conventions so it slots into the same k3s cluster.

Docker Compose

`docker-compose.go.yml` is the Go analog of the project's root `docker-compose.yml`. It builds one image per service from the shared Dockerfile, runs a Redis instance as the realtime backplane, reads the same `.env.production`, and exposes the gateway as the single ingress. The gateway's upstreams are wired to the Compose service names; the catch-all points back at the host's Node web app via `host.docker.internal`.

```
docker compose -f go-services/docker-compose.go.yml \
  --env-file .env.production up -d --build
```

The Helm chart

`deploy/helm/rmhstudios-go` renders the whole fleet. It reuses PR #121's patterns: a generic Deployment template that ranges over a `services` map, a Service template for the ported services, a Traefik Ingress, and — crucially — the same `rmhstudios-secrets` Secret (via `envFrom`) and `rmhstudios-data` PVC. The one deliberate departure from PR #121's single-image chart is noted in the chart description: because each Go service is a distinct static binary, each is a distinct image (`<registry>/<prefix>-<svc>:<tag>`).

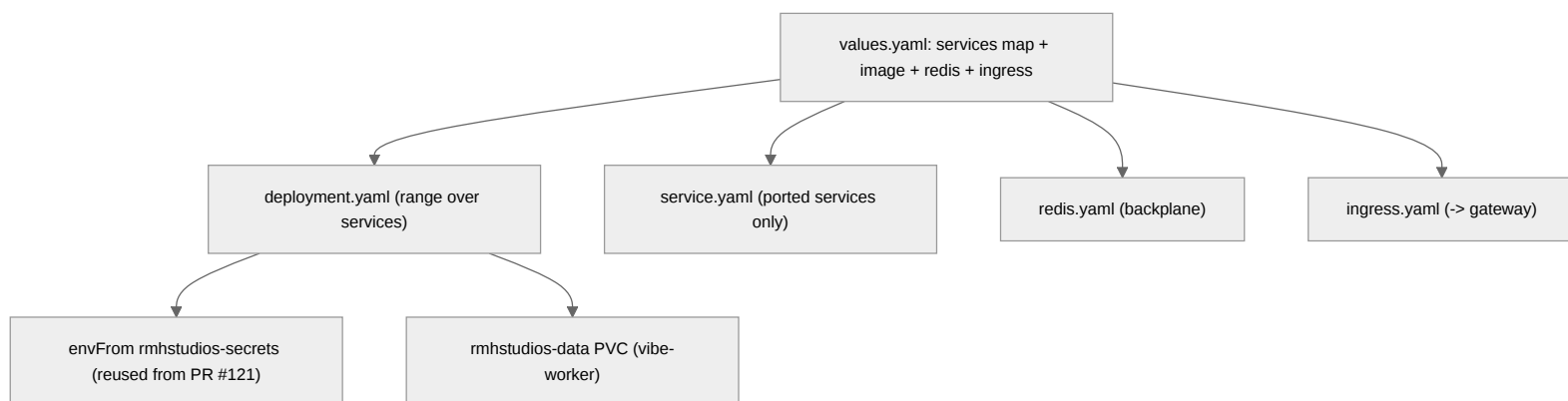


Figure 19.1 — Chart structure: one values map drives a generic Deployment, Services, Redis, and Ingress.

The services map

The chart is data-driven: each service is an entry in `values.services` declaring its client port (if any), replica count, ingress path, and whether it is a worker (no Service, probes hit `/health` on the metrics port) or needs the shared storage. Probes target the right port automatically via a helper.

Key	Effect
<code>port</code>	Container port + ClusterIP Service + probe port.
<code>worker: true</code>	No Service; probes hit <code>/health</code> on <code>metricsPort</code> (9090).
<code>replicas</code>	Per-service replica count (prod scales gateway/gamehub).
<code>storage: true</code>	Mounts the shared PVC (vibe-worker thumbnails).
<code>path</code>	Documents the ingress prefix the gateway proxies.

Gateway upstream wiring

rmhstudios-go

The gateway's reverse-proxy upstreams default to Compose-style hostnames, which do not resolve in Kubernetes. The chart therefore injects the correct in-cluster Service DNS names into the gateway's environment (`GAMEHUB_UPSTREAM=http://rmhstudios-go-gamehub:7001` , etc., and `WEB_UPSTREAM` for the Node SSR Service). This was a blocking deploy bug caught in senior review and fixed before this document was written.

The backplane in-cluster

When `redis.enabled` (default), the chart deploys a single in-memory Redis (no persistence, no PVC) and points every realtime service at it via `REDIS_URL` . With it, the realtime services can scale beyond one replica behind sticky-session ingress; without it, they run single-replica exactly as today.

Validated rendering

The chart was validated, not just written: `helm lint` passes for both default and production values, and the rendered manifests — 35 resources across the default and prod value sets — pass `kubeconform -strict` with zero invalid resources.

```
$ helm lint deploy/helm/rmhstudios-go      # 0 failed
$ helm template ... | kubeconform -strict  # 35/35 valid, 0 invalid
```

The fleet has a CI pipeline that gates every change on build, vet, race-tested units, formatting, end-to-end tests, and chart validation; a deploy script that mirrors PR #121's single-node-or-registry switch; and a documented, reversible cutover plan.

Continuous integration

`.github/workflows/go-microservices.yml` runs three jobs on changes under `go-services/` or the chart:

Job	Steps
build-test	<code>go build</code> · <code>go vet</code> · <code>go test -race</code> · <code>gofmt</code> check
e2e	Postgres service container → load <code>schema.sql</code> → <code>run.sh --no-docker</code> (build + launch binaries + ws-client tests)
helm	<code>helm lint</code> · <code>helm template</code> render

The `--no-docker` mode of the e2e script exists specifically for CI: GitHub Actions already provides a Postgres service container and a Go toolchain, so the script builds and launches the binaries directly on the runner instead of nesting Docker. (This branch in `run.sh` was added during review when the CI/script contract didn't match.)

The deploy script

`deploy/deploy-go.sh` is the Go analog of PR #121's `deploy-k8s.sh`. It builds one image per service (with the right `EXTRA_PKGS` per service), then either imports them straight into k3s's containerd for single-node, or pushes to a registry for multi-node — the same switch PR #121 introduced — and finishes with an atomic Helm upgrade.

```
# single-node k3s (imports into containerd):
./deploy/deploy-go.sh production
# multi-node (push to a registry):
REGISTRY=registry.example.com ./deploy/deploy-go.sh production

helm upgrade --install rmhstudios-go deploy/helm/rmhstudios-go \
  -f values-prod.yaml --set image.tag=$SHA --atomic --wait
```

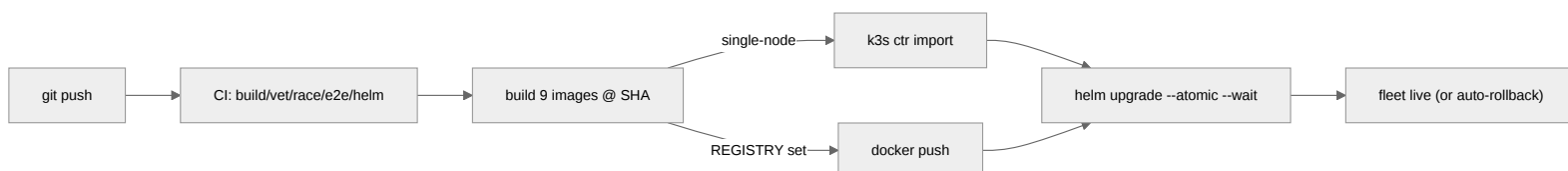


Figure 20.1 — From commit to cluster, with atomic rollback on failure.

Cutover & rollback

Because the gateway proxies anything it doesn't own back to the existing Node SSR app, cutover can be incremental and is fully reversible:

1. Deploy `rmhstudios-go` beside the existing stack; it shares the DB and Secret.
2. Smoke-test the gateway and each realtime service in-cluster.
3. Shift one websocket prefix at a time to the Go service at the ingress (e.g. start with `/socket/` → `gamehub`), leaving the rest on the Node services.
4. Watch metrics; if a service misbehaves, route that prefix back to the Node equivalent — both run against the same database.
5. **Full rollback:** point the ingress back at the Node stack and `helm uninstall rmhstudios-go`. No data migration is involved because both stacks use the same tables.

The realtime services do not yet speak the Socket.IO wire protocol (see Section 6). A real cutover of a websocket prefix requires the client-side adapter described there. Until then, the Go realtime services are exercised by the Go e2e clients and the new-protocol path, not by the existing browser clients — this is the single most important cutover caveat and is stated plainly.

Operational notes

- **Migrations:** unchanged — PR #121's Prisma hook owns the schema; the Go deploy assumes it has already run.
- **Scaling:** `kubectl scale deploy/rmhstudios-go-gamehub --replicas=N` once Redis + sticky sessions are in place.
- **Secrets:** rotate via the existing `rmhstudios-secrets` flow; the Go fleet picks them up on next rollout.

Verification was treated as a first-class deliverable, not an afterthought. The fleet is validated at four levels — compilation, static analysis, race-tested unit/integration tests, and end-to-end tests against real infrastructure — and every level is green.

The test pyramid

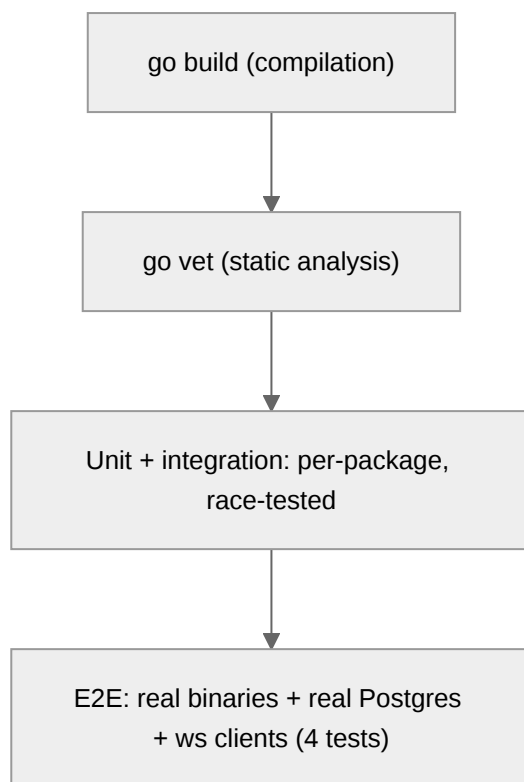


Figure 21.1 — Four verification levels, all passing.

Build, vet, format

The whole module compiles (`go build ./...`), passes `go vet ./...` with no findings, and is `gofmt` -clean (CI fails on any unformatted file).

Unit & integration tests under the race detector

Every internal package and the load-bearing shared packages have tests, run with `go test -race` . Highlights:

Package	What the tests prove	rmhstudios-go
pkg/ratelimit	Sliding-window limits, expiry, per-conn isolation, forget.	
pkg/realtime	Envelope round-trip, atomic seq monotonicity under 1000 goroutines, grace-timer cancel/reschedule.	
internal/doctrine	mulberry32 + seed are bit-exact against Node golden vectors; weekday difficulty.	
internal/gamehub	Full KK lifecycle: create/join/input/state/rematch/disconnect.	
internal/rmhbox	Lobby FSM transitions, seq numbering, per-player scoped state, atomic stat merge.	
internal/rmhtube	Leader video projection, seq monotonicity, queue reorder.	
internal/rmhmusic	Playback drift projection, seq deltas, host auto-reassignment.	
internal/vibeworker	Poll/optimistic-update orchestration with a fake capturer + repo.	
internal/recap	Embed/ranking, due-poll, always-clear with fakes.	
internal/discordbot	safePath traversal guard, 6000-char embed packing, 5-min auto-unlock.	
internal/gateway	Longest-prefix routing, auth header inject/strip.	

Race detector clean. `go test -race ./...` passes for every package. This matters: the race detector found a real data race during review (a timer goroutine reading a field `Cancel()` mutated); it was fixed and the suite is now clean. See Section 22.

End-to-end tests against real infrastructure

The e2e suite (build tag `e2e`) is the proof that the services actually work together, not just that the units pass.

`scripts/e2e/run.sh` starts a real PostgreSQL, loads the schema, builds and launches the real `gamehub` and `rmhtube` binaries, waits for their `/health`, then runs Go tests that connect as real websocket clients.

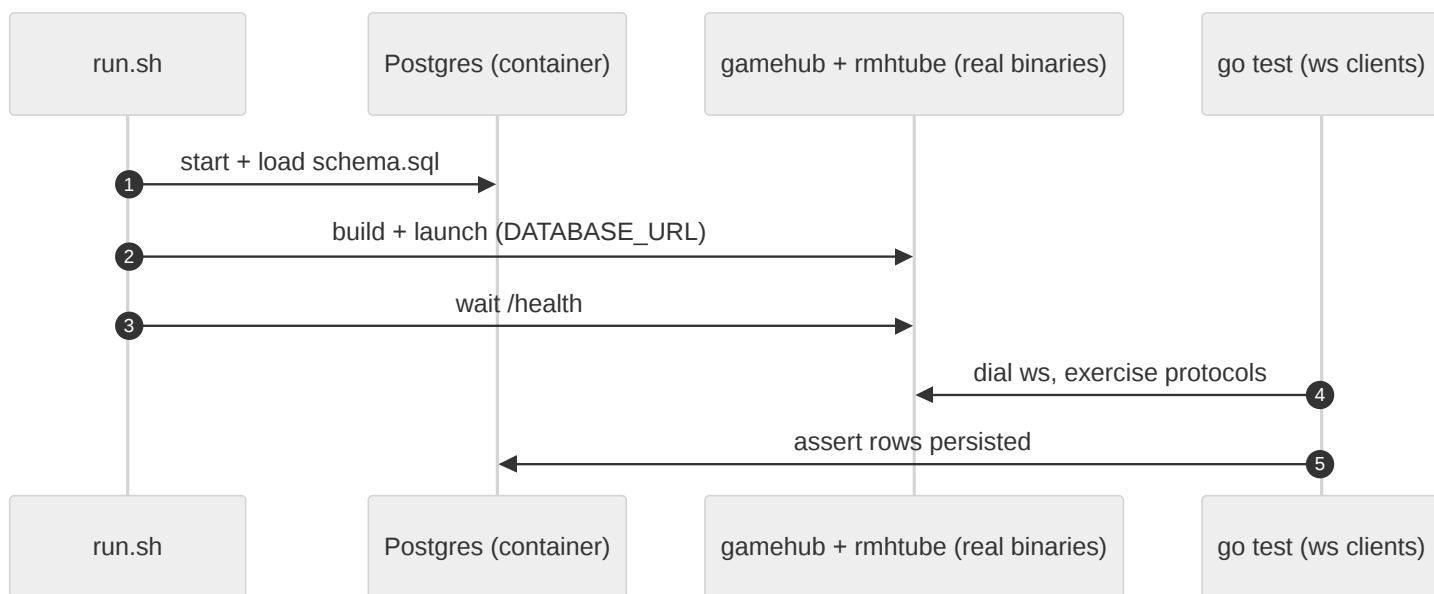


Figure 21.2 — The e2e harness: real Postgres, real binaries, real websocket clients.

The four e2e tests and their results:

Test	Proves	Result
TestSessionValidationAndRoomCreate	Seed user+session → authed ws → create room → row persisted with correct host id.	PASS
TestRejectsBadToken	Bogus token → connection treated as anonymous (documents real behavior).	PASS
TestKKRelayFlow	create_room → join → input relay (guest→host) → game_state (host→guest) → opponent_disconnected.	PASS
TestKKJoinUnknownRoom	Joining a non-existent code returns an error.	PASS

```

--- PASS: TestSessionValidationAndRoomCreate (0.12s)
--- PASS: TestRejectsBadToken (0.00s)
--- PASS: TestKKRelayFlow (0.00s)
--- PASS: TestKKJoinUnknownRoom (0.50s)
ok  github.com/rmhstudios/rmh-go/e2e  0.63s

```

Infrastructure validation

Beyond Go: `helm lint` passes, the rendered manifests pass `kubeconform -strict` (35/35 valid), and the container image builds and runs. The verification toolchain itself runs through Docker (the pinned `golang:1.23` image) so it is reproducible on any machine with Docker, with no local Go install required.

Before this document was written, the entire Go fleet went through a three-part senior engineering review — one reviewer per subsystem — exactly as a real pre-merge review would. The reviewers found genuine issues, including a blocking concurrency bug and a real data race. This section records the findings and the fixes honestly, because the remediation loop is part of the work.

The review process

Three independent reviews ran in parallel: (1) the shared foundation + gateway, (2) the four realtime services, (3) the workers + discord-bot + the deploy layer. Each reviewer read the code directly and reported prioritized findings (blocking / high / medium / low) with concrete fixes. Findings were then fixed and the full verification suite re-run, including the race detector.

Blocking findings (fixed)

Finding	Fix
Send-on-closed-channel panic. <code>Conn.close()</code> closed the <code>send</code> channel while other goroutines (heartbeats, timers, broadcasts) could still write to it — a guaranteed crash under exactly the load realtime is built for.	Reworked the connection lifecycle: a dedicated <code>closed chan struct{}</code> (closed once) signals shutdown; <code>send</code> is never closed; all producers go through <code>trySend</code> , which selects on <code>closed</code> . The write pump exits on <code>closed</code> .
Gateway upstreams unresolvable in k8s. The Helm chart didn't set the gateway's <code>*_UPSTREAM</code> env, so its compose-style defaults (e.g. <code>http://gamehub:7001</code>) wouldn't resolve to the in-cluster Service names.	The chart now injects the correct in-cluster DNS (<code>http://rmhstudios-go-gamehub:7001</code> , etc.) plus <code>WEB_UPSTREAM</code> for the gateway Deployment.
CI/script contract mismatch. CI invoked <code>run.sh --no-docker</code> , which the script didn't support.	Added a real <code>--no-docker</code> mode that builds + launches the binaries on the runner against the CI-provided Postgres.

The data race the detector caught

The most instructive finding came not from a human read but from `go test -race`: in `rmhbox's StateSync`, the timer-broadcast goroutine read `h.stop` in its `select` while `TimerHandle.Cancel()` set `h.stop = nil` under the mutex — an unsynchronized read/write on the same field.

```
// before: Cancel niled the channel the goroutine was reading
t.stop = nil // RACE vs the select below
// goroutine:
select { case <-h.stop: return ... }

// after: never nil it; capture once; close-once is the signal
stop := h.stop // captured before goroutine starts
go func(){ for { select { case <-stop: return ... } } }()
// Cancel(): close(t.stop) guarded by a cancelled flag; no nil
```

Outcome: after the fix, `go test -race ./...` is clean across every package. This is the value of running the detector in CI: a bug that is invisible to the eye and intermittent in production was caught deterministically.

High-severity findings (fixed)

rmhstudios-go

Finding	Fix
discord-bot <code>/rmhbot</code> session lock was a check-then-set TOCTOU race; two interactions could run two agent loops on one worktree.	Per-session mutex; atomic <code>tryLock / unlock</code> ; history mutated only under the lock.
vibe-worker built the <code>data:</code> URL by string concatenation; arbitrary HTML with <code># / %</code> truncated or mis-decoded in Chromium.	Base64-encode the payload (<code>data:text/html;base64,...</code>).
Gateway stripped only three known identity headers, leaving future <code>X-Rmh-*</code> headers forgeable.	Strip the entire <code>X-Rmh-</code> prefix namespace before injecting trusted values.

Selected medium findings (fixed)

- **rmhbox grace timers** captured a stale lobby pointer and didn't cancel preload/ingame keys on force-end → re-resolve the live lobby in each closure; cancel all grace keys in `clearLifecycleTimers` .
- **rmhbox** pause/resume used a concrete type-switch → replaced with a `baseGame()` interface method the compiler enforces.
- **rmhtube** didn't prune skip/queue votes on disconnect and disbanded outside the room lock → prune on member removal; disband under the lock with a join re-check.
- **rmhmusic** `onPlay` released and re-acquired the lock, allowing interleaved track changes → single critical section; seq stamping aligned with rmhtube.
- **workers** logged-and-continued on a metrics-port bind failure, leaving pods unprobeable → fail fast on bind error.
- **auth** treated DB errors as "anonymous" silently → log/meter non-authentication errors.
- **discord-bot** ran interactions on `context.Background()` → derive from the lifecycle context with a timeout; make the GitHub repo env-driven.

What the reviewers checked and found clean

Not everything needed fixing. The reviewers explicitly signed off on: the auth SQL (parameterized, correct expiry), the pgx pool configuration, graceful shutdown, the doctrine determinism (golden-tested), the rmhbox atomic `SELECT ... FOR UPDATE` stat merge, per-player state scoping (no info leak), the safePath traversal guard, and the consistent room→manager lock ordering across the realtime services. The Dockerfile (static build, tzdata, non-root) and the Helm probe-port helper were also verified correct.

Performance was not the stated motivation for this migration — correctness and faithful behavior were — but a Go rewrite of a realtime tier brings real efficiency characteristics worth recording, along with the specific optimizations baked into the implementation.

The concurrency model

Each websocket connection runs exactly two goroutines (a read pump and a write pump) over a buffered send channel — the idiomatic gorilla pattern. Go's scheduler multiplexes thousands of such goroutines onto a handful of OS threads, so a single replica can hold a large number of idle connections cheaply, where the Node event loop served the same role single-threaded. Per-room state is guarded by fine-grained per-room mutexes rather than one global lock, so unrelated rooms never contend.

Optimizations in the implementation

Optimization	Where	Effect
Per-room mutexes + atomic seq	realtime, all services	No global lock; sequence numbers via <code>atomic.AddUint64</code> , no lock at all.
Non-blocking broadcast	realtime hub/room	A slow consumer is evicted, never blocks a shared room fan-out.
Snapshot-then-broadcast	realtime services	Member list copied under the lock, then sent outside it — lock held briefly.
Connection pooling	pkg/db	One <code>pgxpool</code> per binary sized to <code>SERVER_DB_POOL_SIZE</code> ; pgx's binary protocol is faster than text.
Atomic stat merge	rmhbox	<code>SELECT ... FOR UPDATE</code> in a tx — correct under concurrency and one round-trip.
Optimistic concurrency	vibe-worker	Lock-free flag clear; no row locks held during slow Chromium renders.
Static binaries	build	Fast cold-start (no JIT/warmup); ~33 MB images pull quickly.
Bounded, GC'd rate limiter	pkg/ratelimit	Memory capped; stale entries swept every 30s.

Footprint

The runtime footprint is modest by construction: a static Go binary on Alpine, a few MB of RSS at idle, and image sizes in the tens of MB. The Helm defaults request 25m CPU / 32Mi memory per pod with 1 CPU / 256Mi limits — deliberately small, on the assumption that connection-bound realtime work is cheap per connection and scales by replica count rather than per-pod size.

Scaling characteristics

The dominant scaling axis is concurrent websocket connections. With the Redis backplane enabled, a realtime service scales horizontally: add replicas behind sticky-session ingress and the bus keeps rooms coherent across them. Workers scale differently — they are singletons by nature (one scheduler, one poller), so they scale *up* (more DB pool, larger batches) rather than out, and the chart keeps them at one replica.

Honest caveat: no load testing was performed as part of this work. The performance claims here are structural (what the design enables), not measured benchmarks. Establishing connection-count and latency baselines is explicit future work (Section 25).

Versus the Node services

The Go services should use less memory per connection and start faster than the Node equivalents, and the per-room locking gives true parallelism across rooms on multiple cores (where Node's single event loop did not). These are reasons a Go

realtime tier is attractive *if* connection volume ever justifies it — which is precisely the metrics-driven condition PR #121 set for Stage 4. This migration provides the implementation; the metrics to justify adopting it are the missing input.

rmhstudios-go

This section is the deliberate counterweight to the rest of the document. A migration report that only lists accomplishments is untrustworthy. Here is a frank assessment of what is risky, what is incomplete, and whether this work should actually be adopted.

The Socket.IO wire-protocol gap (largest risk)

The existing browser clients speak the Socket.IO v4 protocol. The Go realtime services speak a clean JSON-over-raw-WebSocket protocol with the same *semantics* but a different *wire format*. They are not interoperable. A production cutover of any websocket prefix requires a client-side adapter (mapping existing event names onto the new transport) that is **designed but not built**. Until then, the Go realtime services are validated by the new-protocol e2e clients, not by the live browser clients.

This is the single most important limitation and is stated everywhere it is relevant. It does not affect the workers (doctrine, recap, vibe) or the discord-bot, which have no browser websocket clients.

Scaffolded surfaces

Surface	State	Risk if adopted as-is
rmhbox minigames	1 of 9 ported; 8 compiling stubs	Only Rhyme Time is playable; the others must be implemented before rmhbox replaces the Node service.
RMHBot agentic editor	Structure + safety + git + chat real; several tools stubbed	Typecheck and PR-creation are stubs; <code>/rmhbot</code> can't complete a full edit-and-PR cycle yet.
gamehub non-KK games	Generic relay framework + stub registry	Only Kowloon Knockout is wired; other games need registration.

Behavioral substitutions

- vibe-worker downscaling:** Chromium device-scale replaces Node's `sharp`. Visual parity of thumbnails should be eyeballed before relying on it.
- recap presence:** discordgo's gateway replaces a hand-rolled opcode loop — an improvement, but a behavioral change in how presence is maintained.
- IDs:** Go generates UUID primary keys where Prisma used cuid; both are opaque TEXT, but the format differs.
- rmhbox rhyme check:** a phonetic-suffix heuristic substitutes for the Node CMU-dictionary lookup; scoring may differ at the margins.

What is unverified

- No load testing.** Performance claims are structural, not measured.
- No live-traffic validation.** The services have never seen production clients (see the protocol gap).
- chromedp under load.** Headless Chromium stability for the thumbnail worker at scale is untested in Go.
- DeepSeek streaming.** The Go chat client is non-streaming where the Node version streamed; UX differs.

Should this be adopted?

Honest recommendation. Not wholesale, and not now — for the same reasons PR #121 gave. The Node services work; the React/Better Auth/ Prisma core must stay JavaScript regardless; and the biggest realtime scaling win (the Redis backplane + sticky sessions) is language-agnostic and could be applied to the existing Node services. This Go fleet is most valuable as: (1) a complete, tested reference for what a Go target looks like; (2) a source of genuinely good ideas already extracted here — the single auth-

validation query, the events.Bus backplane abstraction, the atomic stat-merge fix; and (3) an incremental option, since the shared session table means one service could be migrated and measured in isolation. Adopting it should be metrics-driven and per-service — exactly the Stage-4 condition that was set.

Where the risk is low

Conversely, several pieces are low-risk and could be adopted on their own merits: the **doctrine-worker** (bit-exact, fully ported, deterministic), the **recap** runner, and the **gateway** as an auth-enforcing edge in front of the existing stack. These have no Socket.IO dependency and clear, tested behavior.

If this branch were to move forward, the work breaks into a clear, ordered backlog. The architecture was built so that each item is additive — a stub implemented against an existing interface, or an operational step — rather than a structural change.

1. Close the Socket.IO gap

Build the client-side adapter described in Section 6: a thin TypeScript layer that presents the existing Socket.IO event API to the React components while speaking the new JSON-envelope protocol underneath. This unblocks cutting over any single websocket prefix and is the highest-leverage next step.

2. Fill the scaffolded surfaces

- Port the remaining eight rmhbox minigames against the existing `Minigame` interface.
- Register the other gamehub games onto the generic `RelayGroup`.
- Complete the RMHBot tool set (typecheck, PR creation, code search) behind the existing tool-loop.

3. Realize the scaling path

Enable the Redis backplane in production, add sticky-session ingress, and load-test to establish connection-count and latency baselines per service. Then wire HorizontalPodAutoscalers — completing PR #121's Stage 2 for the Go tier.

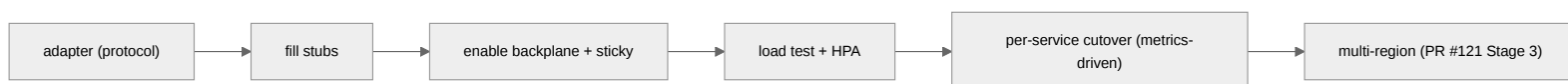


Figure 25.1 — Ordered backlog from reference implementation to production adoption.

4. Complete observability

The instrumentation exists (Section 17); the remaining work is operational: Prometheus scrape config for the new `/metrics` endpoints, dashboards for connections/messages/job-runs, and alerts on the auth-error and bus-publish signals already emitted.

5. Quality hardening

- Restore DeepSeek streaming in the discord-bot for chat UX parity.
- Visual-diff the vibe-worker thumbnails against the `sharp` output and tune if needed.
- Add contract tests that assert the Go event payloads match what the React clients expect, byte for byte.
- Add per-render and per-job tracing spans (OpenTelemetry) on top of the existing metrics.

6. Decide, with data

Finally — and most importantly — the adoption decision should be made against real metrics, per service, exactly as the Stage-4 condition states. The low-risk services (doctrine-worker, recap, gateway) could be adopted first as a trial; the realtime services follow only if connection volume justifies the investment and after the protocol adapter exists.

The through-line: nothing in this backlog requires re-architecting what was built. The interfaces, the backplane, the chart, and the verification harness are in place; the future work is to fill them in, measure, and decide.

A complete file inventory of the `go-services/` module plus the deploy artifacts that ship it. Everything here is enumerated directly from the tree: 85 Go source files across 28 packages, organized as a small shared `pkg/` toolkit, nine `cmd/` entrypoints, one `internal/<svc>` implementation package per service, an end-to-end suite, and a Helm chart + deploy script that reuse PR #121's conventions. Line counts are approximate (from `wc -l`) and round to the source as committed.

85 .GO FILES	~18.5k LOC (GO)	28 PACKAGES	13 _TEST.GO FILES	9 CMD ENTRYPOINTS
-----------------	--------------------	----------------	----------------------	----------------------

Shared toolkit — `go-services/pkg/`

The cross-service library every binary links against: config, DB access, auth, the realtime WebSocket framework, the Redis backplane, HTTP plumbing, logging, rate limiting and metrics. This is where the "we don't need the JS libraries" decisions live (notably `pkg/auth` validating Better Auth sessions straight against Postgres, and `pkg/realtime` replacing Socket.IO).

File	~LOC	Purpose
<code>pkg/auth/auth.go</code>	110	Validates Better Auth sessions directly against Postgres into trusted identity.
<code>pkg/config/config.go</code>	106	Env-driven config: <code>LoadCommon</code> plus typed <code>Get{String,Int,Bool,Duration,CSV}</code> helpers.
<code>pkg/db/db.go</code>	78	Shared pgxpool access layer; <code>Open</code> / <code>WaitForReachable</code> for workers.
<code>pkg/db/models.go</code>	163	Typed models for the subset of the Prisma schema the Go fleet reads/writes.
<code>pkg/events/events.go</code>	152	Cross-instance pub/sub backplane (Redis adapter equivalent) for horizontal scaling.
<code>pkg/httpx/httpx.go</code>	116	Shared HTTP building blocks: <code>/health</code> handler, graceful server, signal wait.
<code>pkg/log/log.go</code>	66	slog wrapper reproducing the Node logger's JSON shape (level/service/timestamp).
<code>pkg/ratelimit/ratelimit.go</code>	120	In-memory sliding-window limiter keyed by (connection, event), with GC.
<code>pkg/ratelimit/ratelimit_test.go</code>	56	Sliding-window limiter tests.
<code>pkg/realtime/hub.go</code>	272	WebSocket hub: connection accept, origin/auth gate, handler dispatch, rooms.
<code>pkg/realtime/conn.go</code>	190	Per-connection read/write pumps and lifecycle.
<code>pkg/realtime/room.go</code>	97	Room membership and sequence-numbered broadcast (<code>BroadcastSeq</code>).
<code>pkg/realtime/grace.go</code>	60	Disconnect grace period for transient drops / reconnects.
<code>pkg/realtime/protocol.go</code>	76	The JSON <code>Envelope{event,payload,seq,ts}</code> wire frame and codec.
<code>pkg/realtime/realtime_test.go</code>	93	Hub/envelope protocol tests.
<code>pkg/telemetry/telemetry.go</code>	55	Prometheus registry + standard instruments and <code>/metrics</code> handler.

Entrypoints — `go-services/cmd/`

One `main.go` per binary. Realtime services wire a `realtime.Hub` and serve a WS path; workers expose only `/health` + `/metrics` on `METRICS_ADDR`.

File	~LOC	Purpose	rmhstudios-go
cmd/gateway/main.go	65	Edge/BFF on PORT_WEB (7005): auth middleware + reverse proxy + static.	
cmd/gamehub/main.go	78	Unified games socket server, WS <code>/socket/</code> on SOCKET_PORT (7001), soft auth.	
cmd/rmhmusic/main.go	79	Synced music rooms, WS <code>/rmhmusic-ws/</code> (7002), auth required, DB mandatory.	
cmd/rmhtube/main.go	90	Watch-party server, WS <code>/rmhtube-ws/</code> (7003); restores rooms from DB on boot.	
cmd/rmhbox/main.go	81	Jackbox-style party server, WS <code>/rmhbox-ws/</code> (7676); DB optional.	
cmd/recap/main.go	95	Lights Out daily-recap scheduler + Discord presence on RECAP_PORT (7004).	
cmd/doctrine-worker/main.go	55	Doctrine Engine scheduler worker (no client HTTP).	
cmd/vibe-worker/main.go	76	Renders vibe-page thumbnails via headless Chromium (no client HTTP).	
cmd/discord-bot/main.go	90	Standalone Discord gateway bot (<code>/chat</code> , <code>/rmhbot</code>); no client HTTP.	

Service implementations — `go-services/internal/`

`internal/gateway/`

File	~LOC	Purpose
gateway/proxy.go	196	Longest-prefix router + WS-passthrough reverse proxy; upstream config.
gateway/auth_middleware.go	75	Validates the session and injects trusted identity headers for upstreams.
gateway/gateway_test.go	160	Routing/prefix-selection tests.

`internal/gamehub/`

File	~LOC	Purpose
gamehub/manager.go	106	Hub registration and per-game wiring; RelayGroup TODO registrations.
gamehub/relay.go	247	Generic host-authoritative 1v1 relay framework + connRegistry.
gamehub/kk.go	334	Kowloon Knockout: faithful standalone port with rematch handshake.
gamehub/kk_test.go	232	Kowloon Knockout relay/handshake tests.

File	~LOC	Purpose
rmhmusic/manager.go	166	Service manager: hub handler registration and lifecycle.
rmhmusic/room.go	540	Room model: membership, host control, snapshots, seq mutations.
rmhmusic/queue.go	133	Shared queue add/remove/reorder.
rmhmusic/sync.go	183	Playback sync (play/pause/seek) and heartbeat.
rmhmusic/chat.go	47	Room chat handling.
rmhmusic/events.go	67	Event-name constants (rmhmusic:*) + action types and tunables.
rmhmusic/repo.go	59	Postgres repo for the room row.
rmhmusic/types.go	104	Payload/state structs.
rmhmusic/util.go	55	Room-code generation and helpers.
rmhmusic/rmhmusic_test.go	150	Room/queue/sync tests.

internal/rmhtube/

File	~LOC	Purpose
rmhtube/manager.go	231	Manager: registration, lifecycle, DB room restore.
rmhtube/room.go	1038	Room core: membership, roles, bans, settings, invites, leadership.
rmhtube/queue.go	453	Media queue: add/remove/reorder/play/skip/vote/shuffle.
rmhtube/sync.go	189	Video sync: host state, play/pause/seek, speed.
rmhtube/snapshot.go	189	Room state snapshot assembly.
rmhtube/chat.go	223	Chat, typing indicators, reactions, pins.
rmhtube/events.go	93	Event-name constants (rmhtube:*) C2S/S2C maps.
rmhtube/repo.go	375	Postgres repo (pgx) + NopRepo for in-memory mode.
rmhtube/types.go	131	Payload/state structs.
rmhtube/utils.go	99	Helpers (codes, validation).
rmhtube/rmhtube_test.go	228	Room/queue/sync tests.

File	~LOC	Purpose
rmhbox/coordinator.go	980	Game coordinator: settings, preload, countdown, rounds, snapshots, results.
rmhbox/lobby.go	789	Lobby core: membership, host, snapshots, disband, game action relay.
rmhbox/minigames.go	735	Concrete minigame implementations.
rmhbox/lobby_ops.go	535	Lobby operations: kick, browse, session results.
rmhbox/manager.go	405	Manager: all rmhbox:* handler registration + leaderboard fetch.
rmhbox/minigame.go	348	Minigame interface and shared game-action plumbing.
rmhbox/repo.go	305	Postgres persistence (match results, leaderboard).
rmhbox/types.go	213	Lobby/player/spectator/game structs.
rmhbox/statesync.go	207	Lobby state snapshot synchronization.
rmhbox/registry.go	66	Static minigame metadata registry.
rmhbox/rmhbox_test.go	404	Lobby/coordinator/minigame tests.

internal/recap/, internal/doctrine/, internal/vibeworker/, internal/discordbot/

File	~LOC	Purpose
recap/runner.go	300	Scheduler loop: posts due daily recaps to Discord every 5 min.
recap/lightsout.go	430	Lights Out puzzle helpers (seed/shapes) for the recap embed.
recap/recap.go	176	Recap construction / Discord message assembly.
recap/recap_test.go	245	Recap/runner tests.
doctrine/worker.go	429	Doctrine scheduler: puzzle gen, Sahur activation, reputation decay.
doctrine/seed.go	115	Deterministic seeding for daily puzzle generation.
doctrine/schedule.go	40	Tick/schedule helpers.
doctrine/seed_test.go	103	Seed determinism tests.
vibeworker/worker.go	192	Poll loop for thumbnailStale vibe pages with optimistic concurrency.
vibeworker/capture.go	149	chromedp headless-Chromium HTML render + downscale to PNG.
vibeworker/repo.go	88	Postgres repo for vibe_page rows.
vibeworker/worker_test.go	165	Worker poll/concurrency tests.
discordbot/rmhbot.go	853	SCAFFOLDED /rmhbot agentic website editor (git-ops, PR creation).
discordbot/chat.go	367	/chat AI persona ("Alex Wu") with DB-backed session persistence.
discordbot/bot.go	288	Discord gateway session lifecycle and command registration.
discordbot/deepseek.go	192	DeepSeek chat-completions client.
discordbot/interactions.go	98	Slash-command interaction routing.
discordbot/discordbot_test.go	217	Bot/chat tests.

Honest scope: discordbot/rmhbot.go is the acknowledged partial-scope piece — the agentic editor's git/JS-toolchain steps are stubbed and depend on a real GITHUB_TOKEN . In gamehub , only Kowloon Knockout is ported verbatim; the remaining games are left as compiling RelayGroup TODO registrations in manager.go .

End-to-end suite — go-services/e2e/

Build-tagged (//go:build e2e) tests that dial the real service binaries over WebSockets and assert actual protocol flows.

File	~LOC	Purpose
e2e/harness.go	173	Spawns binaries, manages a test Postgres, dials WS clients.
e2e/gamehub_e2e_test.go	103	Live Kowloon Knockout create/join/relay flow.
e2e/auth_rmhtube_e2e_test.go	158	Gateway auth + rmhtube room flow end to end.

Module & build files — go-services/ root

File	Purpose
go.mod / go.sum	Module github.com/rmhstudios/rmh-go and pinned dependencies.
Dockerfile	Multi-stage build; --build-arg SERVICE selects the binary, EXTRA_PKGS adds runtime deps.
.dockerignore	Build-context exclusions.
docker-compose.go.yml	Local fleet compose for the Go services.
Makefile	Build/test/lint targets.
README.md / FOUNDATION.md	Module overview and the foundational design/skeleton doc.
scripts/e2e/run.sh	Runs the build-tagged e2e suite.
scripts/e2e/schema.sql	Minimal schema for the e2e test database.

Deploy — deploy/ (repo root)

The Helm chart deploy/helm/rmhstudios-go (Chart 0.1.0 , appVersion 1.0.0) deliberately mirrors PR #121's rmhstudios chart: a generic Deployment over a services map, Traefik ingress, and the reused rmhstudios-secrets Secret. Its one departure is one image per service (each Go binary is a separate static image).

File	Purpose
deploy/deploy-go.sh	Build one image per service, import to k3s or push to registry, then <code>helm upgrade --install --atomic --wait</code> .
deploy/helm/rmhstudios-go/Chart.yaml	Chart metadata (Stage-4 Go fleet alongside the SSR web app).
deploy/helm/rmhstudios-go/values.yaml	Defaults: per-service ports/paths/replicas, redis, secret name, PVC.
deploy/helm/rmhstudios-go/values-prod.yaml	Prod overrides: ingress on, scaled realtime replicas, image SHA placeholder.
deploy/helm/rmhstudios-go/templates/deployment.yaml	Generic Deployment rendered per service from the services map.
deploy/helm/rmhstudios-go/templates/service.yaml	ClusterIP Service per non-worker entry.
deploy/helm/rmhstudios-go/templates/ingress.yaml	Traefik ingress routing path prefixes to services.
deploy/helm/rmhstudios-go/templates/redis.yaml	Redis backplane Deployment + Service (pkg/events).
deploy/helm/rmhstudios-go/templates/_helpers.tpl	Template helpers (names, labels, image ref).
deploy/helm/rmhstudios-go/templates/NOTES.txt	Post-install notes.
deploy/helm/rmhstudios-go/.helmignore	Packaging exclusions.

The complete configuration surface of the Go fleet: every environment variable any binary reads, the client/metrics ports each service binds, the WebSocket path prefixes the gateway routes, and a note on the single reused production Secret. All names and defaults are taken from `pkg/config/config.go`, each `cmd/*/main.go`, and `internal/gateway/proxy.go`. The fleet is intentionally a drop-in for the legacy Node services: it reads the SAME `.env.production` the Helm chart from PR #121 already mounts (DATABASE_URL, the PORT_* family, DISCORD_*, DEEPSEEK_*, BETTER_AUTH_*).

Shared / common variables

Read by `config.LoadCommon` for every service. `DATABASE_URL` is required (the call errors without it); `REDIS_URL` is optional and, when set, enables the `pkg/events` backplane so realtime services can run more than one replica.

Variable	Services	Purpose	Default
DATABASE_URL	all	Single Postgres DSN for the whole platform. Required (LoadCommon errors if empty).	— (required)
REDIS_URL	realtime (gamehub, rmhbox, rmhtube, rmhmusic)	Cross-instance pub/sub backplane; empty => single-instance in-memory.	"" (disabled)
NODE_ENV	all	Environment label (development/staging/production).	production
SERVER_DB_POOL_SIZE	all	pgx pool size (mirrors the Node default).	5
METRICS_ADDR	all (workers bind here)	host:port for the Prometheus <code>/metrics</code> + <code>/health</code> server.	:9090
LOG_LEVEL	all	slog level: debug / info / warn / error.	info

Gateway

The edge/BFF binds `PORT_WEB` and proxies to upstreams resolved from env (defaults mirror the Compose/Helm service names). When `STATIC_DIR` points at a real directory, built assets are served directly before falling through to the web upstream.

Variable	Service	Purpose	Default
PORT_WEB	gateway	Client-facing port for the edge.	7005
WEB_UPSTREAM	gateway	Catch-all upstream: the React SSR/Nitro app.	http://web:3000
GAMEHUB_UPSTREAM	gateway	Upstream for the <code>/socket/</code> prefix.	http://gamehub:7001
RMHBOX_UPSTREAM	gateway	Upstream for <code>/rmhbox-ws/</code> .	http://rmhbox:7676
RMHTUBE_UPSTREAM	gateway	Upstream for <code>/rmhtube-ws/</code> .	http://rmhtube:7003
RMHMUSIC_UPSTREAM	gateway	Upstream for <code>/rmhmusic-ws/</code> .	http://rmhmusic:7002
STATIC_DIR	gateway	Optional dir of built assets served locally before the web upstream.	"" (off)

Realtime services

Each realtime binary reads its own PORT_* and an optional CORS allow-list. `SOCKET_CORS_ORIGIN` is comma-separated; empty means allow all origins.

Variable	Service(s)	Purpose	Default rmhstudios-go
SOCKET_PORT	gamehub	WS port for the unified games server.	7001
RMHMUSIC_PORT	rmhmusic	WS port for synced music rooms.	7002
RMHTUBE_PORT	rmhtube	WS port for watch-party rooms.	7003
RMHBOX_PORT	rmhbox	WS port for party-game lobbies.	7676
SOCKET_CORS_ORIGIN	gamehub, rmhmusic, rmhtube, rmhbox	CSV allow-list of WS origins; empty => allow all.	"" (allow all)

rmhmusic sets `RequireAuth: true` and opens the DB unconditionally (session validation is mandatory). gamehub, rmhtube and rmhbox use soft auth (`RequireAuth: false`) and degrade to anonymous / in-memory operation when the DB is unavailable.

Recap service

Ports the env contract of `server/recap/index.ts` , including the fallback chains for the app ID and site URL.

Variable	Purpose	Default
RECAP_PORT	Client-facing port for <code>/health</code> + <code>/metrics</code> .	7004
DISCORD_ACTIVITY_BOT_TOKEN	Bot token for the recap presence + posting; empty disables posting.	""
VITE_DISCORD_ACTIVITY_CLIENT_ID	Discord application ID.	(falls back below)
DISCORD_ACTIVITY_CLIENT_ID	Fallback application ID when the VITE_ form is unset.	""
SITE_URL	Public site URL used in recap embeds.	(falls back below)
VITE_BETTER_AUTH_URL	Fallback site URL when SITE_URL is unset.	https://rmhstudios.com

Discord bot

No client HTTP — health/metrics bind `METRICS_ADDR` . The token falls back from `DISCORD_BOT_TOKEN` to `DISCORD_ACTIVITY_BOT_TOKEN` ; one of them is required.

Variable	Purpose	Default
DISCORD_BOT_TOKEN	Primary gateway bot token (required, or the ACTIVITY token below).	""
DISCORD_ACTIVITY_BOT_TOKEN	Fallback bot token.	""
DISCORD_DEV_GUILD_ID	Guild for fast dev-command registration.	""
OWNER_ID	Owner Discord user id (gates owner-only commands).	""
DEEPSEEK_API_KEY	API key for the /chat persona's DeepSeek client.	""
DEEPSEEK_MODEL	DeepSeek model name.	deepseek-chat
RMHBOT_WORKTREES_DIR	Working dir for the /rmhbot agentic editor's git worktrees.	""
GITHUB_TOKEN	Token for /rmhbot PR creation (scaffolded path).	""
GITHUB_REPO	"owner/name" target repo for /rmhbot; falls back to a built-in default.	"" (built-in default)

Variable	Purpose	Default
PLAYWRIGHT_CHROMIUM_EXECUTABLE_PATH	Path to the Chromium/Chrome binary for chromedp; empty => chromedp auto-discovery.	""

The vibe worker writes thumbnails to `<cwd>/db/vibe-thumbs` (the shared volume the web app serves at `/api/vibe/thumb/{slug}`); the path is derived at runtime, not from an env var.

Service ports & WebSocket paths

Client ports double as the host for `/health` and `/metrics` . Workers expose no client port — their probes hit `/health` on the metrics port (default `:9090`). WS paths are served as-is by each hub and proxied through unchanged by the gateway (no prefix stripping).

Service	Port	WS path	Helm replicas (default / prod)
gateway	7005	— (edge; routes the prefixes below + catch-all <code>/</code>)	2 / 3
gamehub	7001	<code>/socket/</code>	2 / 3
rmhmusic	7002	<code>/rmhmusic-ws/</code>	1 / 1
rmhtube	7003	<code>/rmhtube-ws/</code>	1 / 2
recap	7004	— (scheduler; health/metrics only)	1 / 1
rmhbox	7676	<code>/rmhbox-ws/</code>	1 / 2
doctrine-worker	9090 (metrics)	— (worker, no client port)	1 / 1
vibe-worker	9090 (metrics)	— (worker, no client port)	1 / 1
discord-bot	9090 (metrics)	— (worker, no client port)	1 / 1

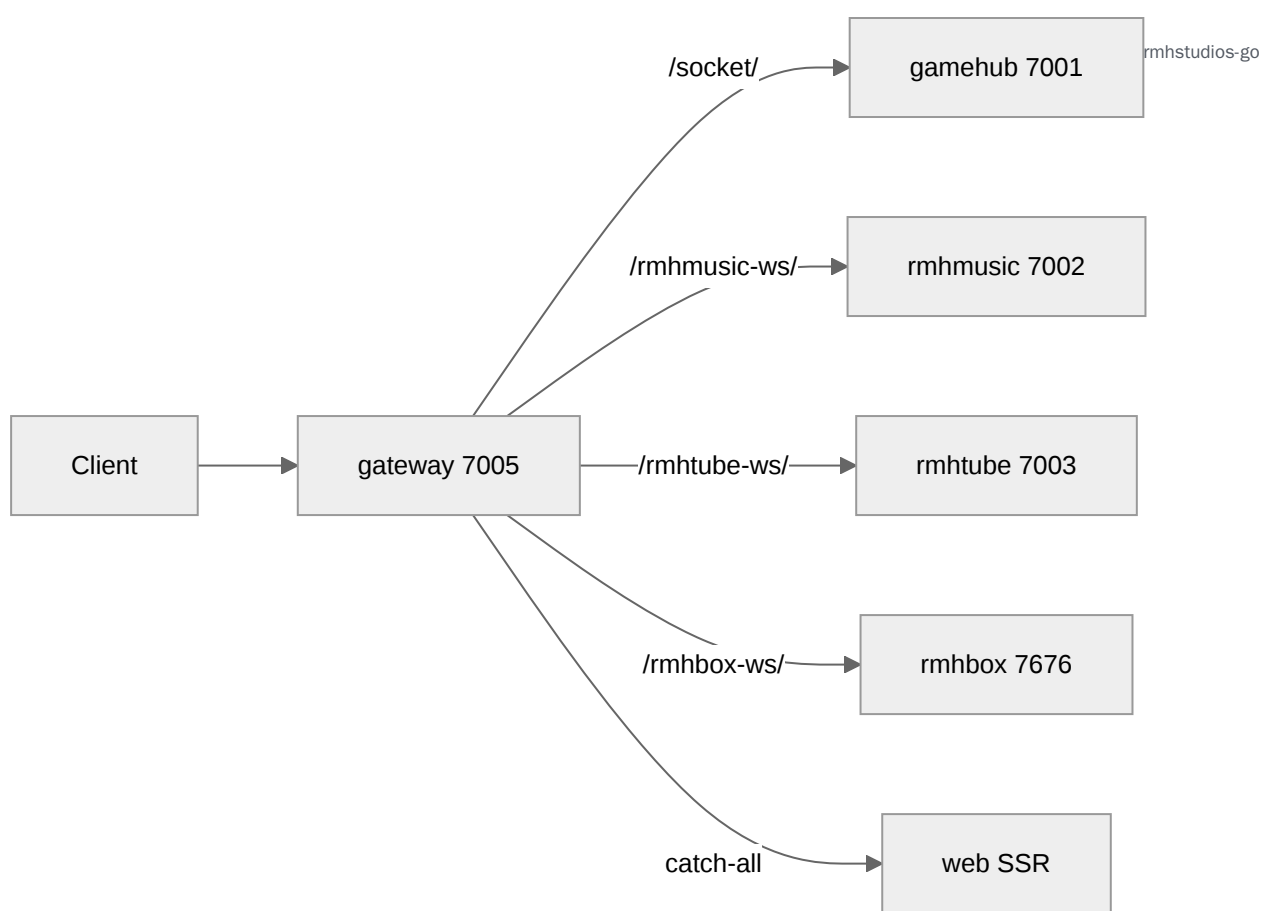


Figure: gateway path-prefix routing to the realtime services.

The reused `.env.production` Secret

The chart does not define its own credentials. The `env.secretName` value defaults to `rmhstudios-secrets` — the exact Kubernetes Secret PR #121's deploy flow already creates out-of-band with `kubectl create secret --from-env-file .env.production`. The Go fleet reads the SAME variables (`DATABASE_URL`, `DISCORD_*`, `DEEPSEEK_*`, `BETTER_AUTH_*`), so no new secret management is introduced.

```
# deploy/helm/rmhstudios-go/values.yaml
env:
  secretName: rmhstudios-secrets # created by PR #121 from .env.production
```

`deploy/deploy-go.sh` is parallel to PR #121's `deploy-k8s.sh` and deliberately reuses the same Secret and the same single-node-vs-registry switch; it expects the `rmhstudios` (SSR web) chart to be up first.

The realtime event catalog for the four WebSocket services. Every name below is taken from the actual source — the `events.go` constant maps (`rmhmusic` , `rmhtube`), the `kk*` constants in `internal/gamehub/kk.go` , and the literal event strings in `internal/rmhbox/*.go` . The names carry their service prefix (`kk:` , `rmhmusic:` , `rmhtube:` , `rmhbox:`) and are kept byte-for-byte identical to the legacy TypeScript C2S/S2C maps so the existing client adapters map cleanly onto the new transport.

The common envelope

Every frame in both directions is one JSON object — the `realtime.Envelope` defined in `pkg/realtime/protocol.go` . This replaces the Socket.IO v4 wire protocol while preserving its semantics: named events plus the sequence-numbered mutation broadcasts the Node delta protocol relied on.

```
{ "event": "rmhtube:room:join", "payload": { ... }, "seq": 42, "ts": 1718700000000 }
```

Field	Type	Meaning
event	string	The prefixed event name (the rows below).
payload	JSON (omitempty)	Event-specific body; absent for e.g. heartbeats.
seq	uint64 (omitempty)	Server-assigned, monotonic per room for mutation broadcasts (ordering guarantee).
ts	int64 (omitempty)	Server send time in epoch milliseconds.

The `*:room:action` / `rmhbox:game:action` broadcasts are the seq-numbered "delta" frames: their payload carries a `type` discriminator (e.g. `MEMBER_JOINED` , `CHAT_MESSAGE` , `NOW_PLAYING`) plus the mutation data, reproducing the legacy delta protocol.

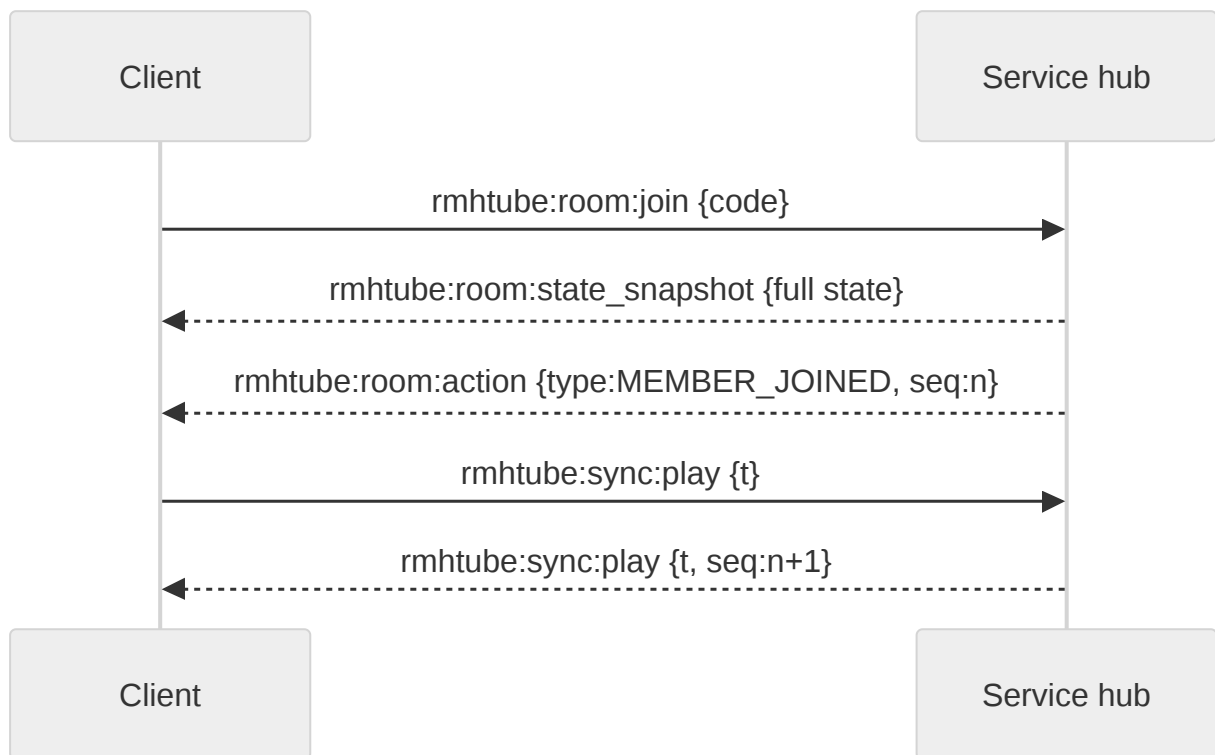


Figure: typical join + sync exchange (snapshot then seq deltas).

A 1v1 host-authoritative relay: host creates a room, a guest joins, then input (guest → host) and game_state (host → guest) frames are relayed; a fighter_ready handshake from both seats re-arms a rematch. Defined in `internal/gamehub/kk.go` .

Client → Server

Event	Meaning
kk:create_room	Host creates a room with its chosen fighter class.
kk:join_room	Guest joins by room code with a fighter class.
kk:input	Guest input frame relayed to the host (authoritative sim).
kk:game_state	Host's authoritative game-state frame relayed to the guest.
kk:fighter_ready	Seat signals ready for the (re)match handshake.
kk:leave	Leave the room.

Server → Client

Event	Meaning
kk:room_created	Acknowledges room creation; carries the code.
kk:room_joined	Acknowledges a successful join.
kk:opponent_ready	The other seat is ready (rematch handshake progress).
kk:opponent_disconnected	The peer dropped.
kk:error	Error (bad code, room full, etc.).

Only Kowloon Knockout is ported verbatim. The remaining games (Slice It, Neon Driftway, ...) are left as compiling RelayGroup TODO registrations and emit no events yet.

rmhmusic (`rmhmusic:`)

Synchronized group music rooms: the host controls playback, members share a queue and chat. Names from `internal/rmhmusic/events.go` .

Event	Meaning
rmhmusic:room:create	Create a room.
rmhmusic:room:join	Join a room by code.
rmhmusic:room:leave	Leave the current room.
rmhmusic:room:browse	Browse/list public rooms.
rmhmusic:room:chat	Send a chat message.
rmhmusic:room:transfer_host	Hand host control to another member.
rmhmusic:music:play	Host: start/resume playback.
rmhmusic:music:pause	Host: pause playback.
rmhmusic:music:seek	Host: seek to a position.
rmhmusic:music:skip	Host: skip to the next track.
rmhmusic:queue:add	Add a track to the shared queue.
rmhmusic:queue:remove	Remove a queued track.
rmhmusic:queue:reorder	Reorder the queue.

Server → Client

Event	Meaning
rmhmusic:room:created	Room created; carries code/initial state.
rmhmusic:room:state_snapshot	Full room state on join/resync.
rmhmusic:room:action	Seq-numbered delta (MEMBER_JOINED, MEMBER_LEFT, MEMBER_DISCONNECTED, HOST_TRANSFERRED, CHAT_MESSAGE, NOW_PLAYING, QUEUE_ITEM_ADDED).
rmhmusic:room:browse_result	Result of a browse request.
rmhmusic:room:disbanded	Room closed.
rmhmusic:room:not_in_room	Action rejected: caller is not in a room.
rmhmusic:music:play	Broadcast play to members.
rmhmusic:music:pause	Broadcast pause.
rmhmusic:music:seek	Broadcast seek position.
rmhmusic:music:track_changed	The now-playing track changed.
rmhmusic:sync:heartbeat	Periodic playback-position heartbeat for drift correction.
rmhmusic:queue:updated	The shared queue changed.
rmhmusic:error	Error response.

rmhtube (rmhtube:)

Watch-party video rooms with roles, bans, invites, a shared queue, synced playback (incl. speed), reactions and chat. Names from [internal/rmhtube/events.go](#) ; the maps are organized in phased feature groups.

Event	Meaning
rmhtube:room:create	Create a room.
rmhtube:room:join	Join a room.
rmhtube:room:leave	Leave the room.
rmhtube:room:kick	Host: kick a member.
rmhtube:room:transfer_host	Hand host to another member.
rmhtube:room:update_settings	Update room settings.
rmhtube:room:browse	Browse public rooms.
rmhtube:room:chat	Send a chat message.
rmhtube:room:set_leader	Assign a co-leader/leader role.
rmhtube:room:ban	Ban a user from the room.
rmhtube:room:unban	Lift a ban.
rmhtube:room:create_invite	Create an invite link.
rmhtube:room:set_status	Set the member's status/presence.
rmhtube:room:check_history	Query watch-history status.
rmhtube:sync:host_state	Host pushes authoritative playback state.
rmhtube:sync:play	Host: play.
rmhtube:sync:pause	Host: pause.
rmhtube:sync:seek	Host: seek.
rmhtube:sync:set_speed	Host: set playback speed.
rmhtube:queue:add	Add a media item to the queue.
rmhtube:queue:remove	Remove a queued item.
rmhtube:queue:reorder	Reorder the queue.
rmhtube:queue:play_item	Jump to and play a specific item.
rmhtube:queue:skip	Skip the current item.
rmhtube:queue:vot skip	Cast a vote to skip.
rmhtube:queue:vot	Vote on a queued item.
rmhtube:queue:shuffle	Shuffle the queue.
rmhtube:reaction:send	Send an emoji reaction.
rmhtube:chat:typing	Signal typing.
rmhtube:chat:react	React to a chat message.
rmhtube:chat:pin	Pin a chat message.

Event	Meaning
rmhtube:room:created	Room created; carries code.
rmhtube:room:state_snapshot	Full room state on join/resync.
rmhtube:room:action	Seq-numbered room delta broadcast.
rmhtube:room:browse_result	Browse result.
rmhtube:room:kicked	You were kicked.
rmhtube:room:disbanded	Room closed.
rmhtube:room:not_in_room	Rejected: caller not in a room.
rmhtube:room:invite_created	An invite link was created.
rmhtube:room:history_status	Watch-history status response.
rmhtube:sync:state	Authoritative playback state broadcast.
rmhtube:sync:play	Broadcast play.
rmhtube:sync:pause	Broadcast pause.
rmhtube:sync:seek	Broadcast seek.
rmhtube:sync:media_changed	The active media item changed.
rmhtube:sync:speed_changed	Playback speed changed.
rmhtube:queue:updated	The queue changed.
rmhtube:reaction:broadcast	Broadcast a reaction to the room.
rmhtube:chat:typing_indicator	Someone is typing.
rmhtube:error	Error response.

rmhbox (rmhbox:)

Jackbox-style party lobbies with a host, players, spectators, a minigame coordinator and a leaderboard. These names are literal strings registered in `internal/rmhbox/manager.go` and emitted across `lobby.go` , `lobby_ops.go` , `coordinator.go` , `minigames.go` and `statesync.go` .

Event	Meaning
rmhbox:lobby:create	Create a lobby.
rmhbox:lobby:join	Join a lobby by code.
rmhbox:lobby:leave	Leave the lobby.
rmhbox:lobby:kick	Host: kick a participant.
rmhbox:lobby:transfer_host	Hand host to another participant.
rmhbox:lobby:update_settings	Update lobby settings.
rmhbox:lobby:end_session	Host: end the game session.
rmhbox:lobby:toggle_ready	Toggle the player's ready state.
rmhbox:lobby:request_promotion	Spectator requests promotion to player.
rmhbox:lobby:promote_spectator	Host promotes a spectator to player.
rmhbox:lobby:browse	Browse public lobbies.
rmhbox:lobby:chat	Send a lobby chat message.
rmhbox:game:pick	Pick a minigame.
rmhbox:game:select	Select/confirm a game choice.
rmhbox:game:force_skip	Host: force-skip the current game.
rmhbox:game:force_end	Host: force-end the current game.
rmhbox:game:pause_timer	Host: pause the game timer.
rmhbox:game:ready_to_render	Client signals it is ready to render.
rmhbox:game:confirm_settings	Confirm game settings.
rmhbox:game:update_settings	Update game settings.
rmhbox:game:input	In-game player input.
rmhbox:player:away	Mark the player away.
rmhbox:player:active	Mark the player active.
rmhbox:spectator:select_player	Spectator follows a specific player.
rmhbox:leaderboard:fetch	Request leaderboard data.

Event	Meaning
rmhbox:lobby:created	Lobby created; carries code.
rmhbox:lobby:state_snapshot	Full lobby state on join/resync.
rmhbox:lobby:browse_result	Browse result.
rmhbox:lobby:kicked	You were kicked.
rmhbox:lobby:disbanded	Lobby closed.
rmhbox:game:settings_opened	Game settings phase opened.
rmhbox:game:settings_updated	Game settings changed.
rmhbox:game:instructions	Per-game instructions for the round.
rmhbox:game:preload_start	Asset preload phase started.
rmhbox:game:preload_progress	Preload progress update.
rmhbox:game:countdown	Pre-round countdown.
rmhbox:game:started	The round started.
rmhbox:game:state_snapshot	Full in-game state snapshot.
rmhbox:game:action	Seq-numbered in-game delta broadcast.
rmhbox:game:round_results	Results for the completed round.
rmhbox:game:session_results	Final results for the whole session.
rmhbox:leaderboard:data	Leaderboard data response.
rmhbox:error	Error response.

As with the other services, `rmhbox:lobby:state_snapshot` / `rmhbox:game:state_snapshot` deliver full state on join or resync, while `rmhbox:game:action` carries the seq-numbered incremental deltas during play.

Terms and components referenced throughout this report.

Glossary

Term	Meaning
Better Auth	The JavaScript auth library that issues and signs rmhstudios sessions. Kept in JS; Go only validates its <code>session</code> table.
backplane	The Redis pub/sub layer (<code>pkg/events</code>) that fans realtime broadcasts across service replicas.
BFF	Backend-for-frontend; here, the gateway that fronts the fleet and the SSR app.
chromedp	Go library driving headless Chromium via the DevTools protocol; used by vibe-worker.
cuid / uuid	Opaque TEXT primary-key formats. Prisma used cuid; Go generates uuid.
DeepSeek	The OpenAI-compatible LLM API powering the discord-bot's <code>/chat</code> .
discordgo	Go Discord client used by recap (presence + REST) and discord-bot (gateway + commands).
Envelope	The realtime frame: <code>{event, payload, seq, ts}</code> .
FSM	Finite state machine; the rmhbox lobby/coordinator lifecycle.
gateway	The edge service: reverse proxy + auth trust boundary + static serving.
grace timer	A keyed, cancellable delayed action (e.g. disconnect grace, empty-room GC).
Helm	Kubernetes package manager; the chart <code>rmhstudios-go</code> renders the fleet.
Hub	The per-service websocket runtime in <code>pkg/realtime</code> .
k3s	Lightweight Kubernetes; the cluster PR #121 introduced.
Kowloon Knockout	A 1v1 fighting game; the fully-ported gamehub relay.
kubeconform	A Kubernetes manifest schema validator; used to validate the rendered chart.
leader-authoritative sync	rmhtube model where the room leader's reported video state is the source of truth.
mulberry32	A small deterministic PRNG; ported bit-exact for doctrine puzzle seeding.
optimistic concurrency	Write-if-unchanged (compare <code>updatedAt</code>); used by vibe-worker.
pgx / pgxpool	The Go PostgreSQL driver and connection pool used fleet-wide.
PR #121	The infrastructure PR (Compose → k3s/Helm/Terraform) this work builds on and reconciles with.
RMHBot	The discord-bot's agentic website editor (<code>/rmhbot</code>); scaffolded.
snapshot+delta	Protocol pattern: full state on join, then seq-numbered incremental mutations.
soft auth	Anonymous connections allowed; identity attached if a valid token is present.
SSR	Server-side rendering; the React/TanStack Start web tier, kept in JS.
sticky sessions	Ingress pinning a client to one replica; required for multi-replica realtime.
trust boundary	The single place (gateway) where identity headers are asserted for upstreams.
Traefik	The ingress controller bundled with k3s.

